

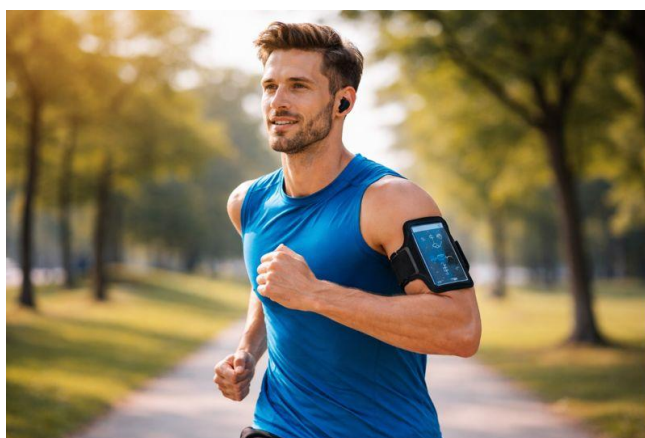


ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

אפליקציית מעקב ריצה



שם הסטודנט: שליו אבקסיס

ת"ז: 330589664

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 20/05/2026

שם המכללה: כנפי רוח קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

3	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
4	2. מבוא
4	2.1 הרקע לפרויקט
5	2.2 תהליך המחקר
6	2.3 רקע תיאורטי וסקירת ספרות מקצועית
6	2.4 אתגרים מרכזיים במחקר
9	3. מטרות ויעדים לפיתוח המערכת
10	4. אתגרים בבניית המערכת
11	5. הגדרת מדדי הצלחה למערכת
11	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
11	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
15	8. אפיון המערכת המוצעת
18	8.1 ניתוח הדרישות מהמערכת
18	8.2 מודלי המערכת
21	8.3 אפיון פונקציונלי וביצועים עיקריים
22	8.4 אילוצי המערכת
23	9. תיאור הארכיטקטורה של המערכת
23	9.1 ארכיטקטורת המערכת
23	9.2 מודל שרת לקוח
26	9.3 תהליכים של מע' ההפעלה
26	9.4 תיאור פרוטוקולי תקשורת
27	9.5 ארכיטקטורת רשת
27	9.6 שירותים חיצוניים
29	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
29	10.1 תרשימי תרחיש Use case Diagram
29	10.2 תרשימי רצף Sequence Diagram
37	11. תיאור מסכים וממשק משתמש
37	11.1 תרשימי היררכיית המסכים
37	11.2 תיאור המסכים
50	12. תיאור התוכנה
50	12.1 סביבת הפיתוח של התוכנה
52	12.2 תיאור מבנה הפרויקט והמחלקות
100	12.4 מבני נתונים בהם נעשה שימוש
100	12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
110	13. תיאור מסד הנתונים
113	14. מדריך למשתמש
116	15. בדיקות והערכה
117	16. מסקנות
118	17. פיתוחים עתידיים
119	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

פרויקט זה מתרכז בפיתוח מערכת מתקדמת למעקב, ניתוח וניהול אימוני ריצה, המיועדת לספק מענה מותאם אישית לספורטאים חובבים ומקצועיים כאחד. המערכת מאפשרת מעקב מבוסס מיקום בזמן אמת אחר מדדים פיזיולוגיים ומרחביים שונים, כגון מרחק מצטבר, קצב וזמני ריצה, תוך שימוש בחיישני המכשיר הנייד.

מעבר ליכולות הניטור והתיעוד הסטנדרטיות, הפרויקט מתמודד עם אתגרים טכנולוגיים של עיבוד נתונים דינמי בסביבה ניידת, תוך דגש על שיפור דיוק הנתונים, התגברות על שיבושי קליטה מרחביים, וניהול משאבים יעיל. המידע הנאסף מעובד ומוצג למשתמש באופן ויזואלי וברור, ומאפשר לו לנתח את ביצועיו, לזהות מגמות שיפור לאורך זמן ולבחון את מידת עמידתו ביעדים שהציב לעצמו.

לב החדשנות של הפרויקט טמון בפתרון בעיית ניתוב ומציאת מסלולים אופטימליים תחת אילוצים קשיחים) בעיית ה-Orienteeing-המוגדרת במדעי המחשב כבעיה חישובית קומבינטורית מורכבת ביותר מסוג NP-Hard. המערכת שואפת לקלוט את העדפותיו האישיות של הרץ – דוגמת מרחק מבוקש, תנאי שטח משתנים ופרמטרים גאוגרפיים – ולייצר עבורו מסלול ריצה איכותי ומותאם אישית העונה על אילוצים אלו במלאם. בשל המורכבות החישובית הגבוהה המונעת מציאת פתרון מדויק בזמן אמת, משולב במערכת אלגוריתם אבולוציוני (DPSO - Discrete Particle Swarm Optimization) המותאם למרחבים דיסקרטיים, לצד מנגנוני אופטימיזציה ושיפור מקומיים. שילוב זה מאפשר להשיג איזון מיטבי בין איכות המסלול המופק לבין זמן החישוב, ובכך מספק מענה אפקטיבי ומייד לכוויית המשתמש בזמן אמת.

מבחינה מערכתית, הפרויקט מושתת על תפיסת הנדסת תוכנה מודרנית המפרידה באופן מוחלט בין שכבת ממשק המשתמש, שכבת הלוגיקה העסקית ושכבת ניהול הנתונים. המערכת בנויה במודל לקוח-שרת, המאפשר לאפליקציית המובייל הניידת להתנהל ביעילות מול רכיב שרת מרכזי המופקד על ביצוע החישובים האלגוריתמיים המורכבים, ניתוח המידע הסטטיסטי וניהול תהליכי המערכת. נתוני המשתמשים, המסלולים והאימונים שמורים במסד נתונים מנוהל התומך בזמינות גבוהה ובסנכרון מהיר. מבנה מודולרי זה מבטיח הפרדת סמכויות, קלות בתחזוקה ויכולת הרחבה עתידית, תוך מתן אפשרות לעבודה רציפה ויציבה של האפליקציה גם בתנאי שטח משתנים

תיאור הנושא הפרויקט

הפרויקט יבנה כאפליקציית Android. האפליקציה מיועדת לרצים - חובבים ומתקדמים - המעוניינים לעקוב בצורה נוחה וברורה אחר הריצות שלהם ולבחון את התקדמותם לאורך זמן. האפליקציה מאפשרת למשתמש לתעד ריצות באמצעות GPS, כך שבסיומה של כל ריצה מוצג מסלול הריצה שבוצע בפועל, יחד עם נתונים וסטטיסטיקות המסכמות את הביצועים.

המערכת מנתחת את נתוני הריצות ומציגה אותם בצורה ויזואלית וברורה באמצעות גרפים, טבלאות ואלמנטים גרפיים נוספים, במטרה לאפשר למשתמש להבין את מצבו: האם חל שיפור בביצועים, באיזה היבטים הוא התקדם, ועד כמה הוא קרוב ליעדים שהציב לעצמו. כך יכול המשתמש להסיק מסקנות מעשיות לגבי האימונים שלו ולזהות נקודות לשיפור.

בנוסף למעקב אחר ריצות שבוצעו, האפליקציה מאפשרת למשתמש להזין העדפות למסלול ריצה רצוי, כגון מרחק, הפרש גבהים מקסימלי וקרבה למיקום מסוים. על סמך העדפות אלו, המערכת מציעה למשתמש מסלולי ריצה מתאימים, המדורגים לפי איכות ההתאמה לדרישותיו, ובכך מסייעת לו לבחור מסלול ריצה המתאים ליכולותיו ולמטרותיו.

הגדרת הבעיה האלגוריתמית

הבעיה האלגוריתמית אשר בה המחקר עוסק היא בעיית ה-Orienteeing שהיא בעיה בלתי ניתנת לפתירה בזמן סביר (NP-Hard Problem) במשפחת בעיות המסלול שבה צריך למצוא מסלול מיטבי שמתחיל בנקודה ומסתיים בנקודה (בין אם נקודות שונות ובין אם אותה נקודה), כאשר לאורך המרחב קיימים מספר יעדים (נקודות) אפשריים, ולכל יעד יש ניקוד. המטרה היא לבחור תת-קבוצה של יעדים ולתכנן מסלול שעובר בהם כך שסך ניקוד היעדים יהיה מקסימלי, תחת מגבלות מסוימות שלא ניתן לחצות (מגבלות קשיחות).

2. מבוא

2.1 הרקע לפרויקט

בעשורים האחרונים ניכרת עלייה חסרת תקדים במודעות הציבורית לאורח חיים בריא, כאשר ריצה והליכה חובבנית הפכו לפעילויות הספורט הפופולריות ביותר. עם זאת, מתאמנים רבים נתקלים בשתי בעיות מרכזיות המשפיעות על התמדתם:

- **מונוטוניות ושחיקה:** ריצה חוזרת ונשנית באותם מסלולים קבועים ומוכרים מייצרת תחושת שיעמום ומפחיתה את המוטיבציה של הספורטאי לאורך זמן.
- **קושי בתכנון ידני:** ניסיון של משתמש לגוון את מסלולו ולתכנן באופן ידני נתיב חדש (במיוחד מסלול מעגלי החוזר לנקודת המוצא) שיעמוד בדיוק במרחק היעד שהציב לעצמו, הוא משימה מורכבת ומתישה הדורשת ניסוי וטעייה על גבי מפות דיגיטליות.

הפרויקט הנוכחי נולד מתוך הצורך לגשר על פער זה, ולספק פתרון טכנולוגי מתקדם המשלב אפליקציית מובייל אינטואיטיבית עם שרת לוגיקה חזק. המערכת המוצעת מאפשרת למשתמש להגדיר את אילוציו,

ומפיקה עבורו באופן אוטומטי ומיידי מסלולים חכמים, בטיחותיים, ומותאמים אישית העומדים בדיוק בתקציב המרחק שהוגדר, תוך שילוב נקודות עניין סביבתיות ואיכותיות.

2.2 תהליך המחקר

תהליך המחקר של הפרויקט התנהל כזרם מובנה ומתודולוגי, שהחל מזיהוי הפערים הקיימים בשוק הצרכני ועד לגיבוש המודל הרעיוני המלא של המערכת. בשלב הראשוני, המחקר התרכז במיפוי וניתוח של פתרונות הניווט והכוונת הדרכים הנפוצים כיום בעולם התוכנה. מתוך בחינה זו עלה כי מרבית הכלים המסחריים מתוכננים ומותאמים עבור כלי רכב, וממוקדים באופן בלעדי במציאת המסלול המהיר או הקצר ביותר מנקודת מוצא מוגדרת לנקודת יעד מרוחקת. מחקר שוק זה הבהיר כי המערכות הקיימות אינן נותנות מענה לצורך הפיזי והפסיכולוגי של ספורטאים ורצים חובבים, המבקשים לגוון את מסלוליהם באמצעות נתיבים חווייתיים ומעגליים החוזרים לנקודת המוצא, תוך עמידה קשיחה בתקציב מרחק מוגדר מראש.

עם הבנת הפער המעשי, המחקר עבר לשלבו השני שבו תורגמה הבעיה הפיזית מהעולם האמיתי למונחים מתמטיים ומדעיים מוגדרים בתחום תורת הגרפים והנדסת התוכנה. שלב זה דרש מידול תיאורטי של המרחב הגאוגרפי כרשת מורכבת של צמתים המייצגים נקודות עניין סביבתיות, וקשתות המייצגות את עלויות המעבר ביניהן. הבעיה הוגדרה מבחינה מחקרית כבעיית אופטימיזציה קומבינטורית קשה, שמטרתה למקסם את איכות המסלול והתגמול הסביבתי עבור המשתמש, תחת אילוץ קשיח וסגור של אורך נתיב מקסימלי.

בשלב הבא, לאור העובדה שמדובר באתגר חישובי מורכב שאינו ניתן לפתרון מלא בזמן סביר באמצעות שיטות סריקה סטנדרטיות, התמקד המחקר בבחינה ובהשוואה של גישות מתקדמות מעולם הבינה המלאכותית והחישוב האבולוציוני. נחקרו מודלים שונים של אופטימיזציה מבוזרת ואינטליגנציה קבוצתית, המסוגלים לסרוק ביעילות מרחבי אפשרויות עצומים, להתגבר על מלכודות של פתרונות מדומים (אופטימום מקומי), ולהתכנס במהירות ובזמן אמת לפתרונות איכותיים ומאוזנים המתאימים להרצה על גבי שרת רשת.

בשלב המסכם של המחקר, גושר הפער שבין המודל המתמטי המופשט לבין אילוצי השטח האמיתיים של עולם הריצה. במסגרת זו נחקרו דרכי הפעולה וההצמדה של נקודות לוגיות אל רשתות כבישים ושבילים קיימות במציאות, שילוב של מנגנוני שיפור גאומטריים מקומיים למניעת כשלים מבניים (כמו הצטלבויות דרכים או לולאות מיותרות), ודרכי האינטגרציה הדינמית מול תשתיות מיפוי עולמיות לקבלת עלויות מרחק ונתוני דרך ריאליים. תהליך מחקר רציף זה הבטיח כי המערכת המוצעת לא רק תפתור את הבעיה ברמה האלגוריתמית התאורטית, אלא תספק מסלולים ישימים, בטיחותיים ומדויקים עבור משתמש הקצה במכשיר הנייד.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

תחום המערכות למעקב וניתוח ריצה על בסיס מיקום הינו מעטפת לתחום העיקרי של הפרויקט שהוא מציאת מסלול מיטבי תחת אילוצים. לתחום מציאת המסלול המיטבי נחשפתי לאחר שחיפשתי ברחבי האינטרנט אלגוריתם אשר מתייחס לבעיה ופותר אותה. במהלך המחקר נחשפתי לעולם של בעיות המסלול ובפרט לבעיית המסלול שבה הפרויקט עוסק - Orienteering. מקור השם Orienteering הוא בענף ספורט המתקיים בדרך כלל באזורים מיוערים. ביער ממוקמות מספר "נקודות בקרה" שלכל אחת מהן ציון משויך. למתחרים יש מצפן ומפה והם נדרשים לבקר בתת-קבוצה של נקודות בקרה מנקודת ההתחלה כדי למקסם את הציון הכולל שלהם ולחזור לנקודת הסיום בתוך פרק זמן שנקבע מראש כך שמתחרים שמגיעים באיחור נפסלים. כיום יש מספר מערכות מוכרות המאופיינות עם בעיית ה-Orienteeing, כמו Waze, Wolt, Wanderlog וכדומה.

2.4 אתגרים מרכזיים במחקר

המחקר שבבסיס פרויקט זה מורכב מכמה וכמה אתגרים אלגוריתמיים ומדעיים שלובים המצויים בחזית עולם האופטימיזציה הקומבינטורית:

מורכבות חישובית קומבינטורית – מצבים במדעי המחשב שבהם הפתרון לבעיה מבוסס על שילוב, בחירה וסידור של מספר אלמנטים בדידים – במקרה של פרויקט זה, בחירת תת-קבוצה של צמתים גאוגרפיים וקביעת סדר הביקור המדויק בהם מתוך רשת גיאוגרפית שלמה. האתגר המרכזי בבעיות קומבינטוריות מסוג זה, נובע מעצם היותן בעיות NP-Hard – בעוד שמספר נקודות העניין ברשת גדל באופן ליניארי ומתון, כמות השילובים והמסלולים הפוטנציאליים ביניהן צומחת בקצב מעריכי (אקספוננציאלי) או פקטוריאלי עצום.

צמיחה דרסטית זו של מרחב הפתרונות מייצרת מחסום חישובי חריף, המונע לחלוטין את היכולת להשתמש בפתרונות דטרמיניסטיים קלאסיים (כגון אלגוריתמים מדויקים או סריקה מלאה של כל האפשרויות בשיטת כוח גס - Brute Force). עבור רשתות גאוגרפיות בעלות עשרות או מאות צמתים, פתרונות קלאסיים אלו ידרשו זמן חישוב של שנים ויביאו לקריסת משאבי המערכת.

הסבת אלגוריתם רציף (PSO) למרחב דיסקרטי וגאוגרפי – באלגוריתם אופטימיזציה נחיל חלקיקים קלאסי (Standard PSO), החלקיקים "עפים" במרחב וקטורי רציף ואינסופי (כמו מערכת צירים תלת-ממדית), שבה כל מיקום מיוצג על ידי קואורדינטות מספריות. במרחב כזה, מושג המהירות הוא פשוט וקטור מתמטי המתווסף למיקום הנוכחי באמצעות פעולות חיבור וחיסור פשוטות, ומאפשר לחלקיק לנוע בחופשיות ובקוויים ישרים לכל נקודה במרחב.

לעומת זאת, בעולם הגאוגרפי והתחבורתי האמיתי, מרחב החיפוש אינו רציף אלא מוגדר כגרף דיסקרטי (רשת בדידה) של צמתים (הצטלבויות או נקודות עניין) וקשתות (הכבישים והשבילים המחברים ביניהן). במרחב קשיח זה, חלקיק אינו יכול לשהות בכל נקודה באוויר ומיקומו מוגבלים לחלוטין; מיקום של חלקיק אינו סתם סט של קואורדינטות, אלא הוא חייב לייצג מסלול ריצה שלם וחוקי – כלומר, רצף סדרתי של צמתים המחוברים פיזית זה לזה על ידי כבישים קיימים.

הפער התיאורטי העמוק מתבטא בכך שלא ניתן להפעיל פעולות אלגבריות קלאסיות על רכיבים של גרף; אי אפשר לחבר וקטור מהירות מספרי לצומת רחובות, ואי אפשר לחסר מסלול גאוגרפי אחד ממסלול אחר בצורה מתמטית רגילה, שכן התוצאה תהיה מסלול שבור שאינו קיים במציאות.

רגישות פרמטים – רגישות בכווןן וכיוול ערכי הפרמטרים של אלגוריתם ה-DPSO. אלגוריתמים אבולוציוניים, ובפרט אלו המבוססים על אינטליגנציית נחיל, תלויים בצורה מוחלטת במערכת של היפר-פרמטרים קשיחים המכתיבים את ההתנהגות הלוגית והמתמטית של המערכת כולה. בפרויקט זה, הפרמטרים המרכזיים כוללים את מקדם האינרציה (w), המקדם הקוגניטיבי ($c1$), והמקדם החברתי ($c2$). כל אחד מהמשתנים הללו משפיע ישירות על המשקל שהאלגוריתם מעניק לכל מקור מידע בעת בניית רשימת אומגה הזמנית לצורך עדכון המסלולים.

הקושי המחקרי המושרש בנושא זה נובע מתופעה הידועה שאין סט קבוע ויחיד של ערכי פרמטרים המסוגל להניב תוצאות אופטימליות עבור כל סוגי הקלטים והגרפים. כיוול לא מאוזן של הפרמטרים הללו מוביל מיד לפגיעה דרסטית בחוסן ובביצועים של הנחיל. אם, למשל, מקדם האינרציה (w) ייקבע כערך גבוה מדי, האלגוריתם יתמקד בחיפוש גלובלי מפוזר, יתעלם מנקודות ציון איכותיות שהתגלו, וימשיך "לעופף" במרחב החיפוש מבלי להצליח להתכנס למסלול סופי ויציב. מצד שני אם המקדמים $c1$ ו- $c2$ יהיו דומיננטיים מדי בשלב מוקדם, הנחיל יסבול מתופעה הרסנית של התכנסות מוקדמת (Premature Convergence) – כל החלקיקים יימשכו במהירות עצומה למסלול הסביר הראשון שיתגלה, יאבדו את המגוון הנדרש לסריקת המפה, ויבלעו לחלוטין בתוך מלכודת האופטימום המקומי.

מורכבות מחקרית זו מחריפה ומקבלת משנה תוקף נוכח האופי הדינמי של אפליקציית מעקב הריצה. רשת הצמתים הגאוגרפית משתנה לחלוטין מקריאה לקריאה בהתאם למיקומו המשתנה של הרץ ולמרחק היעד שהוא מגדיר באותו הרגע במכשיר הנייד. גרף קטן ומצומצם (העלול להיווצר כאשר משתמש מבקש ריצה קצרה של 2 קילומטרים בסביבה עירונית צפופה) דורש קצב התכנסות ואיזון פרמטרים שונה לחלוטין מגרף רחב ומבוזר המייצר מרחב חיפוש עצום עבור ריצות ארוכות טווח של עשרות קילומטרים.

2.4.1 התמודדות עם הבעיה

ההתמודדות עם האתגר הראשון של המורכבות החישובית מיושמת באמצעות בחירה באלגוריתם (DPSO) (Discrete Particle Swarm Optimization). במקום לבצע סריקה מלאה ויקרה של כל מסלול אפשרי, האלגוריתם מנהל "נחיל" של פתרונות מועמדים (חלקיקים) הסורקים את מרחב החיפוש בצורה מקבילית וחכמה, ומצליחים להפיק פתרונות באיכות פנומנלית בזמן קצר המשתלב היטב בדרישות של מערכת אינטראקטיבית בזמן אמת.

ההתמודדות עם האתגר המחקרי השני – הסבת האלגוריתם למרחב דיסקרטי – מיושמת על ידי הגדרה מתמטית חדשה של מושג ה"תנועה" של החלקיק. כל חלקיק בנחיל מיוצג כמסלול ריצה חוקי המורכב מרצף צמתים גאוגרפיים. עדכון המיקום והמהירות של החלקיק מתבצע באמצעות לוגיקה הסתברותית ייחודית. מנגנון זה משלב בין רכיב האינרציה (w) השומר על צמתים מהמסלול הנוכחי, הרכיב הקוגניטיבי ($c1$) המושך את החלקיק אל עבר שיא העבר האישי שלו ($p\ best$), והרכיב החברתי ($c2$) המכוון את החלקיק אל עבר המסלול המוצלח ביותר שהתגלה על ידי הנחיל כולו ($g\ best$). אם השילוב ההסתברותי יוצר מסלול שחורג ממגבלת המרחק הקשיחה של המשתמש, מופעל מנגנון תיקון אקטיבי המוחק צמתים עודפים עד לחזרה לישימות מתמטית.

וכדי להתמודד עם אתגר השלישי – כיול ערכי הפרמטרים, נדרש לבצע ניתוח רגישות מעמיק ובדיקות נרחבות ולאתר את הערכים מספריים מדויקים עבור המקדמים השונים אשר יעניקו לאלגוריתם חוסן מבני. חוסן זה מבטיח כי המערכת תדע להתאים את עצמה באופן אוטומטי ורציף לכל גודל גרף ולכל אילוף מרחק שיציב המשתמש, ותפיק עבורו את מסלול הריצה הרווחי והמדויק ביותר, תוך עמידה בלתי מתפשרת במגבלת זמן ריצה השומרת על יעילות המערכת בזמן אמת.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

המוטיבציה המרכזית מאחורי בחירת פרויקט זה נובעת מהפער המהותי הקיים כיום בשוק בין אפליקציות הכושר והניווט הפופולריות לבין הצרכים המשתנים והדינמיים של רצים בעולם המודרני. מרבית היישומים הקיימים כיום בשוק מתפקדים ככלים פסיביים בלבד, אשר יודעים לתעד בדיעבד את מסלול הריצה שהמשתמש כבר ביצע, או לחילופין, מציעים ניתוח פשוט של נקודת מוצא ויעד ($A\ to\ B$) מבלי להתחשב במטרות הספורטיביות הייחודיות של הרץ.

המוטיבציה המחקרית הייתה לקחת את עולם הנדסת התוכנה והאלגוריתמיקה המתקדמת, שבדורות הקודמים שימשה בעיקר לבעיות לוגיסטיות תעשייתיות או הפצת סחורות, ולהנגיש אותה בצורה חכמה, אקטיבית ואישית לחיי היום-יום של ספורטאים וחובבי בריאות.

האתגר המרתק של שילוב אלגוריתם מורכב כמו DPSO בתוך סביבה מוגבלת במשאבים כמו מכשיר טלפון נייד, היווה מנוע דחיפה משמעותי לבחירת הנושא. הידיעה שניתן לייצר כלי אשר הופך את חוויית התכנון המייגעת והידנית של מסלולי ריצה לתהליך אוטומטי, מבוסס מדע ומתמטיקה, יצרה עניין אקדמי ומעשי רב. פיתוח אפליקציה מסוג זה מאפשר לחקור כיצד לוגיקה עסקית מורכבת וחשובים כבדים יכולים להתבצע מאחורי הקלעים בשרת (Backend) בצורה יעילה, תוך שמירה על ממשק משתמש קליל, אינטראקטיבי ונגיש בצד הלקוח (Frontend), ובכך למעשה להציע פתרון שלם המפגיש בין תיאוריה מדעית מופשטת לבין יישום הנדסי פרקטי ושימושי.

2.4.3 על איזה צורך הפרויקט עונה ?

פרויקט זה עונה על צורך ממשי ויומיומי של קהילת הרצים הגדלה והולכת, המתמודדת באופן קבוע עם מגבלות של זמן, שגרה ובטיחות. רצים רבים, בין אם חובבים ובין אם מקצועיים, חווים תחושת מיצוי ושחיקה מונוטונית כאשר הם נאלצים לחזור פעם אחר פעם על אותם מסלולי ריצה קבועים ומוכרים בסביבת מגוריהם. מן העבר השני, הניסיון לגוון ולתכנן מסלול חדש באופן ידני על גבי מפה הוא משימה מורכבת ומייגעת, שלעיתים קרובות נכשלת במבחן המציאות בשל חוסר יכולת להעריך במדויק את המרחק הכולל, את תנאי השטח או את התאמת הדרך למגבלות האימון האישיות.

האפליקציה שפותחה עונה במדויק על הצורך הזה בכך שהיא משמשת כמנווט מסלולים אישי אוטומטי. המערכת מסירה מהמשתמש את נטל התכנון ומאפשרת לו פשוט להזין את אילוציו לאותו היום – כגון מרחק יעד מדויק (למשל, "אני רוצה לרוץ בדיוק 7.5 קילומטרים") או רצון לעבור בנקודות ציון ספציפיות – ולקבל בלחיצת כפתור מסלול מיטבי, מעגלי או קווי, המנצל בצורה מקסימלית את נקודות העניין והבטיחות בסביבתו. מעבר לכך, הפרויקט עונה על הצורך בריכוז נתונים מתקדם, ניתוח מדדים פיזיולוגיים בזמן אמת, והצגת גרפים השוואתיים המאפשרים למשתמש לעקוב אחר שיפור ביצועיו ולבחון את עמידתו ביעדים לטווח ארוך, ובכך מעניק מעטפת טכנולוגית מלאה המשפרת את חוויית האימון ואת המוטיבציה האישית של הרץ.

3. מטרות ויעדים לפיתוח המערכת

מטרת העל של הפרויקט היא לפתח מערכת תוכנה שלמה מקצה לקצה, המספקת פתרון טכנולוגי מקיף וחכם עבור רצים ומתאמנים. המערכת נועדה להחליף את הצורך בתכנון מסלולים ידני ומייגע, וללוות את הספורטאי משלב התכנון ועד לניתוח ביצועיו בשטח.

מתוך מטרה כללית זו, נגזרו מספר יעדים מעשיים ופרקטיים שעל המערכת להגשים:

- **הפקה אוטומטית של מסלולי ריצה מותאמי מרחק:** יצירת מנגנון חכם בצד השרת המסוגל לקבל מהמשתמש אילוץ מרחק פשוט (לדוגמה: בקשה לרוץ בדיוק 5 קילומטרים) ומיקום נוכחי,

ולתרגם אותם באופן מיידי לנתיב ריצה פיזי שלם, חוקי ובר-ביצוע, המשורטט על גבי המפה הדיגיטלית.

- **טיוב איכות המסלול ושילוב נקודות עניין סביבתיות:** המערכת שואפת לא סתם למצוא רצף רחובות אקראי, אלא לייצר מסלול אטרקטיבי וחוויתי. יעד זה יושג על ידי פנייה לשירותי מפה חיצוניים, איתור אזורים מועדפים לריצה (כמו פארקים, גינות ציבוריות או מסלולי הליכה) ושילובם בתוך תוואי הדרך בהתאם לרמת הפופולריות שלהם בשטח.
- **תמיכה דינמית במסלולים מעגליים וקווים:** אספקת מענה גמיש לסוגי אימון שונים. המערכת נדרשת לדעת לתכנן הן מסלול קווי מנקודה א' לנקודה ב', והן מסלול מעגלי מורכב, המבטיח חזרה מדויקת של הרץ אל נקודת המוצא ממנה התחיל, מבלי לחרוג מתקציב המרחק שהוגדר.
- **ניטור אקטיבי ומדידת מדדי שטח בזמן אמת:** פיתוח רכיב מובייל המלווה את המתאמן במהלך הריצה הפיזית ומבצע מעקב מיקום רציף. האפליקציה נדרשת למדוד ולהציג דינמית נתונים קריטיים כגון מרחק מצטבר, זמני פעילות (נטו וברוטו), קצב תנועה נוכחי וספירת צעדים, תוך מתן שליטה מלאה למשתמש על מצבי האימון (הפעלה, השגחה ועצירה).
- **ריכוז היסטוריית אימונים וניתוח ויזואלי של ביצועים:** יצירת תשתית אחסון בענן השומרת ומגבה כל אימון שהסתיים. המערכת נדרשת לאפשר למשתמש לגשת בנוחות להיסטוריית הפעילויות שלו, להציג עבורו סיכומים מפורטים של מדדים פיזיולוגיים וטופוגרפיים (כמו הפרשי גבהים וקצבים ממוצעים), ולספק ניתוח גרפי ויזואלי של מגמות השיפור שלו לאורך זמן.

4. אתגרים בבניית המערכת

בניית המערכת ומימושה בקוד הציבו מספר אתגרים הנדסיים מורכבים בשלבי הפיתוח, התכנות והאינטגרציה של רכיבי התוכנה השונים:

- **ניהול תהליכים ברקע ומניעת השבתה:** מערכת ההפעלה Android ידועה במדיניות קשוחה של חיסכון בסוללה, הנוטה לסגור אפליקציות שרצות ברקע. אחד האתגרים המרכזיים בפיתוח היה מימוש שירות רקע יציב (Foreground Service) שיועד להאזין לחיישן ה-GPS ולספור צעדים בצורה רציפה לאורך זמן, מבלי שמערכת ההפעלה תהרוג את התהליך בעיצומה של הריצה.
- **ניהול מצבי מערכת ומחזור חיים:** ניהול המעברים הדינמיים בין מצבי האימון השונים (הפעלה, השגחה, חזרה ועצירה) דרש סנכרון ארכיטקטוני עדין. האתגר היה להבטיח שגם אם המשתמש ממזער את האפליקציה, מכבה את המסך, או מקבל שיחת טלפון, נתוני המדידה לא יימחקו מזיכרון המכשיר והתהליכים המקביליים של השעון והמרחק יוקפאו ויחזרו לפעול בדיוק מאותה נקודה.

- **תקשורת אסינכרונית וטיפול בהשהיות רשת:** השרת נדרש לבצע קריאות רשת מרובות מול ממשקים חיצוניים, לעבד נתוני מפות כבדים ולתרגם אובייקטים מורכבים (JSON). האתגר בפיתוח היה לנהל את התקשורת הזו בצורה אסינכרונית ויעילה בצד השרת, כדי שזמני ההמתנה של חילופי המידע והמרחקים לא יגרמו לעיכובים (Timeouts) ולא יתקעו את זרימת התוכנה באפליקציה.
- **התאמת מרחב גאוגרפי רציף ללוגיקה בדיסקרטית:** אחד האתגרים הייחודיים בפיתוח הליבה היה הצורך לקחת מפה מהעולם האמיתי – שהיא מרחב פיזי רציף, דינמי ואינסופי של כבישים ושבילים – ולתרגם אותה לגרף לוגי ממוקד ובדיד שהשרת מסוגל לעבד. המימוש דרש פיתוח של שלב הכנה חכם שמסנן נקודות גולמיות, מייצר רשת של "שכנים קרובים" ומחשב מרחקי הליכה ריאליים, כדי שהאלגוריתם המרכזי יוכל לרוץ על מבנה נתונים קטן ויעיל, מבלי לאבד את הדיוק הגיאוגרפי הנדרש בשטח.
- **איזון דינמי בין שאיפת תגמול לאילוץ תקציב קשיח:** ליבת השרת נדרשת להתמודד עם מתח מתמטי מורכב בזמן אמת: מצד אחד, הרצון למקסם את איכות וחווית המסלול (על ידי משיכת הנתיב לעבר אזורים ירוקים, פארקים ונקודות עניין פופולריות), ומצד שני, האילוץ להפסיק את החיפוש ולחזור הביתה ברגע שהמסלול נושק למגבלת המרחק של המשתמש. האתגר ההנדסי היה פיתוח מנגנון תיקון דינמי בתוך האלגוריתם, שמונע מהמערכת "להתפתות" לנקודות אטרקטיביות מדי שיגרמו לחריגה מהתקציב, ומבטיח סגירה חוקית ומדויקת של הלולאה בנקודת המוצא.

5. הגדרת מדדי הצלחה למערכת

- כדי להעריך את תקינות המערכת ואיכותה בתום תהליך הפיתוח, נקבעו מדדי הצלחה הבאים:
- **סטיית מרחק מינימלית:** המסלול הפיזי שהמערכת מפיקה בפועל יהיה קרוב ככל הניתן למרחק היעד שהגדיר המשתמש, עם אחוז סטייה נמוך וזניח.
 - **מהירות הפקה וזמינות:** השרת יצליח לעבד את הנתונים, להפעיל את שיטות האופטימיזציה ולהחזיר מסלול מוגמר ומתוכנן אל מסך המכשיר הנייד בתוך שניות בודדות מרגע הלחיצה.
 - **יציבות ואמינות המעקב אקטיבי:** האפליקציה תפעל בצורה חלקה לאורך כל זמן הריצה, תאסוף את הקואורדינטות ברקע ללא קריסות, ותציג נתונים סטטיסטיים מדויקים ומסונכרנים.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

מאחר והבעיה אינה ניתנת לפתירה בזמן סביר, ישנם מספר אלגוריתמי שיפור שהוצעו במהלך היסטורית מחקר הבעיה:

מעבר לבעיה שבה הפרויקט עוסק, ישנן מספר בעיות נוספות היושבות תחת קטגוריית בעיות המסלול:

בעיית הסוכן הנוסע (Travelling Salesman Problem) - מתייחסת לבעיה בה צריך למצוא מסלול מינימלי שעובר בדיוק פעם אחת בכל נקודה, וחוזר לנקודת ההתחלה. לבעיה זו יש תת גרסאות עבור מקרים ספציפיים כגון: Prize-Collecting TSP - בעיית TSP עם איסוף פרסים (ניקודים) ועמידה במכסת סכום הפרסים שניתן לצבור תוך שמירה על אורך מסלול קצר ביותר, k-TSP - יש k סוכנים שצריכים לבקר בכל הצמתים מתחנת מוצא משותפת, והמטרה היא לחלק ביניהם את הצמתים כך שסך אורכי המסלולים (או המסלול הארוך ביותר) יהיה מינימלי.

בעיית ניתוב רכבים (Vehicle Routing Problem) - מתייחסת לבעיה בה צריך לתכנן מסלולים עבור כמה שליחים ולא עבור אחד, כך שכל היעדים יקבלו שירות תוך עמידה במגבלות (כמו: קיבולת, שעות, מרחק, אנרגיה). גם לבעיה זו יש מספר וריאציות כמו: Capacitated VRP - בעיית VRP שבה לכל רכב יש קיבולת מוגבלת (למשל נפח משאית או מספר חבילות) ויש לספק את כל הלקוחות או לנקודות היעד, תוך שמירה על מגבלת הקיבולת, ולמזער עלות/מרחק כולל, Multi-Depot Vehicle Routing Problem - בעיית VRP שבה יש מספר נקודות התחלה ויש לתכנן מסלולים עבור כל רכב שיוצא מנקודת התחלה מסוימת, וכו'.

בעיית המסלול הקצר ביותר (Shortest Path Problem) - מתייחסת למציאת המסלול הקצר ביותר עבור גרף נתון. יש מספר אלגוריתמים העוסקים בפתרון בעיה זו (עם מעט הבדלים) כמו: Dijkstra - הוא אלגוריתם למציאת המסלול הקצר ביותר מצומת התחלה אחת אל כל שאר הצמתים בגרף שבו המשקלים אינם שליליים. Bellman-Ford - הוא אלגוריתם למציאת המסלול הקצר ביותר מצומת מקור יחיד לכל שאר הצמתים בגרף עם משקולות קשתות (כולל משקולות שליליות). Floyd-Warshall הוא אלגוריתם למציאת המסלולים הקצרים ביותר בין כל זוג צמתים בגרף.

גם בתוך בעיית ה-Orienteering יש מספר וריאציות: Orienteering with Variable Profits - זו וריאציה, שבה הרווח (profit) של כל נקודה אינו קבוע וידוע מראש, אלא משתנה בהתאם לנסיבות מסוימות, במהלך המסלול Team Orienteering - וריאציה שבה יש כמה שליחים, ולכל אחד צריך למצוא מסלול עם מגבלת זמן/מרחק, כך שסך הרווח מכל המסלולים ביחד יהיה מקסימלי.

התחום בו המערכת שלי מתרכזת הוא תחום ה-Orienteeing הקלאסית כפי שהגדרנו בהגדרת הבעיה האלגוריתמית (ונקראת גם Generalized TSP). מאחר והבעיה אינה ניתנת לפתירה בזמן סביר, ישנם מספר אלגוריתמי שיפור שהוצעו במהלך היסטורית מחקר הבעיה:

אלגוריתם גנטי (Genetic Algorithm) - אלגוריתם גנטי הוא אלגוריתם בהשראת תהליכים אבולוציוניים בטבע. הוא פועל על אוכלוסייה של פתרונות אפשריים, כאשר כל פתרון מכונה פרט (Individual) ומיוצג בדרך כלל כמחרוזת גנים (למשל וקטור, רצף צמתיים, ביטים וכו'). בכל איטרציה האלגוריתם מבצע "אבולוציה": הוא מעריך את איכות כל פרט לפי פונקציית מטרה, בוחר את הפרטים הטובים ביותר ("שורדים"), יוצר צאצאים באמצעות Crossovers (שילוב בין גנים של שני פרטים איכותיים), ומבצע Mutations (שינויים אקראיים קלים) כדי למנוע קיבעון ולאפשר חיפוש גלובלי. כך נוצרות אוכלוסיות חדשות שהן לרוב טובות יותר מהקודמות. **יתרונות:** מתאים לבעיות גדולות ומורכבות, מסוגל לברוח ממינימום מקומי, עובד היטב על בעיות קומבינטוריות כמו מסלולים וגרפים, ומאפשר שילוב קל עם טכניקות Local Search. **חסרונות:** אינו מבטיח אופטימליות, רגיש להגדרות פרמטרים (גודל אוכלוסייה, שיעור מוטציה וכו'), עלול להיות איטי אם לא מכוון היטב, ולעיתים זקוק להרצות רבות כדי להבטיח פתרון איכותי.

Branch & Cut - הוא אלגוריתם מדויק (Exact), כלומר שואף למצוא את הפתרון האופטימלי המוחלט לבעיה קומבינטורית, ואחד מהכלים המרכזיים לפתרון בעיות NP-Hard כמו TSP, Orienteering, VRP ועוד. הוא מבוסס על פתרון אינטגרלי של בעיות תכנות ליניאריות בשלמים (ILP). תחילה האלגוריתם פותר את גרסת ה-LP relaxation - כלומר מתיר למשתנים לקבל ערכים רציפים בין 0 ל-1. התוצאה היא חסם עליון לפתרון הטוב ביותר. לאחר מכן בודקים אם הפתרון רציף (לא שלם). אם כן - האלגוריתם מבצע Branching, מפצל את הבעיה לשתי תתי-בעיות ומוסיף אילוצים שמכריחים את המשתנים להתקבע לערכים אינטגרליים (0 או 1). במקביל האלגוריתם מבצע Cutting Planes - מוסיף אילוצי חיתוך שמטרתם לחתוך אזורים לא חוקיים במרחב ה-LP ולשפר את החסם. התהליך נמשך עד שכל העלים בעץ נותנים פתרונות שלמים או נחתכים. **יתרונות:** מספק פתרון אופטימלי ודטרמיניסטי, נותן הוכחה לרמת האיכות של התוצאה, יעיל יחסית בזכות Cuts שמקטינים את מרחב החיפוש. **חסרונות:** זמן ריצה אקספוננציאלי במקרה הגרוע, הופך לאיטי מאוד בבעיות עם מאות צמתים, דורש הרבה זיכרון, ומרגיש "כבד" לתפעול בעולם הפרקטי כשצריך תוצאה מהירה.

אלגוריתם חמדני (Greedy Algorithm) - אלגוריתם חמדני הוא אלגוריתם שבכל צעד בוחר את האפשרות שנראית הטובה ביותר ברגע הנוכחי, בלי להתחשב בעתיד או בתמונה הכוללת. ברוב המקרים הוא פועל בצורה מאוד פשוטה: מתחילים מפתרון ריק, ובכל שלב מוסיפים את הרכיב הבא עם

הערך הטוב ביותר (למשל הצומת בעל הרווח הגבוה ביותר ליחידת זמן, הקשת הקצרה ביותר, או הבחירה עם העלות המינימלית). ההחלטות הן לוקליות בלבד, ללא תיקון טעויות ובלי חזרה אחורה. האלגוריתם מהיר ביותר (כמעט תמיד זמן פולינומי פשוט), קל ליישום, אינו דורש פרמטרים או כיוון, ופועל טוב כבסיס לפתרונות התחלתיים באלגוריתמים מתקדמים (כמו PSO, GA, B&C). אך עם זאת, הוא עלול להיתקע בפתרון גרוע, אינו מבטיח אופטימליות, לא יודע "להתחרט", תלוי באופן קריטי בפונקציית הבחירה שלו, ולעיתים קרובות מספק פתרון באיכות הרבה פחות טובה בהשוואה לאלגוריתמים מתוחכמים יותר.

Discrete Particle Swarm Optimization - האלגוריתם DPSO הוא אלגוריתם מטא-היוריסטי אבולוציוני בהשראת התנהגות של להקות ציפורים ודיוני דגים. ה-DPSO הוא גרסה מותאמת של PSO שמיועדת לעבוד על בעיות שבהן הפתרונות הם דיסקרטיים, כמו בבעיית ה-Orienteering שבה כל פתרון אפשרי הוא "חלקיק" (מסלול) שמורכב מרשימת צמתים בתוך מרחב של פתרונות קיימים (אוכלוסייה). פונקציית המטרה (fitness) של כל חלקיק היא סכום הניקוד הכולל של הצמתים שהוא מבקר בהם, כך שהמטרה היא למקסם את סכום הניקוד תוך עמידה במגבלת המסלול T_{max} . הפתרון מיוצג על ידי ה-position של החלקיק שהוא אוסף של הצמתים שבהם המסלול מבקר על פי סדר הביקור. בכל איטרציה האלגוריתם שואף לייצר מסלול טוב יותר על בסיס שילוב בין ניסיון עבר וניסיון קולקטיבי של כל החלקיקים.

לעומת זאת, PSO רגיל מתאים רק למרחבים רציפים שבהם אפשר להגדיר מהירות, כיוון ותנועה וקטורית, אך בעיית המסלול אינה מאפשרת לבצע פעולות כאלה, אי אפשר "להזיז" מסלול בצורה מתמטית כמו שמזיזים נקודה במרחב. לכן DPSO משתמש במקום במנגנון הסתברותי שמרכיב מסלולים חדשים מתוך שילוב של המסלול הנוכחי, המסלול הטוב שהחלקיק מצא בעבר, והמסלול הטוב ביותר שנמצא בכלל האוכלוסייה. באופן הזה האלגוריתם שומר על עקרונות הלמידה של PSO, אבל מתאים אותם למבנה הדיסקרטי של הבעיה. הבחירה ב-DPSO מאפשרת לנצל את היתרונות של PSO - פשטות, ביצועים טובים והתאמה לחיפוש במרחב גדול, תוך התאמה מלאה לבעיה שבה הפתרונות הם מסלולים ולא מספרים רציפים.

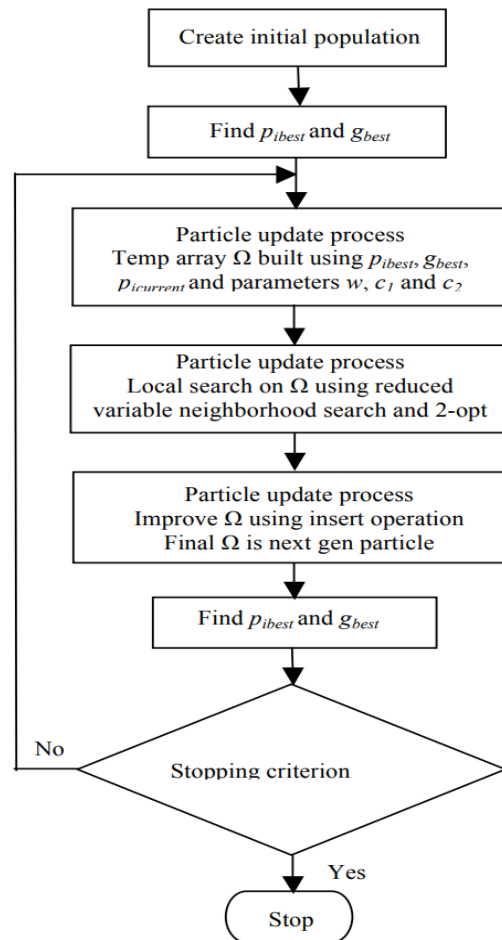
להלן טבלת השוואה וסיכום בין סוגי האלגוריתמים:

אלגוריתם	יתרונות	חסרונות
אלגוריתם גנטי (Genetic Algorithm)	מתאים לבעיות גדולות ומורכבות, ועובד היטב על בעיות קומבינטוריות (מסלולים, גרפים)	אינו מבטיח אופטימליות, רגיש לכיוון פרמטרים (גודל אוכלוסייה, מוטציה), עלול להיות איטי, דורש מספר רב של הרצות לקבלת פתרון איכותי
Branch & Cut	מספק פתרון אופטימלי ודטרמיניסטי, נותן הוכחה לאיכות הפתרון.	זמן ריצה אקספוננציאלי במקרה הגרוע, לא סקילבילי לבעיות גדולות, צורך זיכרון גבוה, כבד ולא מתאים למצבים הדורשים פתרון מהיר
אלגוריתם חמדני (Greedy Algorithm)	מהיר מאוד, פשוט ליישום, אינו דורש פרמטרים או כיוון.	אינו מבטיח אופטימליות, עלול להיתקע בפתרון גרוע, מבצע החלטות ללא חזרה אחורה, תלוי מאוד בפונקציית הבחירה.
Discrete Particle Swarm Optimization (DPSO)	מספק איזון מיטבי בין איכות הפתרון, זמן הריצה והגמישות, המתאימים במיוחד לאופי הבעיה, וקל יחסית למימוש ולשילוב עם Local Search	אינו מבטיח פתרון אופטימלי, רגיש לפרמטרים (גודל להקה, מקדמים וכדומה).

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

החלופה הנבחרת עבור הפרויקט היא **אלגוריתם ה-DPSO**. הסיבה לבחירה בו היא שהוא מאפשר לאזן בין איכות הפתרון לבין זמן חישוב הפתרון באופן הכי טוב מבין האלגוריתמים המוצעים ומאפשר גמישות וכיוון של הפרמטרים על מנת להגיע לאיזון זה בהתאם ליחס איכות לעומת זמן חישוב.

פתרון הבעיה האלגוריתמית מיושם לפי מספר שלבים מוגדרים של האלגוריתם הנבחר (DPSO):



להלן הסבר על כל שלב שמופיע בתרשים:

שלב 1 – בניית פתרון התחלתי איכותי

יש לייצר פתרונות התחלתיים שישמשו נקודת פתיחה טובה לתהליך האופטימיזציה. מטרת שלב זה היא ליצור מסלול ישים - כלומר, כזה שעומד במגבלות הנתונות - תוך ניסיון לנצל ערכים גבוהים ולבנות מבנה ראשוני בעל פוטנציאל לשיפור. פתרון התחלתי איכותי מאפשר לאלגוריתם להימנע מאזורים גרועים במרחב החיפוש ולכוון את תהליך האופטימיזציה לעבר מסלולים מבטיחים כבר מהצעדים הראשונים.

שלב 2 – הפקת פתרונות חדשים על בסיס ניסיון עבר

בשלב זה נוצרים מסלולים חדשים באמצעות שילוב בין מסלול קיים לבין מידע שנצבר מפתרונות מוצלחים בעבר. הרעיון הוא לנצל דפוסים טובים שכבר זוהו - כמו צמתים שהופיעו במסלולים איכותיים או סדרי ביקור שהוכיחו עצמם - ולשלב אותם בתוך מסלולים מועמדים חדשים. תהליך זה מאפשר לאלגוריתם להרחיב את אזורי החיפוש המבטיחים ולייצר וריאציות מתקדמות המבוססות על ידע

שנצבר לאורך הדרך. לצורך כך, עבור כל חלקיק נבנית רשימה זמנית הנקראת Ω (אומגה). עבור החלקיק הנוכחי, עוברים על כל צומת במסלול שלו. מגרילים מספר אקראי Λ בין 0 ל-1, ובודקים אותו מול שלושה פרמטרים: $c_1 w$, c_2 ו- c_1 . הפרמטר w מייצג את רכיב האינרציה, והוא קובע את הסיכוי לשמור צמתים מהמסלול הנוכחי; הפרמטר c_1 הוא הרכיב הקוגניטיבי, והוא שולט בסיכוי לבחור צמתים שעבדו טוב עבור החלקיק בעבר (p_{ibest}); הפרמטר c_2 הוא רכיב הלמידה החברתית, והוא קובע את המשיכה למסלול הטוב ביותר הידוע בכלל הקבוצה (g_{best}). ובכך נבנית רשימת הצמתים Ω , שהיא תערובת הסתברותית של חלקי מסלולים מוצלחים, שלאחר ניקוי כפילויות הופכת לבסיס שממנו נבנה מסלול חדש. מסלול זה נבדק תחילה לשימות בהתאם למגבלת הזמן; ואם הוא חורג, האלגוריתם מוחק ממנו צמתים עד שהפתרון הופך בר-ביצוע.

שלב 3 – שיפור מקומי של מסלולים מועמדים

לאחר יצירת מסלולים חדשים, מתחיל שלב שיפור מקומי (local search) הנקרא RVNS (Reduced Variable Neighborhood Search), שמטרתו לחדד את איכות הפתרונות ללא שינוי מבני עמוק. ה-local search מוגדר באמצעות שתי פעולות בסיסיות: insert, המנסה להוסיף למסלול צומת חדש שאינו מופיע בו כל עוד מגבלת המרחק מאפשרת, ו-exchange, המחליפה צומת קיים בצומת אחר בעל תרומה אפשרית גבוהה יותר. מטרת שני המנגנונים היא לאתר שינויים קטנים שיכולים לשפר משמעותית את הניקוד הכולל של המסלולים.

שלב 4 – ייצוב ושיפור מבני המסלול

בשלב זה התמקדות היא באופטימיזציה של מבנה המסלול עצמו באמצעות פעולת opt-2. המטרה היא לצמצם את אורך המסלול ולשפר את הרציפות והיעילות הגאומטרית שלו. התאמות אלה עשויות לאפשר הקטנת עלות כוללת תוך שמירה על הצמתים החשובים, ולעיתים אף לפתוח אפשרות להוסיף נקודות ביקור נוספות בעתיד. השלב מביא לכך שהמסלול הופך קצר, יציב ויעיל יותר מבחינה מרחבית. פעולת ה-opt-2 עובדת באמצעות החלפת זוג קשתות כך שמסלול חלקי מתהפך, במטרה לקצר את הדרך ולהפחית את עלות הנסיעה הכוללת. אם פעולת opt-2 הצליחה לצמצם את אורך המסלול, האלגוריתם מנסה שוב להוסיף צמתים רווחיים באמצעות insert נוסף, כדי לנצל את "הזמן שהתפנה". המסלול המשופר מתקבל כ-position החדש של החלקיק, ואם הוא טוב יותר מה- p_{ibest} - הוא מחליף אותו; ואם הוא טוב יותר מכל מסלול אחר באוכלוסייה - הוא הופך ל- g_{best} .

שלב 5 – בדיקת ישימות והתאמה למגבלות

לאחר ביצוע השיפורים, נדרש לוודא שהמסלול החדש עדיין עומד בכל אילוצי הבעיה המתמטית. אם השיפור המקומי או המבני הוביל לחריגה ממגבלת האורך, יש לבצע התאמות המבטיחות חזרה

לפתרון ישים. תהליך זה שומר על חוקיות הפתרון לאורך כל שלבי החיפוש ומבטיח שהאלגוריתם אינו סוטה מהגדרות הבעיה.

שלב 6 – בחירת הפתרונות המתאימים והתקדמות לדור הבא

בשלב זה בוחנים את הפתרונות שהתקבלו ומחליטים אילו מהם ימשיכו להשפיע על החיפוש בהמשך. המטרה היא למצוא איזון בין שמירה על פתרונות איכותיים שכבר הובילו לשיפור משמעותי לבין עידוד של חיפוש חדש באזורים לא נחקרים. כך האלגוריתם שומר על מגוון פתרונות ולא נתקע במינימום מקומי.

שלב 7 – עצירת החיפוש והחזרת הפתרון הטוב ביותר

כאשר מגיעים לתנאי העצירה - מספר מוגדר מראש של איטרציות ללא שיפור או מגבלת זמן - האלגוריתם עוצר ומחזיר את המסלול הטוב ביותר שנמצא לאורך כל התהליך. מסלול זה מייצג את השילוב בין ניקוד גבוה, עמידה באילוצים ועילות מרחבית, ומשמש כפתרון הסופי לבעיית ה-Orienteeing.

8. אפיון המערכת המוצעת

8.1 ניתוח הדרישות מהמערכת

ניתוח הדרישות מהמערכת מהווה שלב קריטי בתהליך הנדסת התוכנה, ומטרתו להגדיר בצורה מדויקת וחד-משמעית את היכולות, הפונקציות והמגבלות של המערכת המוצעת. ניתוח זה מבטיח כי הפיתוח יענה במלואו על צורכי משתמשי הקצה, ויספק מעטפת טכנולוגית יציבה וחסינה. הדרישות מהמערכת מסווגות לשני סוגים מרכזיים: דרישות פונקציונליות ודרישות לא-פונקציונליות.

דרישות פונקציונליות (Functional Requirements)

הדרישות הפונקציונליות מגדירות את הפעולות, השירותים וההתנהגות הלוגית שהמערכת מחויבת לבצע עבור המשתמש. במערכת המוצעת הוגדרו דרישות הליבה הבאות:

- **ניהול ואימות משתמשים:** המערכת נדרשת לאפשר למשתמשים לבצע תהליכי הרשמה, התחברות מאובטחת והתנתקות, תוך שמירה ואימות של פרטי הזהות מול מסד הנתונים.

- **הפקת מסלולים מותאמים אישית:** המערכת נדרשת לקלוט אילוצים מהמשתמש (כגון מרחק מבוקש או מיקום), לתשאל שירותי מפה חיצוניים, ולהריץ אלגוריתם להפקת מסלול מיטבי (מעגלי או קווי) והצגתו ויזואלית על המפה.
- **ניהול ומעקב ריצה בזמן אמת:** המערכת נדרשת להפעיל את חיישני המכשיר (GPS) כדי לעקוב אקטיבית אחר מיקום הרץ, למדוד נתוני מרחק וזמן, ולאפשר שליטה במחזור חיי האימון (הפעלה, השהייה ועצירה).
- **היסטוריה וניתוח סטטיסטי:** המערכת נדרשת לתעד ולשמור כל אימון, לאפשר למשתמש לדפדף בהיסטוריית הריצות שלו, ולהציג נתונים אנליטיים וסטטיסטיקות מורחבות באמצעות גרפים ומדדים מצטברים.

דרישות לא-פונקציונליות (Non-Functional Requirements)

- הדרישות הלא-פונקציונליות (דרישות איכות) מגדירות את האילוצים, הסטנדרטים ותכונות המערכת המשפיעים על חוויית השימוש, השרידות והביצועים ההנדסיים:
- **זמן תגובה וביצועים (Performance):** מנוע האופטימיזציה והפקת המסלולים בשרת נדרש לפעול במהירות מרבית, ולהחזיר מסלול מוגמר לצד הלקוח בזמן תגובה סביר של עד כמספר שניות בודדות מרגע שליחת הבקשה, כדי למנוע המתנה ממושכת.
 - **זמינות (Availability):** המערכת נדרשת להציג זמינות גבוהה.
 - **יעילות וניהול משאבים (Resource Efficiency):** תהליכי הניטור והמעקב בצד הלקוח נדרשים להתבצע בצורה אופטימלית, הממזערת את העומס על מעבד הטלפון הנייד וחוסכת בצריכת אנרגיית הסוללה של המכשיר במהלך הריצה.
 - **תאימות וניידות (Compatibility):** אפליקציית הקצה צריכה לפעול בצורה חלקה ויציבה על גבי מגוון גרסאות של מערכת ההפעלה Android, ולהתאים את עצמה דינמית לגדלי מסכים שונים (רספונסיביות).
 - **שלמות ואבטחת נתונים (Data Integrity):** המערכת נדרשת להבטיח כי נתוני המשתמשים ואימוניהם יישמרו בצורה מאובטחת, חסינה מפני שיבושים, וללא אובדן מידע בעת העברתו בפרוטוקולי התקשורת בין הלקוח לשרת.

8.2 מודלי המערכת

מודלי המערכת (רכיבים לוגיים ופונקציונליים)

המערכת בנויה מארבעה מודולים פונקציונליים מרכזיים, המנהלים את זרימת המידע, העיבוד הסטטיסטי והאלגוריתמיקה מקצה לקצה:



1. מודל אימות וניהול משתמשים

מודל זה מופקד על אבטחת הגישה למערכת וניהול פרופילי המשתמשים מול שכבת האחסון המרכזית. הרכיב מנהל את תהליכי הרישום וההתחברות על בסיס מנגנוני הגנה של שלמות נתונים, המאמתים את פרטי הזהות ומונעים כפל חשבונות. בנוסף, המודל מספק תשתית לניהול רשומות המאפשרת שליפת נתונים ועדכון מאפיינים אישיים ופיזיולוגיים, ומשמש כשער אבטחה לוגי המבטיח שרק משתמשים מורשים יקבלו גישה למשאבי המערכת.

2. מודל ניטור וניתוח סטטיסטי

רכיב תפעולי המלווה את המשתמש בזמן אמת ומעבד את נתוני האימון הפיזי. המודל מנהל את מחזור החיים של הפעילות, קולט באופן רציף את נתוני המיקום המשתנים, ומפעיל מנגנוני בקרת איכות וסינון רעשים כדי להתגבר על סטיות חומרה וחוסר דיוק של חיישנים. בסיום הפעילות, המודל מבצע עיבוד אנליטי וגאודזי מורכב: הוא מחשב את המרחק המצטבר הריאלי על פני כדור הארץ, מפריד את משך הזמן הכולל מזמן הפעילות בפועל (על ידי קיזוז זמני ההפסקות), ומפיק מדדי ביצוע מתקדמים הכוללים קצב תנועה ממוצע, ספירת צעדים ושינויי גובה טופוגרפיים מצטברים, אשר נשמרים בהיסטוריית האימונים.

3. מודל ניתוח מרחבי ותשתיות מיפוי

מודל זה משמש כשלב הכנת הנתונים עבור מנוע החיפוש המרכזי, ותפקידו לבנות את רשת הקשרים הגאוגרפית שעליה יופעלו האלגוריתמים. המודל קולט את מיקום המשתמש ואת אילוץ המרחק שהוגדר, ותוחם מרחב גאוגרפי מותאם המחולק לרשת תאים. עבור אזורים אלו, המודל מתשאל שירותי מפה חיצוניים כדי לאתר נקודות עניין רלוונטיות, משקלל עבורן ציון תגמול המבוסס על מדדי פופולריות ואיכות, ומבצע השלמה והצמדה של נקודות אל רשת הכבישים והשבילים הריאלית

במציאות. לבסוף, המודל מייצר רשת קשרים טופוגרפית ממוקדת ומודד את מרחקי ההליכה והריצה הממשיים בין הנקודות השונות, תוך תמיכה דינמית במסלולים מעגליים.

4. מודל אופטימיזציית מסלול הריצה

הלב החשובי והמחקרי של המערכת, המופקד על פתרון בעיית הניתוב תחת מגבלת תקציב מרחק קשיחה. המודל מריץ אלגוריתם מטא-היוריסטי מבוסס בינה קבוצתית (אינטליגנציית נחיל) הבוחן ומקדם במקביל מגוון מסלולים פוטנציאליים במרחב חיפוש בדיסקרטי באמצעות פעולות שינוי, החלפה והכנסה המושכות את הפתרונות לעבר אזורי רווחיים. המודל מפעיל מנגנון תיקון הבדוק באופן רציף את תקציב המרחק ומבטיח כי המסלול המופק יישאר חוקי – הן עבור מסלולים קווים והן עבור מסלולים מעגליים החוזרים בדיוק לנקודת המוצא. בנוסף, המודל משלב אלגוריתם שיפור מקומי לניתוח גאומטרי ומניעת הצטלבויות דרכים, ולבסוף מתרגם את רצף הצמתים הלוגי לנתיב פיזי רציף ומפותל המוחזר לאפליקציה לצורך הצגה ויזואלית.

8.3 אפיון פונקציונלי וביצועים עיקריים

סעיף זה מגדיר את היכולות המעשיות של המערכת (האפיון הפונקציונלי) לצד מדדי האיכות, היציבות ומהירות העבודה הנדרשים ממנה (ביצועים עיקריים). שילוב זה מבטיח כי האפליקציה תספק מענה מלא לצורכי המשתמש, תוך עמידה בסטנדרטים מחמירים של הנדסת תוכנה וביצועים בזמן אמת.

1. מנגנון הפקת מסלולים חכם (Route Generation Engine)

- **אפיון פונקציונלי:** המערכת מאפשרת למשתמש להגדיר אילוצי מרחק ומיקום, ומפיקה עבורו מסלול אימון מותאם אישית (קווי או מעגלי). המנגנון מתשאל שירותי מפה חיצוניים ברקע, מאתר נקודות עניין רלוונטיות, משקלל את רמת הפופולריות שלהן, ומציג למשתמש נתיב חוקי ובטיחותי המשורטט על גבי המפה במכשיר הנייד.
- **ביצועים עיקריים:** מנוע האופטימיזציה בשרת מחויב לזמן תגובה אפסי של שברירי שניות (Sub-second response time) מרגע שליחת הבקשה ועד להחזרת המסלול המלא. האלגוריתם מבצע מניעה מתמטית מוחלטת של הצטלבויות דרכים או לולאות מיותרות, ומבטיח התכנסות מהירה לפתרון האיכותי ביותר במסגרת תקציב המרחק שהוגדר.

2. מערכת ניטור ומעקב ריצה בזמן אמת (Core Lifecycle & Live Tracking)

- **אפיון פונקציונלי:** האפליקציה מאפשרת למשתמש לשלוט בצורה מלאה במחזור החיים של האימון הפיזי באמצעות פקודות הפעלה, השתתף, חזרה לפעילות ועצירה. בזמן הריצה, המערכת מתקשרת באופן רציף עם חיישן המיקום של המכשיר, דוגמת קואורדינטות גאוגרפיות ומציגה דינמית את מדדי המרחק והזמן הנוכחיים.

- **ביצועים עיקריים:** תהליכי המעקב והמדידה מבוצעים בריצה מקבילית ברקע (Multithreading) כדי למנוע קפיאות או האטה בממשק המשתמש. המערכת מפעילה מסנני חומרה בזמן אמת המנקים רעשי קליטה (כגון התעלמות מסטיות דיוק של מעל 20 מטרים או קפיצות גובה זניחות), ופועלת ביעילות אנרגטית גבוהה השומרת על חיי סוללת המכשיר בתנאי שטח.

3. ניהול נתונים, עיבוד אנליטי וסנכרון (Data Synchronization & Analytics)

- **אפיון פונקציונלי:** המערכת מתעדת ומאחסנת כל אימון ריצה שהסתיים בהצלחה. היא מאפשרת למשתמש לדפדף ברצף היסטוריית האימונים שלו, ולבחור פעילות ספציפית כדי לצפות בניתוח נתונים מורחב ובוויזואליזציה גרפית עשירה של מגמות שיפור, קצבי תנועה ומדדים גאודזיים מצטברים.
- **ביצועים עיקריים:** המערכת מבטיחה שלמות נתונים (Data Integrity) מלאה בעת העברת המידע בין הלקוח, השרת ומסד הנתונים בענן, ומבצעת סנכרון אוטומטי ומהיר מול הענן ברגע שמתחדש החיבור לרשת, תוך שמירה על זמני שליפה וכתובה מינימליים בבסיס הנתונים.

4. מערך אימות והרשאות משתמשים (User Access & Authentication)

- **אפיון פונקציונלי:** המערכת מנהלת את שער הכניסה לאפליקציה באמצעות רכיבי רישום משתמשים חדשים והתחברות מאובטחת של משתמשים קיימים על בסיס דואר אלקטרוני וסיסמה. בנוסף, המערכת מאפשרת למשתמש לצפות במסך הפרופיל שלו ולעדכן באופן דינמי מאפיינים אישיים ופיזיולוגיים.
- **ביצועים עיקריים:** תהליכי אימות הפרטים והרשאות הגישה מבוצעים במהירות מול השרת, תוך הפעלת מנגנוני הגנה המונעים רישום כפול של חשבונות בשכבת הנתונים. המידע האישי מוגן ומנוהל בענן בצורה מבודדת, המונעת גישה או זליגת נתונים לגורמים שאינם מורשים.

8.4 אילוצי המערכת

אילוצי המערכת מייצגים את המגבלות הפיזיות, הלוגיות והטכנולוגיות הקשיחות שמתקיימות בסביבת העבודה של האפליקציה. אילוצים אלו מכתיבים את גבולות הגזרה של הפיתוח, ואינם נתונים לשליטתו של המתכנת:

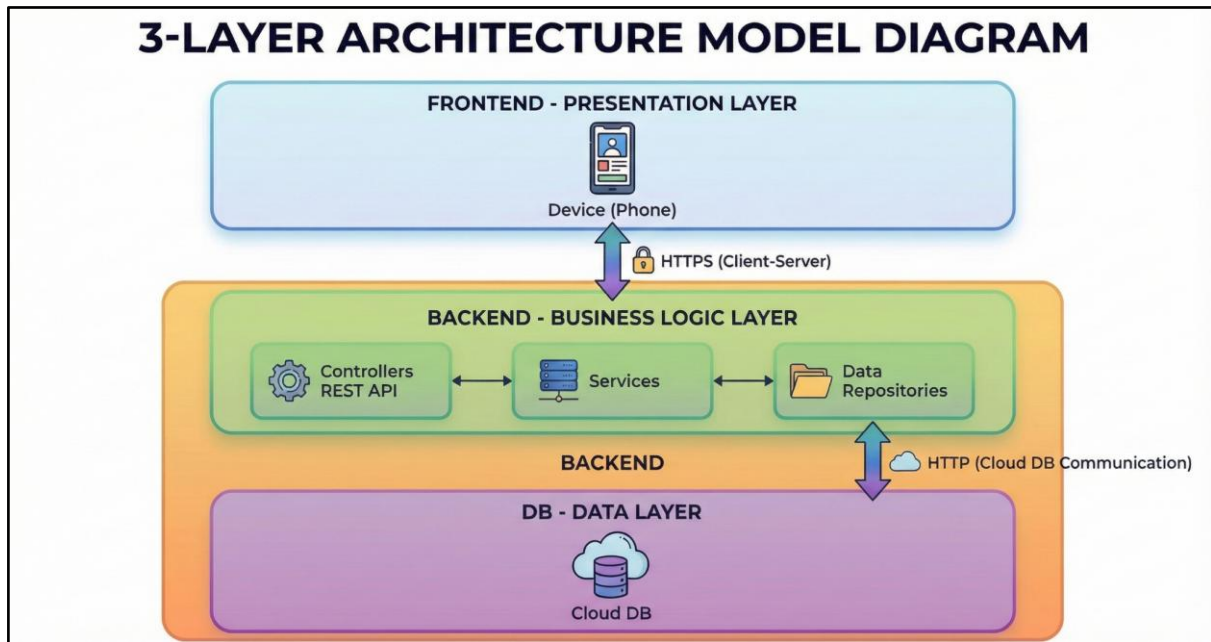
- **אילוץ טווח מרחק האימון (Distance Selection Constraint):** חופש הבחירה של המשתמש מוגבל מראש לטווח מרחקים הגיוני ובר-ביצוע עבור פעילות גופנית של ריצה או הליכה. אילוץ זה מהווה את נקודת המוצא של המערכת כולה, שכן בחירת המרחק מכתובה באופן אוטומטי ודינמי את רדיוס החיפוש הפיזי בשטח ואת כמות הנתונים שייקלטו.

- **אילוח גודל הגרף ומכסת נקודות העיבוד (Graph Size and Candidate Constraint):**
כפועל יוצא ישיר מאילוח המרחק, קיים גבול עליון קשיח לכמות נקודות המועמדים והצמתים הגאוגרפיים שניתן לשלוח לעיבוד במנוע האופטימיזציה בכל סבב. הגבלת כמות הנקודות היא אילוח טכני הכרחי שנועד למנוע מצב שבו מרחב האפשרויות גדל בצורה מעריכית (אקספוננציאלית) המעכבת את פעולת השרת.
- **אילוח מכסת קריאות מוגבלת לשירותי מיפוי חיצוניים (External API Quotas Constraint):** מאחר ומערכת השרת נשענת על פלטפורמת ענן חיצונית לשליפת נתוני הדרכים והמרחקים, הפעילות כפופה לחלוטין למגבלות שימוש, נפח שאילתות ומכסות חינמיות (Free Tier) קשיחות שמציב ספק השירות. אילוח זה מחייב את המערכת לבצע אופטימיזציה קפדנית של תדירות וכמות הפניות לרשת החיצונית, ולצמצם את קשרי הגרף לתוואי ממוקד בלבד, כדי למנוע חריגה מהמכסה המותרת והשבתה של השירות.
- **אילוח אמינות ורציפות נתוני המיקום (Hardware Tracking Constraint):** המערכת תלויה לחלוטין ברכיב חומרת ה-GPS של המכשיר הנייד וביכולת קליטת אות הלוויינים הפיזית שלו בשטח. אילוח זה מכניס אלמנט של חוסר ודאות למערכת, שכן תנאים סביבתיים משבשים את רציפות ודיוק המידע הגולמי, ומאלצים את האפליקציה לבצע סינונים והערכות גאודזיות מבוססות חומרה בלבד.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת

האפליקציה תיבנה לפי מודל 3 השכבות על מנת לאפשר הפרדה, סדר וחוסר תלות בין המשתמש, השרת ומסד הנתונים:



התרשים מציג ארכיטקטורת Layer-3, שבה המערכת מחולקת לשלושה אזורים עיקריים: Frontend, Backend ו-Database, כאשר ה-Backend משמש כצד השרת כולו ומכיל בתוכו שתי שכבות פנימיות נפרדות – שכבת הלוגיקה העסקית ושכבת הנתונים.

שכבת ה-Frontend – Presentation Layer מייצגת את צד הלקוח, ובמקרה זה מכשיר קצה כגון טלפון נייד. שכבה זו אחראית על הצגת המידע למשתמש, קבלת קליטים ויזואליים וביצוע פעולות משתמש. היא אינה מבצעת חישובים עסקיים ואינה ניגשת למסד הנתונים באופן ישיר, אלא פועלת מול השרת בלבד. התקשורת בין ה-Frontend ל-Backend מתבצעת באמצעות פרוטוקול HTTPS במודל Client-Server, דבר המבטיח העברת מידע מאובטחת ומוצפנת.

אזור ה-Backend מייצג את כל צד השרת של המערכת, והוא כולל בתוכו את שכבת ה-Business Logic וכן את שכבת ה-Data. בתוך שכבת הלוגיקה העסקית נמצאים רכיבי ה-Controllers המשמשים כנקודת הכניסה לבקשות המגיעות מה-Frontend. ה-Controllers מקבלים בקשות HTTP, מבצעים בדיקות בסיסיות ומעבירים את הטיפול לרכיב ה-Services. רכיב ה-Services מכיל את הלוגיקה העסקית עצמה – חוקים, תהליכים וחישובים המגדירים את אופן פעולת המערכת. רכיב ה-Data Repositories אחראי על ניהול הגישה לנתונים, ומתווך בין הלוגיקה העסקית לבין מסד הנתונים, תוך הסתרת פרטי המימוש של שכבת הנתונים מהשכבות הגבוהות יותר.

שכבת ה-DB – Data Layer, אשר ממוקמת גם היא בתוך תחום ה-Backend, אחראית על אחסון הנתונים באופן קבוע. בתרשים היא מיוצגת כ-Cloud DB, כלומר מסד נתונים המתארח בענן. התקשורת בין רכיב ה-Data Repositories לבין מסד הנתונים מתבצעת באמצעות תקשורת רשת (כגון HTTP או פרוטוקול ייעודי למסדי נתונים בענן), ומאפשרת שליפה, עדכון ושמירה של מידע בצורה מבוקרת ומאובטחת.

באופן זה, ה-Backend מהווה יחידה שלמה המטפלת הן בלוגיקה העסקית והן בניהול הנתונים, בעוד שה-Frontend משמש אך ורק כממשק משתמש. חלוקה זו יוצרת הפרדה ברורה בין צד הלקוח לצד השרת, משפרת אבטחה, מאפשרת תחזוקה קלה יותר, ותומכת בהרחבה עתידית של המערכת ללא תלות ישירה בין שכבות שונות.

9.2 מודל שרת לקוח

מודל שרת-לקוח (Client-Server Architecture) הוא מודל ארכיטקטוני מבוסס לעיבוד נתונים והנדסת תוכנה, המבוסס על חלוקת תפקידים ברורה והפרדת סמכויות מוחלטת (Separation of Concerns) בין ספק השירות (השרת) לבין צרכן השירות (הלקוח).

הארכיטקטורה פועלת במנגנון מובנה של בקשה ותגובה (Request-Response), שבו רכיב הלקוח יוזם פנייה אל השרת באמצעות פרוטוקול תקשורת מוגדר מראש, והשרת מאזין לפניות, מעבד אותן, מפעיל לוגיקה עסקית ומחזיר ללקוח תגובה מתאימה המכילה את תוצרי העיבוד. תפיסה הנדסית זו מאפשרת להפריד באופן מלא בין שכבת התצוגה וממשק המשתמש לבין שכבת החישוב, ניהול מסדי הנתונים והאלגוריתמיקה הכבדה, ובכך למקסם את יעילות המערכת, אמינותה ויכולת ההרחבה שלה בסביבה מבוססת וניידת.

רכיב הלקוח (Client) במערכת מיושם כאפליקציית מובייל ייעודית, אשר תפקידה המרכזי הוא לנהל את האינטראקציה הישירה מול המשתמש, להאזין לקלט ממנו ולספק חוויית שימוש ויזואלית, זורמת ונגישה. הלקוח מופקד על השכבה הניתוחית הפיזית בצד המשתמש, הכוללת את הפעלת חיישני המיקום המובנים של המכשיר הנייד לצורך איסוף נתונים גאו-מרחביים בזמן אמת במהלך הפעילות. כמו כן, הלקוח אחראי על הצגה ויזואלית וגרפית של המידע הגולמי והמעובד, כגון שרטוט מסלולים על גבי מפות דינמיות והצגת נתונים סטטיסטיים ומדדי אימון מגוונים. מבחינה ארכיטקטונית, הלקוח נמנע מביצוע חישובים לוגיים מורכבים או עיבודי נתונים עצימים; כאשר נדרשת פעולה מורכבת כמו תכנון מסלול אופטימלי, הלקוח אורז את הפרמטרים והאילוצים שהגדיר המשתמש לכדי בקשת נתונים רשמית, שולח אותה אל השרת המרכזי וממתין לקבלת התוצרים כדי להציג אותם בצורה ברורה על מסך המכשיר.

רכיב השרת (Server) משמש בתור הליבה הלוגית ושכבת עיבוד הנתונים המרכזית של המערכת כולה. השרת פועל כרכיב עצמאי ומאזין באופן קבוע לפניות ובקשות המגיעות מצד אפליקציית המובייל. עם קבלת בקשה מהלקוח, השרת מנתב את המידע, מפעיל את הלוגיקה העסקית המתאימה ומבצע את כל העיבודים החשובים והאלגוריתמיים המורכבים של הפרויקט. השרת מופקד על הרצת מנוע האופטימיזציה ומציאת המסלולים תחת אילוצים, ניתוח נתונים מרחביים וחישובי מדדים סטטיסטיים מצטברים. בנוסף, השרת מנהל את הגישה הישירה אל שכבת הנתונים (Data Layer) ומד הנתונים

המרכזי, לצורך שמירה, עדכון ושליפה של פרופילי משתמשים, היסטוריית פעילות ונתונים אישיים, ומחזיר את התוצאות המעובדות ללקוח בפורמט אחיד ומובנה.

התקשורת והחיבור בין הלקוח לשרת מבוצעים באמצעות ארכיטקטורת רשת מודרנית (REST API) מבוססת פרוטוקול HTTP, כאשר חילופי המידע מתבצעים בפורמט JSON הסטנדרטי, המבטיח העברת נתונים מהירה, קלה ומאובטחת לשני הרכיבים.

לחלוקה זו במודל שרת-לקוח ישנן סיבות הנדסיות מכריעות:

- העברת עומס החישוב של אלגוריתמי האופטימיזציה הכבדים מהטלפון הנייד אל שרת ייעודי חוסכת בצורה דרסטית במשאבי מעבד, זיכרון וסוללה של מכשיר הקצה ומבטיחה עבודה חלקה ויציבה של האפליקציה בזמן הריצה.
- ריכוז ניהול הנתונים והלוגיקה בשרת מרכזי מבטיח שלמות נתונים (Data Integrity) ואבטחת מידע גבוהה בענן, שכן המידע אינו נשמר מקומית על המכשיר בצורה פגיעה.
- מודל זה מעניק למערכת מודולריות רבה, המאפשרת לעדכן, לשפר ולתקן את האלגוריתמים והלוגיקה בצד השרת מבלי להידרש לעדכון גרסה של האפליקציה אצל משתמשי הקצה, ולהיפך, מה שמקל באופן ניכר על תחזוקת הפרויקט והרחבתו בעתיד.

9.3 תהליכים של מע' ההפעלה

אין.

9.4 תיאור פרוטוקולי תקשורת

פרוטוקול TCP/IP (Transmission Control Protocol / Internet Protocol)

פרוטוקול זה מהווה את תשתית התקשורת היסודית והמרכזית ביותר ברשת האינטרנט, המקשרת בין שכבת הרשת (IP) לשכבת התעבורה (TCP). תפקידו המרכזי של הפרוטוקול במערכת מבוזרת הוא להבטיח ערוץ תקשורת אמין, יציב ומכוון-חיבור (Connection-Oriented) בין יישום הלקוח לבין השרת המרוחק. פרוטוקול ה-IP אחראי על ניתוב חבילות המידע (Packets) דרך צמתי הרשת השונים באמצעות כתובות לוגיות, בעוד פרוטוקול ה-TCP מופקד על בקרת הזרימה, וידוא הגעת הנתונים וסידורם בסדר הנכון והמדויק בצד המקבל. מנגנון זה מבצע תהליך הקמה קשיח (Three-Way Handshake) ומפעיל מנגנוני אימות ותיקון שגיאות אוטומטיים למקרה של אובדן חבילות מידע במהלך השילוח ברשת. בסביבת שרת-לקוח, פרוטוקול TCP/IP משמש כ"צינור" הפיזי והלוגי שעל גביו מועברים פרוטוקולי השכבות העליונות יותר, והוא מבטיח כי כל בקשה או תגובה תגיע ליעדה בשלמותה וללא עיוותים, ללא קשר לאיכות החיבור או לתנאי הרשת המשתנים של מכשיר הקצה.

פרוטוקול HTTP (Hypertext Transfer Protocol)

פרוטוקול HTTP פועל בשכבת האפליקציה (Application Layer) של מותאם הרשת ונבנה ישירות מעל תשתית התעבורה של TCP/IP. זהו פרוטוקול סטנדרטי המנהל את הלוגיקה המערכתית של חילופי המידע, והוא מבוסס על מודל בקשה-תגובה (Request-Response) שהוא חסר מצב (Stateless) – כלומר, כל פנייה של הלקוח לשרת מטופלת כאירוע עצמאי ומבודד ללא תלות בפניות קודמות. במסגרת הארכיטקטורה המבוזרת, פרוטוקול זה משמש כבסיס ליישום ממשק התקשורת המרכזי (REST API). הלקוח יוזם קריאות HTTP אל עבר נקודות קצה מוגדרות בשרת (Endpoints) באמצעות מתודות תקניות (HTTP Methods) המגדירות באופן חד-משמעי את מהות הפעולה הנדרשת: מתודת GET משמשת לשליפה ומשיכת נתונים ומסמכים מהשרת, מתודת POST משמשת ליצירת ישויות חדשות ולשליחת מידע מורכב וכבד בגוף הפנייה, ומתודות PUT או PATCH משמשות לצורך עדכון והחלפה של ישויות קיימות. השימוש ב-HHTTP מאפשר ניהול מובנה של תעבורת הנתונים, סיווג אופי הפעולות, וטיפול תקני ומאורגן בשגיאות מערכת או שגיאות לוגיות באמצעות קודי תגובה מוסכמים (HTTP Status Codes), דבר המייצר הפרדה ברורה בין תקלות תקשורת חומרתיות לתקלות לוגיות בצד השרת.

פורמט ייצוג הנתונים (JSON (JavaScript Object Notation

כדי שהמידע המועבר על גבי פרוטוקול HTTP יובן, יפוענח ויטופל כהלכה בשני קצוות המערכת, נעשה שימוש ב-JSON כפרוטוקול וכפורמט עצמאי לחילוף נתונים (Data Exchange Format). JSON הוא פורמט טקסטואלי קליל (Lightweight) המבוסס על מבנה מוסכם וקל להבנה של זוגות מפתח-ערך (Key-Value Pairs) ורשימות סדורות, והוא אינו תלוי בשפת תכנות או בפלטפורמת פיתוח מסוימת. תפקידו המרכזי במערכת מבוזרת הוא לשמש כ"שפה המשותפת" והאוניברסלית בין הלקוח לשרת; לפני שליחת מידע מורכב ברשת, אובייקטי התוכנה המקומיים עוברים תהליך של סיראליזציה (Serialization) ומומרים למחרוזת טקסט אחידה בפורמט JSON הממוקמת בגוף ההודעה. בצד המקבל, המערכת מבצעת תהליך הפוך של פרסינג (Parsing / Deserialization) ומשחזרת את הטקסט לאובייקטים לוגיים המוכרים לשפת התכנות המקומית.

9.5 ארכיטקטורת רשת

אין.

9.6 שירותים חיצוניים

Places API

שירות ה-Places API הוא ממשק רשת המאפשר תשאול, איתור ושליפה של מידע מקיף אודות נקודות עניין (Points of Interest), עסקים, מיקומים גאוגרפיים וכתובות פיזיות ברחבי העולם. השירות מאפשר למערכת לבצע חיפושים מבוססי רדיוס או מיקום, ומחזיר נתונים עשירים הכוללים קואורדינטות מדויקות, סיווגים קטגוריאליים ושמות של אתרים במרחב הווירטואלי. בארכיטקטורה של

מערכות ניתוב ואופטימיזציה, שירות זה משמש כתשתית הנתונים הראשונית לצורך הגדרת מרחב החיפוש. הוא מאפשר לקחת מיקום גאוגרפי מופשט של משתמש ולזהות בסביבתו את כל נקודות הציון הרלוונטיות שיכולות להוות יעדים פוטנציאליים לביקור, ובכך הוא מייצר את מאגר הצמתים הבדיד (הגרף) שעליו יופעלו בהמשך האלגוריתמים המתמטיים.

Roads API

שירות ה-Roads API הוא ממשק ייעודי המיועד לפתרון אתגרים הנדסיים הקשורים במסלולי תנועה וברשתות כבישים פיזיות, תוך דגש על עיבוד נתוני מיקום שנאספו מחיישני חומרה (כמו GPS). האתגר המרכזי בעבודה עם קואורדינטות גולמיות ממכשירים ניידים הוא קיומם של רעשים סטטיסטיים, סטיות קליטה וחוסר דיוק המתרחשים לרוב בסביבות עירוניות צפופות. שירות ה-Roads API מתמודד עם קושי זה באמצעות תכונת "הצמדה לכביש" (Snap to Roads), המקבלת רצף של קואורדינטות מפוזרות ומחשבת באופן הסתברותי את הנתיב הריאלי והחוקי הקרוב ביותר שעליו התבצעה התנועה בפועל. שימוש בשירות זה חיוני לצורך ניקוי וטיוב המידע המרחבי, ומבטיח שהמערכת תתבסס על נתוני אמת המייצגים שבילים ודרכים קיימות, ולא על קווים שבורים או שגויים שנוצרו עקב שיבושי קליטה.

Distance Matrix API

שירות ה-Distance Matrix API הוא מנוע חישוב מרחבי המספק מטריצה של מרחקים חזויים עבור קבוצה מוגדרת של נקודות מוצא ונקודות יעד. במקום לבצע חישוב גאומטרי מלא ומורכב של נתיבי נסיעה או הליכה, השירות מייצר אופטימיזציה של המידע ומחזיר רק את ערכי ה"עלות" הפיזיים (במטרים) בין כל זוג נקודות ברשת על בסיס רשת הכבישים המומלצת. שירות זה מהווה את לב התשתית המתמטית עבור פתרון בעיות ניתוב קומבינטוריות (כמו בעיית הסוכן הנוסע או בעיית ה-Orienteering). מאחר ואלגוריתמים מטא-היוריסטיים נדרשים להעריך אלפי שילובים של מסלולים כדי לקבוע מהו הנתיב הרווחי ביותר, הם זקוקים למטריצת עלויות קבועה ויציבה. השירות מספק את המטריצה הזו ישירות ללוגיקה העסקית בשרת, ובכך מאפשר לאלגוריתם לבצע חישובי התכנסות ופונקציות תגמול במהירות מרבית ובחוסן מתמטי מלא.

Directions API

שירות ה-Directions API הוא ממשק רשת מורכב המיועד לתכנון, חישוב והפקה של הוראות ניווט ומסלולי נסיעה או הליכה מפורטים בין נקודות ציון שונות במפה. השירות מקבל קלט של נקודת מוצא, נקודת סיום, וסדרה של נקודות ביקור מוגדרות (Waypoints), ומחשב את המסלול הריאלי המיטבי תוך התחשבות באילוצי דרך, פניות, ותנאי שטח אמיתיים. החשיבות הארכיטקטונית של שירות זה באה לידי ביטוי בשלב הסופי של תהליך האופטימיזציה; לאחר שאלגוריתם החיפוש בשרת סיים את פעולתו וקבע מהו רצף הצמתים הלוגי הטוב ביותר, שירות ה-Directions API מתרגם את הפתרון המתמטי המופשט לנתיב פיזי ממשי. הוא מחזיר קו רציף ומפותל (Polyline) המורכב מאלפי

קואורדינטות עוקבות העוקבות במדויק אחר תוואי השטח, ובכך מאפשר למערכת לשלוח לצד הלקוח ישות מסלול שלמה ומושלמת הניתנת להצגה ויזואלית ברור על גבי מסך המכשיר.

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

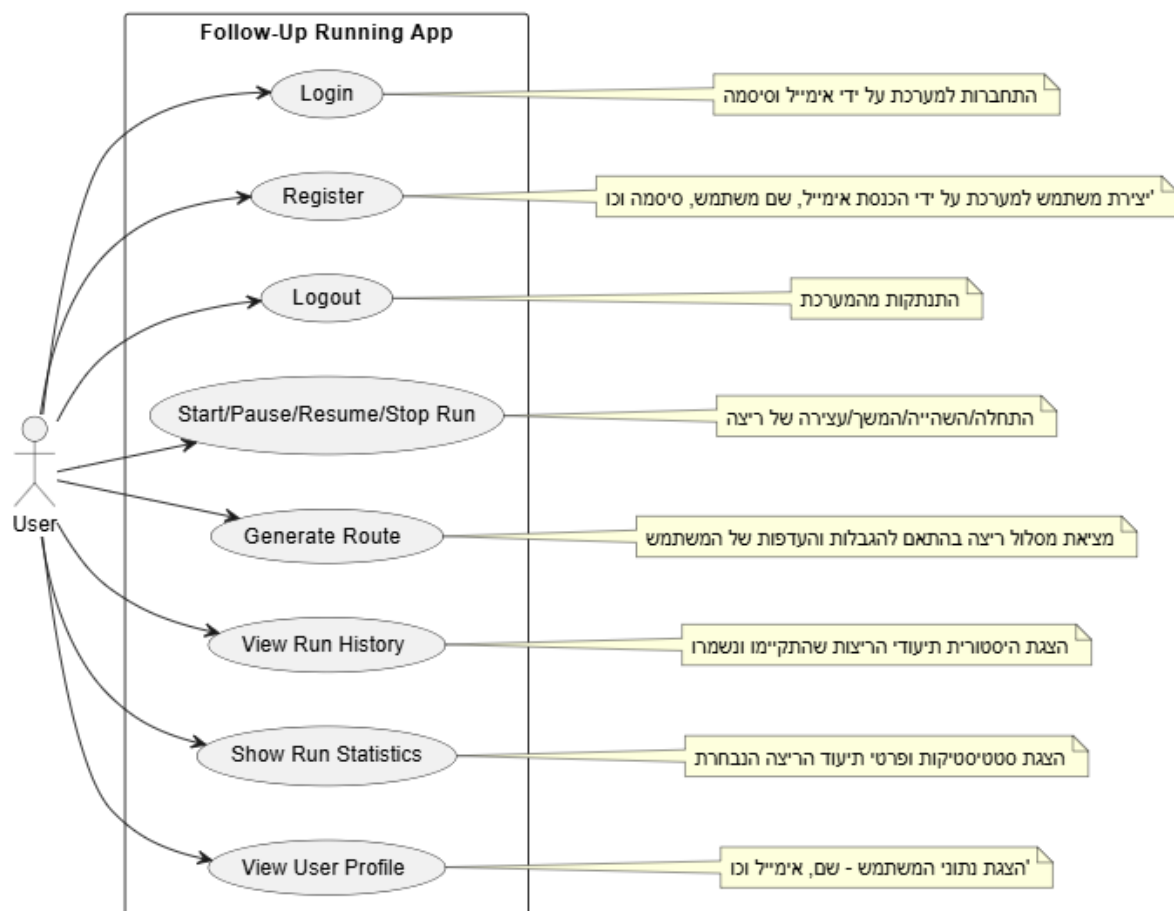
10.1 תרשימי תרחיש Use case Diagram

תרשים מקרי השימוש (UCD) מציג את נקודת המבט הפונקציונלית של המערכת ומגדיר את גבולותיה, על ידי איפיון האינטראקציות המתקיימות בין הגורמים המשתמשים במערכת (Actors) לבין הפעולות השונות שהיא מאפשרת לבצע.

בתרשים זה מוגדר שחקן מרכזי אחד:

- **המשתמש (User / Runner):** ספורטאי או רץ חובב האינטראקטיבי מול אפליקציית המובייל, שעושה שימוש במערכת כדי לתכנן מסלולים, לנהל את אימוניו ולעקוב אחר התקדמותו.

להלן תרשים מקרי השימוש המציג את מרחב היכולות והאפשרויות שהמערכת מעמידה לרשות המשתמש:



להלן הסבר ממוקד עבור כל מקרה שימוש (Use Case) המופיע בתרשים:

- **Login (התחברות למערכת) –** מאפשר למשתמש רשום להיכנס בבטחה לחשבון באפליקציה באמצעות הזנת כתובת אימייל וסיסמה, לצורך גישה מאובטחת לנתוני האישיים השמורים.
- **Register (הרשמה למערכת) –** מאפשר למשתמש חדש ליצור פרופיל באפליקציה על ידי הכנסת פרטי מפתח (שם משתמש, אימייל וסיסמה), ובכך להקים ישות משתמש חדשה בבסיס הנתונים.
- **Logout (התנתקות מהמערכת) –** מאפשר למשתמש לסיים את הסשן הנוכחי שלו ולהתנתק מהחשבון.
- **Start/Pause/Resume/Stop Run (ניהול הריצה) –** תהליך הליבה האופרטיבי המאפשר למשתמש לשלוט במחזור החיים של האימון בזמן אמת – החל מהפעלת מעקב המיקום (GPS), דרך השעייתו והמשכתו במידת הצורך, ועד לעצירתו המוחלטת לצורך סיכום ושמירה.
- **Generate Route (הפקת מסלול מותאם) –** מאפשר למשתמש להגדיר אילוצים והעדפות (כמו מרחק רצוי), ומפעיל מאחורי הקלעים את אלגוריתם האופטימיזציה בשרת על מנת להציג עבורו מסלול ריצה חכם ומותאם אישית על גבי המפה.
- **View Run History (צפייה בהיסטוריית הריצות) –** מאפשר למשתמש לגשת לפרגמנט ייעודי המרכז ומציג בצורה סדורה את כל תיעודי הריצות הקודמות שביצע והתקיימו בעבר במערכת.
- **Show Run Statistics (הצגת סטטיסטיקות ריצה) –** מאפשר למשתמש לבחור אימון ספציפי מתוך ההיסטוריה ולצפות בניתוח נתונים ויזואלי ומורחב של אותה ריצה (כמו גרפים של קצבים, מהירות, זמנים ומרחקים מצטברים).
- **View User Profile (צפייה בפרופיל משתמש) –** מאפשר למשתמש לצפות במסך הפרופיל שלו, המציג את נתוני האישיים והפיזיולוגיים (כמו שם, אימייל ונתונים אישיים), ומקנה לו את היכולת לעדכן ולשנות אותם מול השרת וכלל שכבות המערכת.

10.2 תרשימי רצף Sequence Diagram

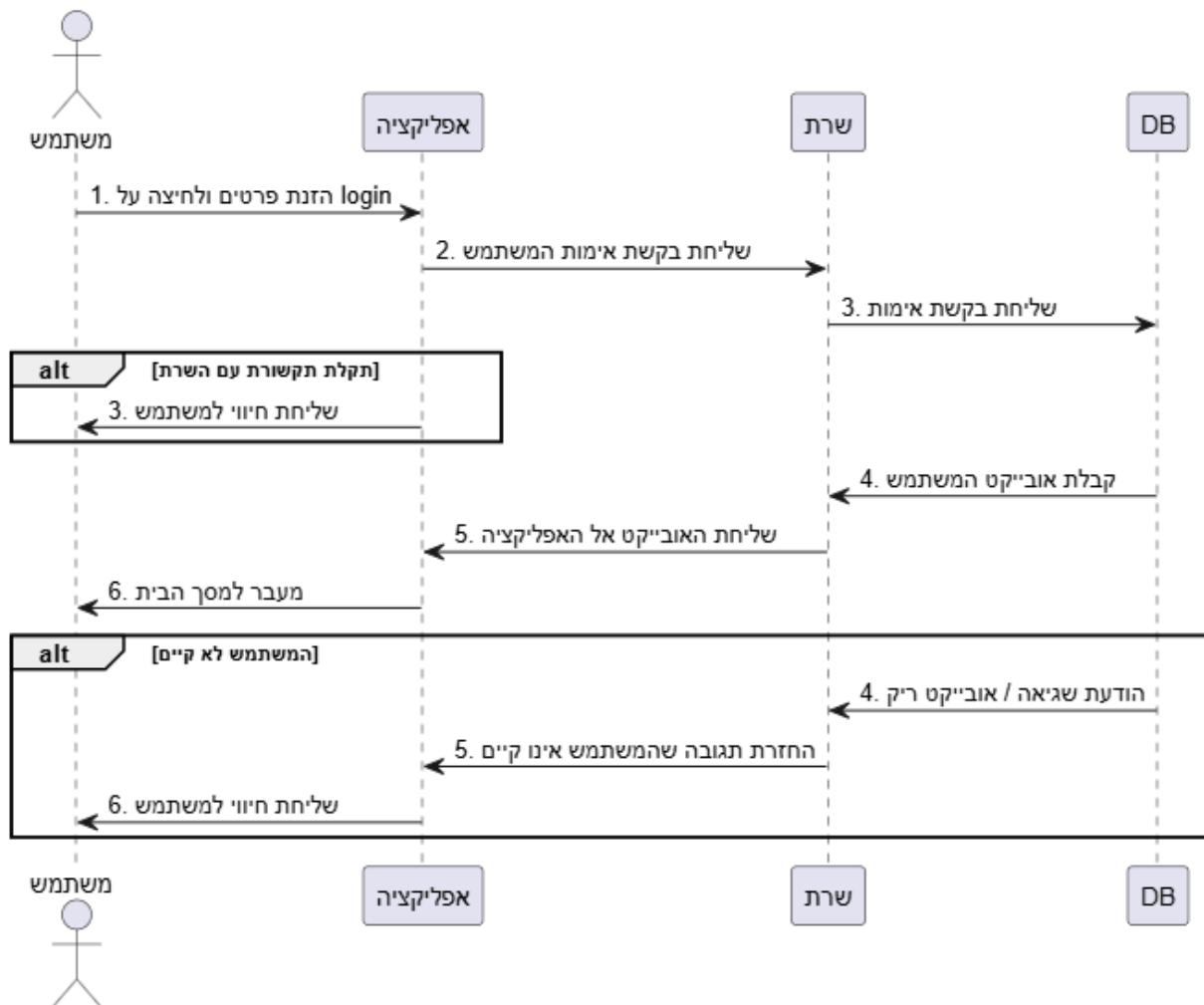
תרשימי הרצף (Sequence Diagrams) מייצגים את המבט הדינמי וההתנהגותי של התוכנה. מטרתם של תרשימים אלו היא לתאר את תזרים האירועים ומחזור החיים של תהליכים מרכזיים במערכת לאורך ציר הזמן, תוך פירוט של סדר העברת ההודעות, הקריאות לפונקציות וחילופי המידע בין הרכיבים השונים (האובייקטים) המרכיבים את הארכיטקטורה המבוצרת.

בתרשימי הרצף של המערכת מוגדרים חמישה קווי חיים (Lifelines) מרכזיים המשתתפים בתהליכים השונים:

- **המשתמש** – הגורם האנושי המפעיל את האפליקציה ומייצר אירועים (כמו לחיצה על כפתור).
- **האפליקציה** – רכיב הקצה המנהל את ממשק המשתמש, קולט קלט, מפעיל חיישנים ומבצע קריאות רשת.
- **השרת** – הליבה הלוגית המאזינה לבקשות ה-HTTP, מפעילה את שכבות ה-Service ומריצה את אלגוריתמי האופטימיזציה.
- **מסד הנתונים (DB)**: שכבת האחסון האחראית על שמירה, שליפה ועדכון של ישויות המערכת ברמת הענן.
- **שירותים חיצוניים (Google REST)**: ממשקי רשת חיצוניים המספקים נתונים גאו-מרחביים וניתוחי דרכים (Places, Distance Matrix, Directions וכו').

להלן פירוט של תרשימי הרצף בעבור תהליכי הליבה והפעולות המרכזיות במערכת:

❖ התחברות ואימות משתמש (Login Sequence Diagram)



תרשים רצף זה מתאר את תהליך ההתחברות והאימות של המשתמש במערכת (Login Flow), ומציג את האינטראקציה הדינמית בציר הזמן בין ארבעת רכיבי הארכיטקטורה: המשתמש, אפליקציית המובייל (הלקוח), השרת המרכזי ומסד הנתונים (DB).

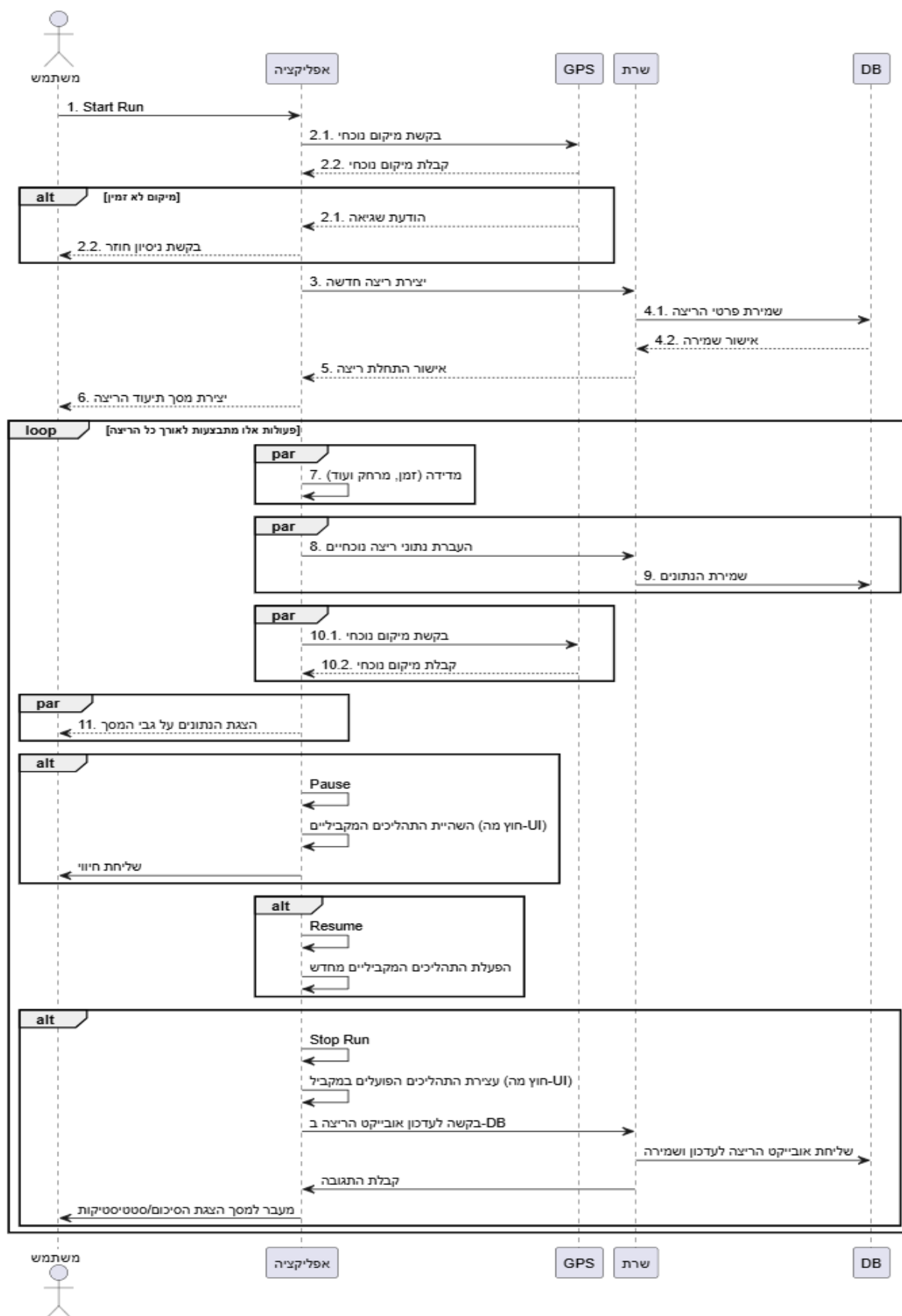
הזרימה המרכזית (המסלול התקין):

1. הזנת פרטים: המשתמש מזין אימייל וסיסמה ומבצע לחיצה על כפתור ההתחברות.
2. בקשת אימות: האפליקציה שולחת בקשת אימות לשרת המרכזי, אשר מעביר את שאלת הבדיקה הלאה אל מסד הנתונים.
3. החזרת הנתונים ומעבר מסך: מסד הנתונים מאתר את הרשומה התואמת ומחזיר את אובייקט המשתמש לשרת, שמנבא אותו בחזרה לאפליקציה. עם קבלת האובייקט, האפליקציה מאשרת את הזהות ומעבירה את המשתמש למסך הבית.

ניהול חריגות ומצבי קצה (בלוקי Alt):

- תקלת תקשורת עם השרת: במידה ומתרחשת בעיית רשת או חוסר זמינות של השרת, האפליקציה מזהה זאת מיד ומציגה חיווי והתרעת שגיאה מתאימה למשתמש על גבי המסך.
- המשתמש לא קיים: אם הוכנסו פרטים שגויים או שהמשתמש אינו רשום, מסד הנתונים מחזיר הודעת שגיאה (או אובייקט ריק) לשרת. השרת מעביר את סטטוס הכישלון לאפליקציה, והיא מציגה למשתמש חיווי ויזואלי המודיע לו כי המשתמש אינו קיים במערכת.

❖ ניהול ומעקב ריצה בזמן אמת (Run Lifecycle Sequence Diagram)



תרשים רצף זה מתאר את מחזור החיים המלא של מעקב הריצה במערכת. הוא מציג את תזרים האירועים בציר הזמן ואת חילופי ההודעות בין המשתמש, אפליקציית המובייל, רכיב ה-GPS, השרת המרכזי ומסד הנתונים (DB).

שלב התחלת הריצה (Start Run):

1. **בקשת מיקום ואתחול:** המשתמש מפעיל את הריצה, והאפליקציה פונה לחייושן ה-GPS לקבלת מיקום ראשוני. אם המיקום אינו זמין (בלוק Alt), מוחזרת שגיאה והמשתמש מתבקש לנסות שוב.

2. **יצירת הישות ב-DB:** עם קבלת המיקום, האפליקציה פונה לשרת ליצירת ישות ריצה חדשה. השרת שומר את פרטי האימון ההתחלתיים ב-DB ומחזיר אישור לאפליקציה, המציגה את מסך התיעוד הדינמי.

שלב המעקב והמידה בזמן אמת (Live Tracking Loop):

לאורך כל הפעילות מתבצעת לולאה מחזורית (Loop) המנהלת מספר תהליכים במקביל (בלוקי Par):

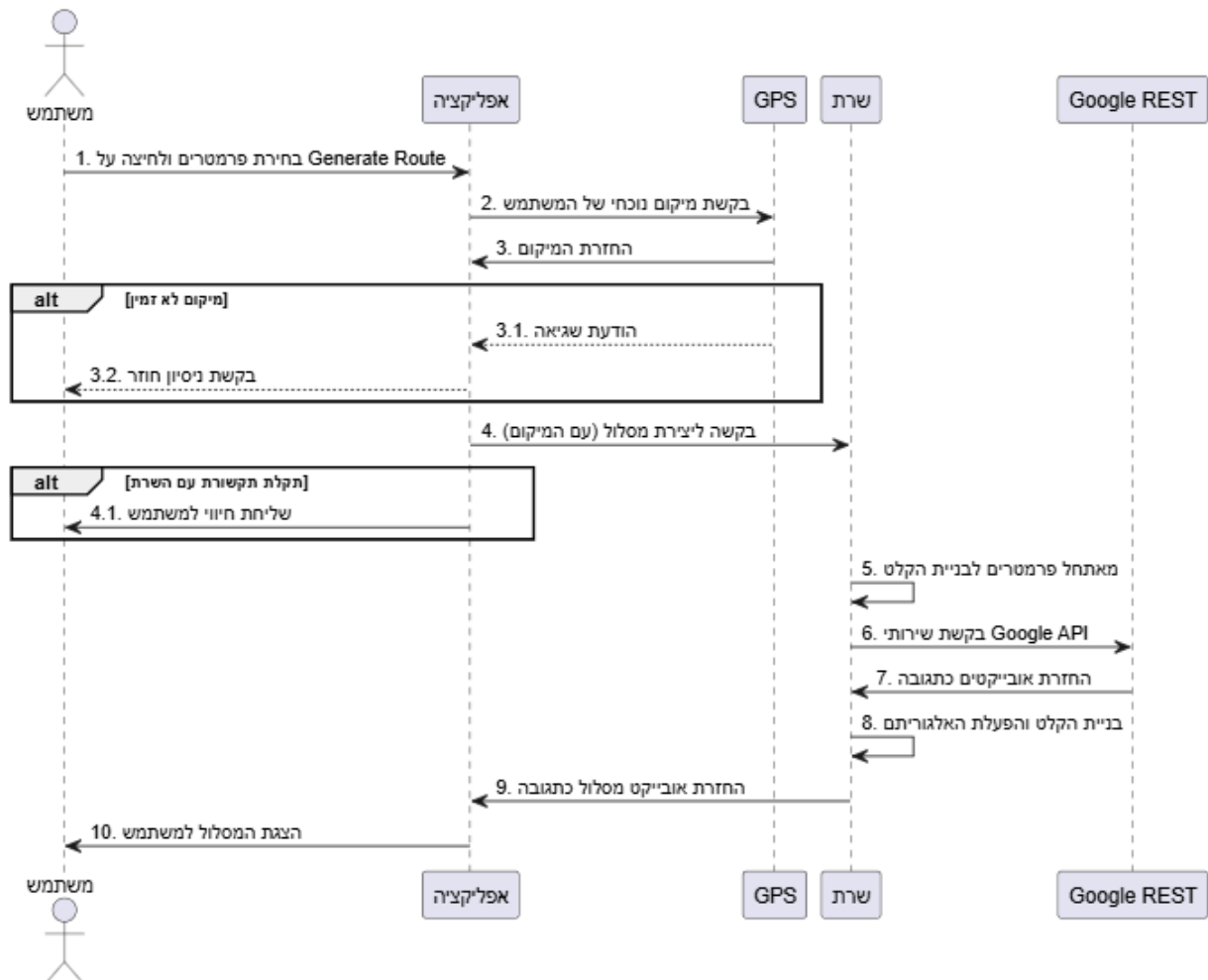
- האפליקציה מודדת עצמאית מדדי זמן ומרחק.
- נתוני הריצה הנוכחיים נשלחים ברקע לשרת ונשמרים ב-DB לצורך גיבוי וסנכרון.
- חייושן ה-GPS דוגם קואורדינטות חדשות ומעבירן לאפליקציה כדי לעדכן את תוואי הדרך.
- האפליקציה מעבדת את כלל המידע ומציגה אותו למשתמש בזמן אמת על גבי המסך.

מנגנוני בקרה ועצירה (בלוקי Alt בתוך הלולאה):

- **השהיה (Pause):** מקפיא את תהליכי המדידה והעברת הנתונים ברקע (למעט ממשק המשתמש) ושולח חייווי ויזואלי לרץ.
- **חזרה לפעילות (Resume):** מפעיל מחדש ובאופן מיידי את כלל תהליכי הניטור והמעקב המקביליים.
- **עצירת הריצה (Stop Run):** מפסיק את חומרת ה-GPS ותהליכי המעקב, שולח את אובייקט הריצה הסופי והמלא לעדכון ושמירה סופית בשרת וב-DB, ומעביר את המשתמש למסך סיכום האימון והסטטיסטיקות המורחבות.

❖ הפקת מסלול מותאם (Generate Route Sequence Diagram)

תרשים רצף זה מתאר את תזרימים העבודה והעברת ההודעות בציר הזמן בעת בקשת המשתמש להפיק מסלול ריצה אופטימלי ומותאם אישית. התהליך ממחיש את השילוב בין חומרת המכשיר, שרת הלוגיקה ושירותי הרשת החיצוניים.



הזרימה המרכזית (המסלול התקין):

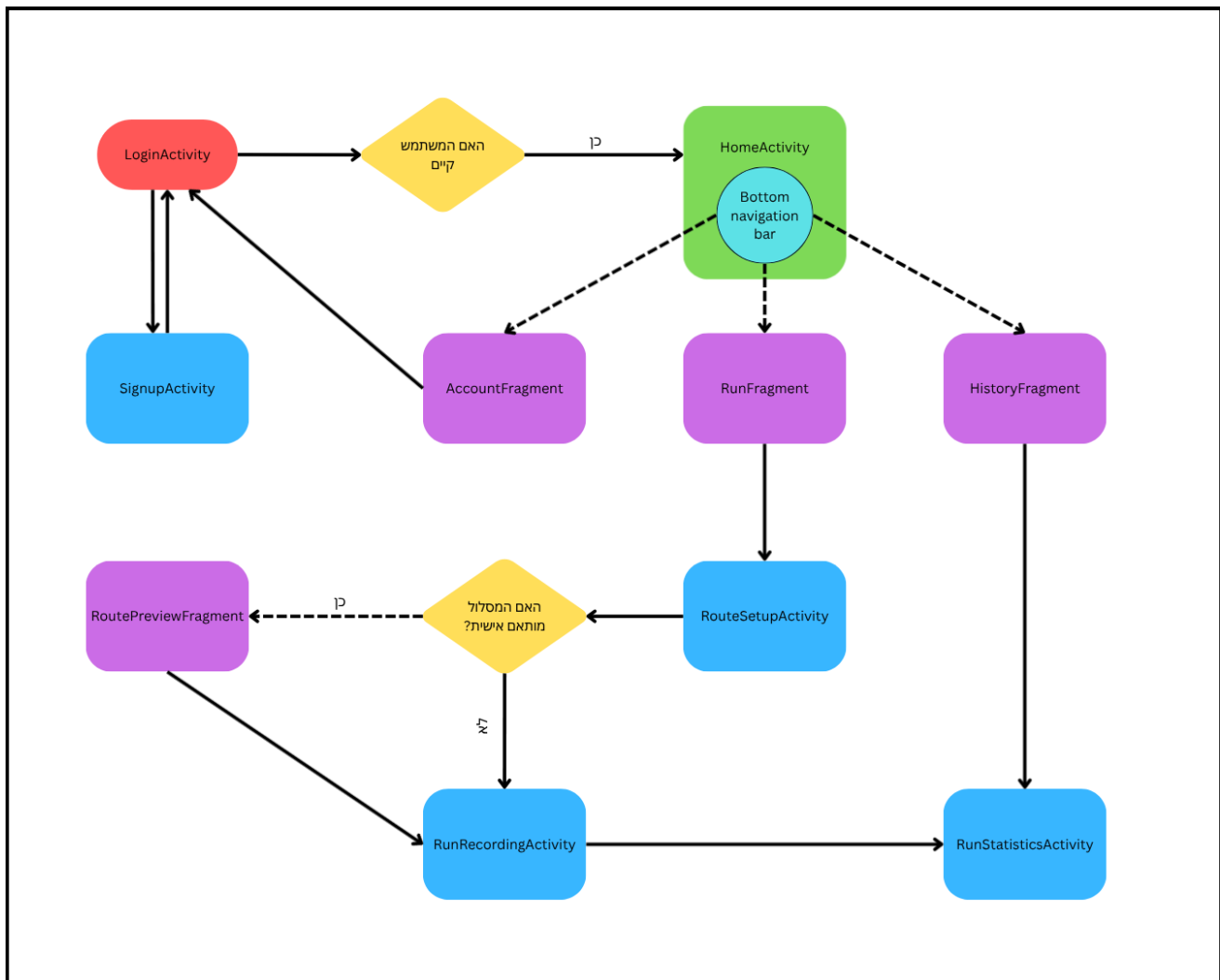
- 1. בקשת המשתמש ואתחול מיקום:** המשתמש מגדיר את אילוצי הריצה ולוחץ על "Generate Route". האפליקציה פונה לחיישן ה-GPS ומקבלת את קואורדינטות המיקום הנוכחיות שלו.
- 2. פנייה לשרת המרכזי:** האפליקציה שולחת בקשת רשת ליצירת מסלול, הכוללת את נתוני המיקום ואילוצי המשתמש, אל השרת.
- 3. תשאול שירותים חיצוניים והפעלת האלגוריתם:** השרת מאתחל את פרמטרים בניית הקלט ומבצע קריאות API אל שירותי הרשת של גוגל (Google REST) לקבלת נתוני המרחב והעלויות. עם קבלת האובייקטים מגוגל, השרת בונה את גרף הקלט, מריץ את אלגוריתם האופטימיזציה (DPSO) ומפיק את הנתוב המיטבי.
- 4. החזרת התוצאה והצגתה:** השרת משלח את אובייקט המסלול הסופי (בפורמט JSON) בחזרה לאפליקציה, וזו מציגה ומשרטטת את המסלול באופן ויזואלי למשתמש על גבי מסך המפה.

ניהול חריגות ומצבי קצה (בלוקי Alt):

- **מיקום לא זמין:** במידה וחיישן ה-GPS אינו זמין ומחזיר הודעת שגיאה, האפליקציה עוצרת את התהליך ומציגה למשתמש בקשה לביצוע ניסיון חוזר.
- **תקלת תקשורת עם השרת:** אם מתרחשת שגיאת רשת או נפילת שרת בעת שליחת הבקשה, האפליקציה קולטת את הכשל ומציגה חיווי והתרעת שגיאה מתאימה למשתמש על גבי המסך.

11. תיאור מסכים וממשק משתמש

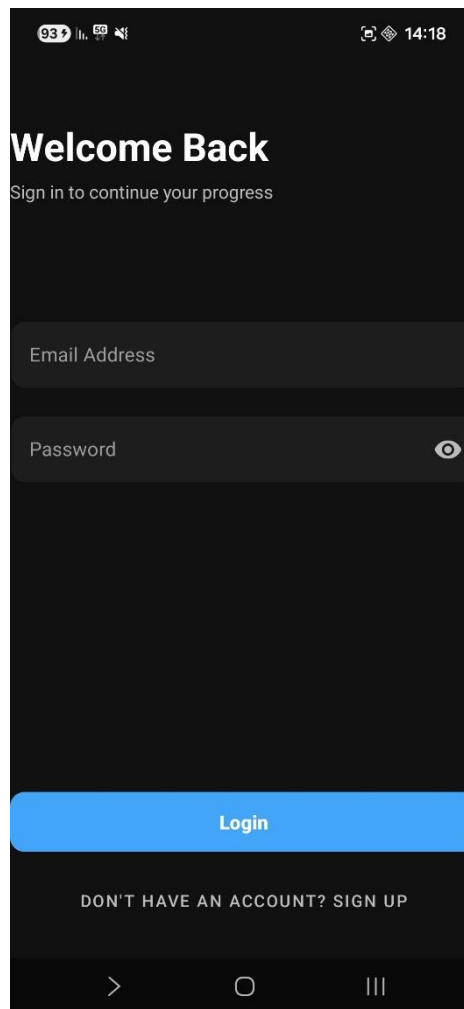
11.1 תרשים היררכיית המסכים



11.2 תיאור המסכים

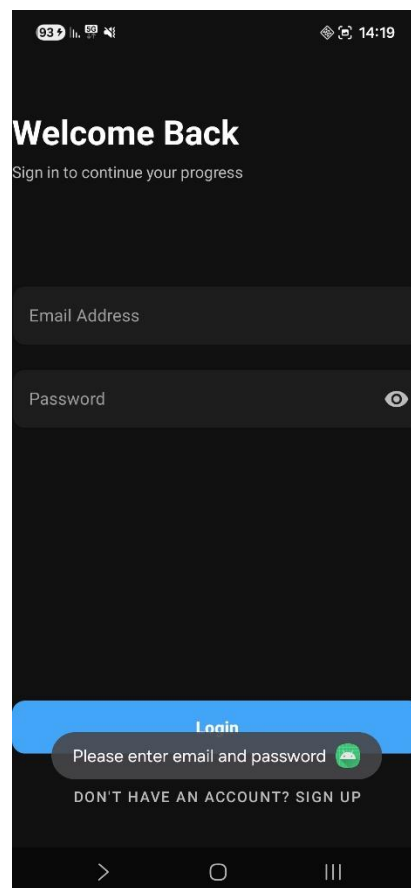
סעיף זה מציג אפיון מפורט ומלא של ממשק המשתמש (UI) באפליקציה הניידת, המבוסס על מבנה קובצי ה-XML של הפרויקט. עבור כל מסך מוגדרים תפקידו המערכתי, רכיבי הממשק השונים המרכיבים אותו ותפקידם התפעולי, לצד פירוט חלונות הדיאלוג וההודעות המוקפצות הרלוונטיות.

1. מסך התחברות (LoginActivity)



- **תיאור תפקידו של המסך:** מסך זה משמש כשער הכניסה הראשי לאפליקציה עבור משתמשים קיימים. הוא מאפשר הזנת פרטי זיהוי (דואר אלקטרוני וסיסמה) ומבצע אימות מול שירותי השרת כדי לאפשר גישה לפרופיל האישי ולשירותי הניתוב.
- **פירוט רכיבי ה-UI במסך:**
 - **כותרת ראשית** (appTitleTV): רכיב טקסט המציג את הודעת הפתיחה "Welcome Back" בגופן מודגש ובגודל בולט.
 - **כותרת משנה** (appSubtitleTV): רכיב טקסט המנחה את המשתמש להתחבר כדי להמשיך בתוכנית האימונים שלו.

- **תיבת קלט לאימייל (EmailET / emailTIL):** רכיב מעוצב מסוג `TextInputLayout` במבנה קופסה מותווחת (`OutlinedBox`) בעל רקע כהה. הרכיב מכיל שדה קלט המותאם דינמית לקליטת כתובות דואר אלקטרוני וכולל תמיכה בהצגת רמז (`Hint`) צבעוני.
- **תיבת קלט לסיסמה (passwordET / passwordTIL):** שדה קלט ייעודי להזנת סיסמה מאובטחת. הרכיב כולל מנגנון טאגל מובנה (`passwordToggleEnabled`) המציג אייקון של עין ומאפשר למשתמש לחשוף או להסתיר את תווים שהוקלדו.
- **כפתור התחברות (login_btn):** כפתור ריבועי מעוגל ורחב בצבע כחול המשמש לשליחת נתוני שדות הקלט אל פונקציות האימות בצד השרת.
- **כפתור מעבר להרשמה (signup_pass_btn):** כפתור טקסט נקי הממוקם בתחתית המסך ומאפשר למשתמשים שאין ברשותם חשבון לעבור ישירות אל מסך יצירת החשבון החדש.
- **הודעות, שגיאות ודיאלוגים מוקפצים:**
 - **הודעות שגיאה מובנות בשדות (TIL Errors):** במקרה שבו המשתמש מותיר את השדות ריקים או מזין אימייל שאינו בפורמט חוקי, האפליקציה מקפיצה חיווי שגיאה אדום ישירות מתחת לתיבת הקלט הרלוונטית.



2. מסך הרשמה ויצירת חשבון (SignupActivity)

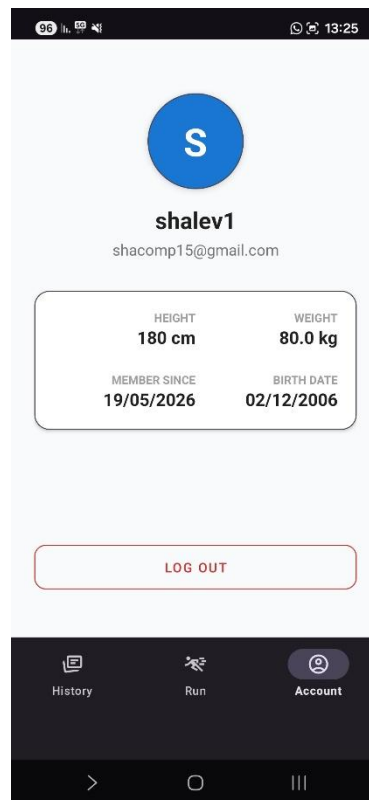
- **תיאור תפקידו של המסך:** מסך זה מאפשר למתאמנים חדשים להירשם במערכת. הוא אוסף את פרטי ההזדהות הבסיסיים שלהם לצד מאפיינים פיזיולוגיים ואישיים, המשמשים בהמשך את שכבת הלוגיקה והאלגוריתמים בשרת לצורך התאמת רדיוס החיפוש, חישובי קלוריות וסטטיסטיקות. מסך זה עטוף ברכיב ScrollView כדי לאפשר גלילה נוחה בכל גודל מסך.
- **פירוט רכיבי ה-UI במסך:**
 - **כותרות דף** (appTitleTV / appSubtitleTV): רכיבי טקסט עליונים המציגים את כותרת יצירת החשבון והנחיה למילוי הפרטים האישיים.
 - **תיבת קלט לשם משתמש** (UsernameET2 / usernameTIL): שדה טקסט מותווך המיועד להזנת כינוי או שם פרטי עבור פרופיל הרץ.
 - **תיבת קלט לאימייל** (EmailET2 / emailTIL): שדה קלט ייעודי להזנת כתובת הדואר האלקטרוני, המשמשת כמפתח הראשי והמזהה הייחודי של החשבון במסד הנתונים בענן.
 - **תיבת קלט לסיסמה** (passwordET2 / passwordTIL): שדה קלט מאובטח להזנת סיסמת החשבון, הכולל אייקון עין מובנה לחשיפת או הסתרת התווים.
 - **תיבת קלט למשקל** (WeightET2 / weightTIL): שדה קלט דיגיטלי המוגבל להקלדת מספרים עשרוניים (numberDecimal). הרכיב משלב סיומת טקסט מובנית קבועה (suffixText) המציגה את יחידת המידה "kg".

- **תיבת קלט לגובה** (HeightET2 / heightTIL): שדה קלט המוגבל להזנת מספרים שלמים ומציג סיומת יחידה מובנית של סנטימטרים "cm".
- **תיבת קלט לתאריך לידה** (BirthDateET2 / birthDateTIL): שדה טקסט מיוחד שהוגדר כלא-ניתן להקלדה חופשית ("focusable=false"), אך מגיב ללחיצה ("clickable=true"). תפקידו להציג חזותית את התאריך שנבחר.
- **כפתור הרשמה** (signup_btn): כפתור MaterialButton רחב ובולט המפעיל את תהליך בדיקת תקינות השדות ושולח בקשת יצירת משתמש חדש אל ה-insertUser בשרת.

• **הודעות, שגיאות ודיאלוגים מוקפצים:**

- **הודעת שגיאה:** הודעה המוקפצת במידה והשרת משליך אקסטפושן המעיד כי כתובת האימייל שהוזנה כבר קיימת במערכת.

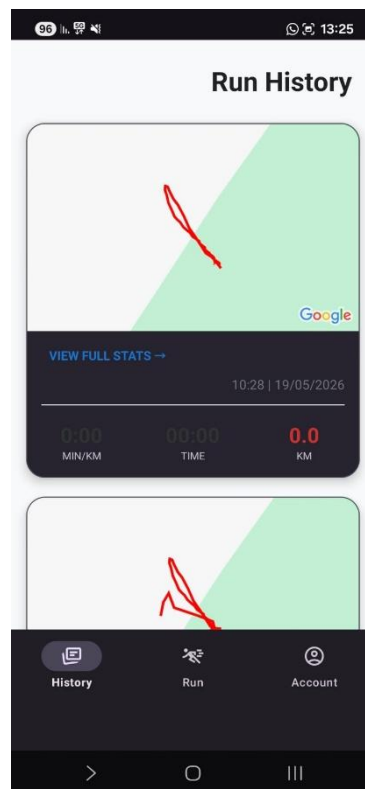
3. מסך פרופיל משתמש (AccountFragment)



- **תיאור תפקידו של המסך:** מסך זה מוצג כפרגמנט בתוך תפריט הניווט הראשי, ותפקידו להציג למתאמן בצורה מרוכזת ואסתטית את פרטי החשבון שלו ואת נתוני הפיזיולוגיים העדכניים שנשלפו מתוך ה-MongoDB. המסך מעוצב על רקע בהיר ונקי.
- **פירוט רכיבי ה-UI במסך:**
 - **כרטיסיית אוטאר עיגולית** (tv_avatar_initial / avatarCard): רכיב כרטיסייה מעוגל לחלוטין המשמש כייצוג חזותי לפרופיל המשתמש, ובמרכזו רכיב טקסט המציג אוטומטית את האות הראשונה של שם המשתמש ברקע כחול בולט.
 - **טקסט שם ואימייל** (tv_profile_email / tv_profile_name): רכיבי טקסט המציגים את שמו הציבורי ואת כתובת הדואר האלקטרוני של המשתמש הנוכחי.
 - **כרטיסיית נתונים אישיים** (statsCard): כרטיסיית חומרית לבנה בעלת פינות מעוגלות והצללה עדינה, המכילה רכיב GridLayout המחלק את המסך לשני טורים שווים. הכרטיסייה מציגה ארבעה תתי-מקטעים סדורים:
 - **WEIGHT** (tv_profile_weight): מציג את משקל הגוף המעודכן של המתאמן בקילוגרמים.
 - **HEIGHT** (tv_profile_height): מציג את גובה המשתמש בסנטימטרים.
 - **BIRTH DATE** (tv_profile_birth): מציג את תאריך הלידה הרשמי של המשתמש.

- MEMBER SINCE (tv_profile_joined): מציג את תאריך יצירת החשבון המקורי, לאחר שהומר מחותמת הזמן הליניארית המאוחסנת בשרת.
- **כפתור התנתקות** (btn_logout): כפתור מותווה (OutlinedButton) המעוצב בצבע אדום וממוקם בתחתית הפרופיל, המיועד לניתוק החשבון והחזרת האפליקציה למצב הזדהות.

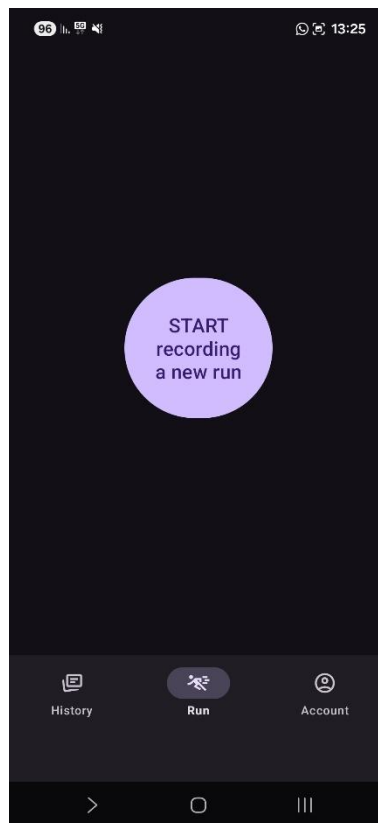
4. מסך היסטוריית ריצות (HistoryFragment)



- **תיאור תפקידו של המסך:** פרגמנט זה מציג למשתמש את יומן האימונים המלא שלו. המסך פונה לשרת, שולף את כל סשני הריצה המשויכים למזהה שלו ומציג אותם בסדר כרונולוגי יורד (מהחדש לישן).
- **פירוט רכיבי ה-UI במסך:**
 - **כותרת דף** (history_title): טקסט גדול ומודגש המגדיר את שם המסך כ-"Run History".
 - **רשימה ממוחזרת** (history_recycler_view): רכיב RecyclerView המהווה את לב המסך. רכיב זה אחראי על הצגה דינמית, חסכונית ויעילה של כרטיסיות הריצה השונות. הרכיב כולל הגדרת ריפוד תחתון מובנית (paddingBottom) כדי להבטיח שהפריטים האחרונים ברשימה לא ייחסמו על ידי סרגל הניווט התחתון של המכשיר.

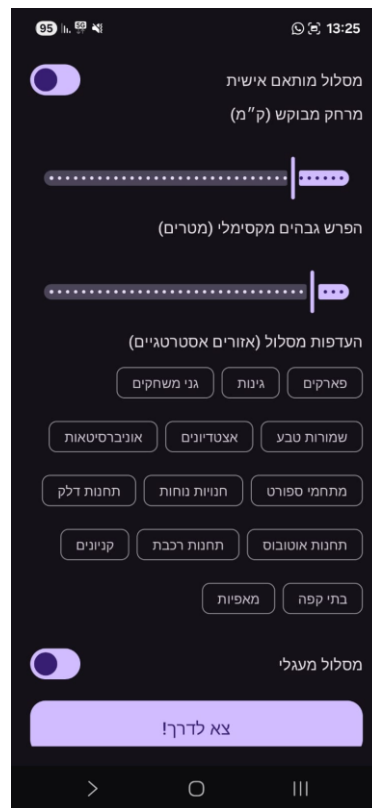
- **רכיב חיווי טעינה (history_loader):** רכיב התקדמות עיגולי (ProgressBar) הממוקם במרכז המסך. הרכיב מוגדר כנסתר כברירת מחדל, ומופיע בצורה אקטיבית רק בזמן שהאפליקציה מבצעת את קריאת הרשת האסינכרונית וממתינה לקבלת רשימת הריצות מהשרת.

5. מסך ראשי לתחילת פעילות (RunFragment)



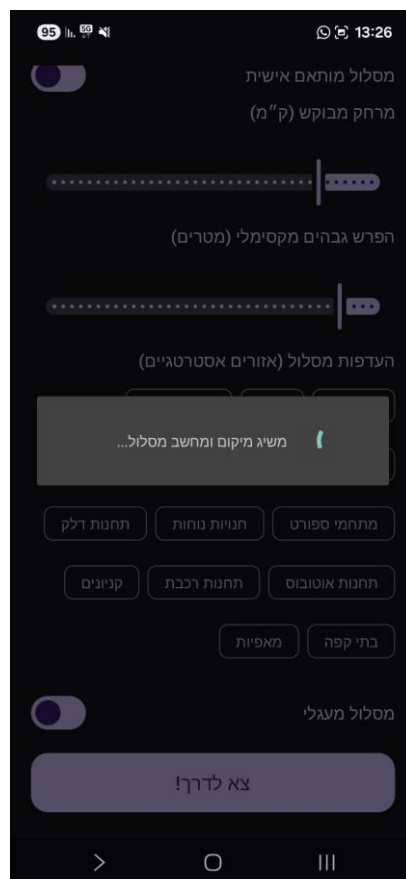
- **תיאור תפקידו של המסך:** משמש כמסך הבית הראשי של אזור הריצה באפליקציה. זהו מסך מינימליסטי וברור שמטרתו לנתב את המשתמש בצורה ישירה ומהירה אל שלב הגדרת ותכנון הריצה החדשה שלו.
- **פירוט רכיבי ה-UI במסך:**
 - **כפתור הקלטת ריצה חדשה (start_run_btn):** כפתור MaterialButton גדול ובולט הממוקם במרכז המדויק של המסך באמצעות אילוצי ציר אנכיים ואופקיים מלאים. הכפתור מעוצב בצורה מרובעת גדולה ומציג את הטקסט "START recording a new run". ומשמש כרכיב הלינק המרכזי המוביל לפתיחת מסך הגדרת האילוצים.

6. מסך הגדרת ואילוצי מסלול (RouteSetupActivity)



- **תיאור תפקידו של המסך:** מסך זה מהווה את ממשק הקלט המרכזי עבור אלגוריתמי הליבה בשרת. הוא מאפשר למתאמן לקבוע אילוצים קשיחים (מרחק, גובה, אופי המסלול) לצד בחירת העדפות סביבתיות, ומעביר את המידע המובנה אל ה-RouteService ב-Backend לצורך הפקת מסלול אופטימלי מותאם אישית.
- **פירוט רכיבי ה-UI במסך:**
 - **כותרת דף:** טקסט מרכזי גדול בגודל sp28 המציג את המשפט "הגדרת הריצה שלך".
 - **מתג הפעלת התאמה אישית (switchCustomized):** רכיב MaterialSwitch המאפשר למשתמש לקבוע האם הוא מעוניין שהמערכת תתכן עבורו מסלול חכם ומותאם אישית המבוסס על אופטימיזציה, או לחלופין לבצע ריצה חופשית ללא מסלול מתוכנן מראש.
 - **אזור קונסטיינר אילוצים (layoutConstraints):** שכבת פריסה ליניארית המרכזת את כלל מדדי הבחירה, הניתנת להסתרה דינמית בהתאם למצב המתג העליון:
 - **בורר מרחק מבוקש (sliderDistance):** רכיב Slider רציף המאפשר בחירת מרחק יעד בטווח של 1.0 ק"מ ועד 25.0 ק"מ, בקפיצות מוגדרות של חצי קילומטר. ערך זה מתורגם ישירות לפרמטר ה-T_MAX של האלגוריתם.

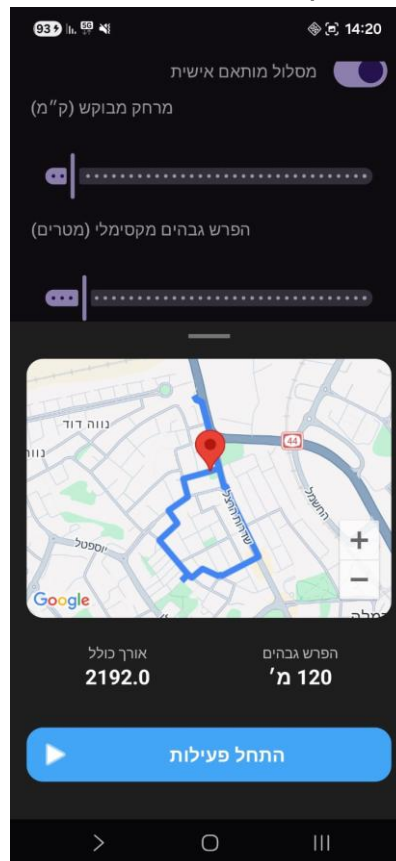
- בורר הפרש גבהים מקסימלי (sliderElevation): רכיב Slider המאפשר הגדרת מגבלת עלייה טופוגרפית בטווח של 0 עד 500 מטרים, בקפיצות של 10 מטרים.
- קבוצת תגיות אזורים אסטרטגיים (chipGroupAreas): קבוצת רכיבי Chip המוגדרים כבני-סימון ("checkable=true"). התגיות מחולקות קטגורית ומאפשרות למשתמש לבחור העדפות סביבתיות (כמו פארקים, גינות, מתחמי ספורט, חנויות נוחות, בתי קפה ועוד), המשפיעות ישירות על שקלול ציון התגמול (Reward) של ה-Candidates באלגוריתם הליבה.
- **מתג מסלול מעגלי** (switchCircular): רכיב מתג הקובע האם מנוע האופטימיזציה יאלץ את המסלול להיסגר כטבעת שלמה החוזרת לנקודת הזינוק, או להפיק מסלול קווי מנקודה לנקודה.
- **כפתור יציאה לדרך** (btnStartAction): כפתור MaterialButton רחב וגבוה בעל פינות מעוגלות הממוקם בתחתית הדף, המבצע את איסוף נתוני השדות ושליחתם לעיבוד בשרת.



- הודעות, שגיאות ודיאלוגים מוקפצים:

- פרוגרס בר – נועד לנעול את ה-UI על מנת לחכות עד שתתקבל תשובה מהשרת לבקשת המסלול.

7. מסך תצוגה מקדימה וסקירת מסלול (Route Preview Bottom Sheet)



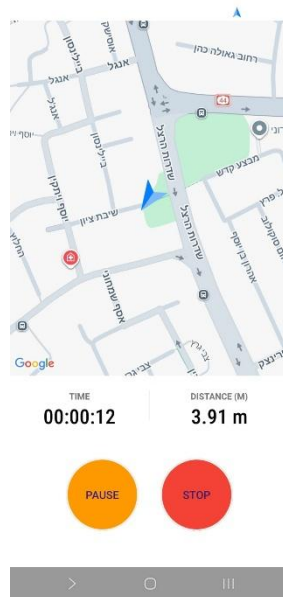
תיאור תפקידו של המסך: רכיב ממשק זה ממומש כחלון נפתח תחתון (Bottom Sheet) המוצג מעל מסך המפה הראשי לאחר ששרת ה-Backend סיים לעבד את האלגוריתם והחזיר את אובייקט המסלול האופטימלי (OptimizedRoute). הוא מאפשר למשתמש לסקור את נתוני המסלול המוצע ולראות את תוואי הדרך לפני היציאה לאימון.

- פירוט רכיבי ה-UI במסך:

- **ידית גרירה עיצובית (drag_handle):** רכיב View קטן וצר במרכז העליון של החלונית, המשמש כרמז חזותי למשתמש המעיד כי ניתן לגרור, להרחיב או למזער את חלון תצוגת הנתונים.
- **חלון המפה המובנה (map_fragment_container / map_card):** רכיב כרטיסייה המכיל את תשתית המפות הרשמית של גוגל (SupportMapFragment). רכיב זה מציג ויזואלית את נקודות העניין שנבחרו ואת קו הדרך הרציף (Polyline) שחושב בשרת.
- **תצוגת אורך כולל (tv_total_distance):** רכיב טקסט המציג בצורה מודגשת את אורכו האמיתי והמדויק של המסלול שהופק (לדוגמה: "5.2 ק"מ").

- **תצוגת הפרש גבהים** (tv_elevation_gain): רכיב טקסט המציג את סך העליות המצטברות לאורך הנתיב המתוכנן (לדוגמה: "120 מ").
- **כפתור התחלת פעילות** (btn_start_recording): כפתור MaterialButton רחב בעל פינות מעוגלות המשלב אייקון מובנה של Play. לחיצה עליו סוגרת את הסקירה ומעבירה את המשתמש באופן מיידי אל מסך ניטור המעקב האקטיבי בשטח.

8. מסך ניטור ומעקב ריצה פעילה (RunRecordingActivity)

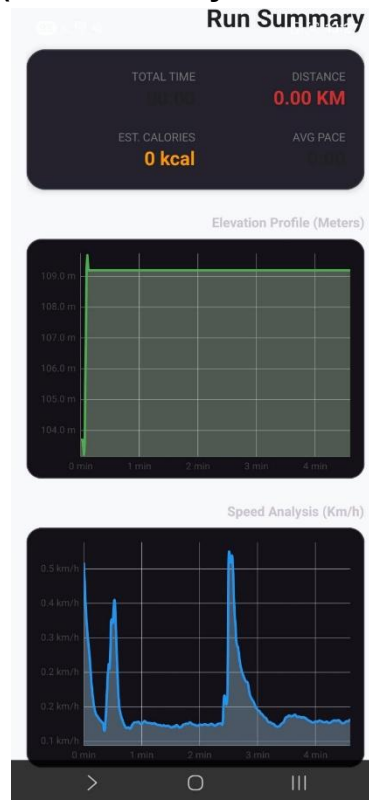


- **תיאור תפקידו של המסך:** מסך תפעולי מרכזי המלווה את המתאמן בזמן אמת לאורך כל משך הריצה הפיזית שלו בשטח. המסך מציג את מיקומו הדינמי על גבי המפה, מנהל את זמני הנטו והברוטו של האימון ומציג מדדי ביצוע אקטיביים.
- **פירוט רכיבי ה-UI במסך:**
 - **רכיב מפה עליון** (map_container): רכיב מפה רחב התופס בדיוק 65% משטח המסך, המתוחם באמצעות קו מנחה אופקי (Guideline). המפה מציגה בזמן אמת את מיקומו הנוכחי של הרץ ומשתרטט דינמית את נתיב הריצה שכבר בוצע.
 - **תצוגת שעון עצר** (timer_tv): רכיב טקסט המעוצב בגופן קומפקטי מיוחד (sans-serif-condensed-medium) והוא מציג את זמן הנטו המדויק של האימון (בפורמט HH:MM:SS) ומנוהל על ידי תהליכים מקביליים המקפיאים את הספירה בעת הפסקה.
 - **תצוגת מרחק מצטבר** (passed_distance_tv): שדה טקסט המציג את המרחק הממשי במטרים או בקילומטרים שהרץ עבר עד כה, בהתבסס על נקודות המיקום המסוננות שאושרו.
 - **כפתור השהיה וחזרה** (btn_pause_resume): כפתור MaterialButton עגול לחלוטין בצבע כתום בולט הנושא את הטקסט "PAUSE". לחיצה עליו מקפיאה את השעון, משנה

את הסטטוס הלוגי של הריצה, ומאפשרת למשתמש לעצור זמנית את המעקב מבלי לפגוע בסטטיסטיקות המרחק.

- **כפתור עצירה וסיום (btn_stop):** כפתור עגול לחלוטין בצבע אדום הנושא את הטקסט "STOP". כפתור זה מסיים את הריצה וקורא לפונקציית העיבוד האנליטית בשרת.

9. מסך סיכום אימון ואנליטיקה (Run Summary Dashboard)



תיאור תפקידו של המסך: מסך זה מוצג למשתמש מיד עם סיומו המוצלח של אימון ריצה או בעת בחירת אימון עבר מתוך מסך ההיסטוריה. תפקידו לספק מצגת אנליטית וויזואלית מקיפה של כלל מדדי הביצוע שנשמרו ועובדו בשרת, תוך שימוש ברכיבים גרפיים מתקדמים לצורך הצגת מגמות ונתונים טופוגרפיים ופיזיולוגיים. המסך עטוף ב-ScrollView המאפשר גלילה חופשית בין הגרפים השונים.

• פירוט רכיבי ה-UI במסך:

- **כותרת דף (tv_run_title):** טקסט עליון מודגש המציג את כותרת הדשבורד "Run Summary".
- **כרטיסיית סיכום נתונים מספריים:** כרטיסיית חומרית רחבה בעלת פינות מעוגלות היטב המכילה GridLayout המפצל את המרחב לארבעה ריבועי נתונים ברורים:
 - מרחק סופי (tv_total_distance): מציג את סך הקילומטרים המעובד שעבר הרץ בצבע אדום בולט.
 - זמן כולל (tv_total_time): מציג את משך זמן הפעילות נטו (בפורמט דקות ושניות).

- קצב ממוצע (tv_avg_pace): מציג את קצב הריצה הממוצע המחושב ביחידות של דקות לקילומטר.
- קלוריות מוערכות (tv_calories): מציג בצבע כתום את אומדן האנרגיה והקלוריות שנשרפו במהלך הריצה, המחושב על בסיס שקלול משקל הגוף של המשתמש ומשך הפעילות.
- גרף פרופיל גובה (chart_elevation): רכיב גרפי מתקדם מסוג LineChart (מתוך ספריית MPAndroidChart) המציג קו מגמה רציף של שינויי הגובה הטופוגרפיים במטרים לאורך נקודות הריצה, ומאפשר לראות עליות וירידות בתוואי השטח.
- גרף ניתוח מהירויות (chart_speed): רכיב גרפי מסוג LineChart המציג ויזואלית את שינויי מהירות הריצה של המתאמן (בקילומטר לשעה) לאורך שלבי האימון השונים, לצורך ניתוח קצבים ומאמץ.

12. תיאור התוכנה

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

מערכת ההפעלה Windows 11: מערכת הפעלה זו משמשת כסביבת המארח (Host OS) שעל גביה מבוצעים כלל תהליכי הפיתוח, הקידוד, הניתוח והבדיקות של הפרויקט מקצה לקצה. המערכת מספקת תשתית יציבה, מאובטחת ובעלת ביצועים גבוהים להרצה סימולטנית של סביבות הפיתוח וההנדסה השונות (Android Studio ו Visual Studio Code) וניהול שרת ה Backend-המקומי. בזכות מנגנוני ניהול משאבי חומרה מתקדמים, תמיכה מלאה בריצה מקבילית ומערכת קבצים מהירה, היא מאפשרת הפעלה חלקה של רכיבי הדמיה ומחשבים וירטואליים (Emulators) לצורך בדיקת האפליקציה, לצד ניהול גמיש של כלי מערכת ומשתני סביבה החיוניים לאינטגרציה וסנכרון רכיבי התוכנה.

12.1.2 סביבות הפיתוח (IDEs)

- **סביבת הפיתוח Visual Studio Code:** כלי זה משמש כסביבת העבודה (IDE) המרכזית עבור פיתוח ה Backend. זוהי סביבה קלה, מהירה וגמישה, התומכת בכתובת קוד מסודרת, מעודדת זרימת עבודה חלקה ומקלה על תחזוקת הפרויקט לאורך זמן. סביבת עבודה זו מאפשרת התאמה אישית גבוהה ומשתלבת היטב בתהליכי פיתוח מודרניים בצד השרת.
- **סביבת הפיתוח Android Studio:** סביבה זו משמשת לצורך בניית אפליקציית הלקוח (Mobile). זוהי סביבת הפיתוח הרשמית והייעודית לפלטפורמת Android, המאפשרת בניית

ממשקי משתמש מתקדמים המותאמים למכשירי הקצה. היא מספקת תהליך עבודה סדור ואחיד, תומכת בארכיטקטורה מסודרת ומאפשרת ניהול נכון של אינטראקציות חומרה, סנכרון עם מחזור חיי האפליקציה והבטחת ביצועים מיטביים בשטח.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

מסגרת העבודה Spring Boot Framework: מסגרת עבודה זו משמשת לפיתוח צד השרת (Backend). היא מספקת תשתית סדורה וברורה לבניית יישומי רשת מודרניים, תוך שמירה על ארכיטקטורה נקייה והפרדה מלאה בין שכבות הלוגיקה, הגישה לנתונים והחשיפה החיצונית. מסגרת העבודה מעודדת כתיבת קוד קריא, תחזוקתי וניתן להרחבה, מאפשרת התאמה קלה לשינויים בדרישות, ומבוססת על עקרונות תכנון מוכחים הנהוגים בתעשיית התוכנה לפיתוח מערכות יציבות בקנה מידה משתנה.

12.1.4 שפות תכנות

שפת התכנות Java 21: הפרויקט מפותח באמצעות שפת התכנות Java 21 – שפת עילית מתקדמת, מונחית עצמים (OOP), המהווה את עמוד התווך לפיתוח מערכות מבוצרות ומספקת תמיכה מלאה בבטיחות טיפוסים, ניהול זיכרון אוטומטי וביצועים גבוהים.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

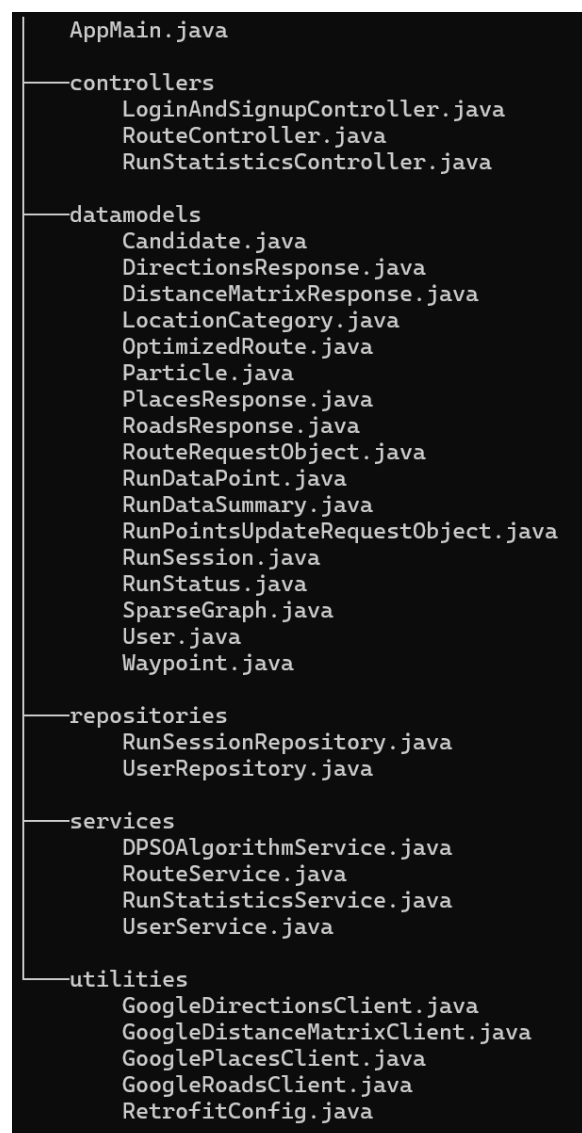
- **ספריית הרשת Retrofit:** ספרייה זו משמשת בצד הלקוח (Android) לצורך ניהול התקשורת וביצוע קריאות ה-HTTP מול שרת ה-Spring Boot. הספרייה הופכת את ממשקי ה-API של השרת לאובייקטים (Interfaces) מוגדרים ובטוחים בשפת Java באמצעות אנוטציות (Annotations). היא מחליפה את הצורך בכתיבת קוד תקשורת ידני ומסורבל, ומבצעת המרה אוטומטית, מהירה ויציבה של מבני נתונים (JSON) לישויות המערכת באפליקציה.
- **ספריית הרשת OkHttp3:** ספרייה זו פועלת בצד הלקוח כליבת התקשורת של המערכת (שכבת התעבורה שעליה נשענת ספריית Retrofit). היא מנהלת בפועל את שליחת הבקשות וקבלת המענים ברמת הרשת, ומספקת מנגנונים מתקדמים לניהול נכון של חיבורים, הגדרת זמני המתנה (Timeouts), טיפול אוטומטי בכשלים זמניים ברשת וייעול חילוף המידע מול השרת לטובת חיסכון במשאבי המכשיר.
- **ספריית העיצוב Material Components:** ספריית רכיבים זו משמשת לעיצוב ופיתוח ממשק המשתמש (UI) באפליקציה הניידת. הספרייה מבוססת על שפת העיצוב הרשמית של גוגל ומספקת רכיבי ממשק מוכנים, מודרניים וסטנדרטיים (כגון כפתורים, שדות קלט, כרטיסי

מידע ותפריטי ניווט). השימוש בה מבטיח חוויית משתמש אחידה, אסתטית ודינמית, המתאימה את עצמה בצורה רספונסיבית לגדלי מסכים שונים ומספקת מראה מקצועי כמקובל בשוק.

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים של הפרויקט

להלן עץ המודולים בצד השרת (VSCode):



להלן עץ המודולים בצד הלקוח (Android Studio):

```
—activities
    HomeActivity.java
    LoginActivity.java
    RouteSetupActivity.java
    RunRecordingActivity.java
    RunStatisticsActivity.java
    SignupActivity.java
    SplashActivity.java

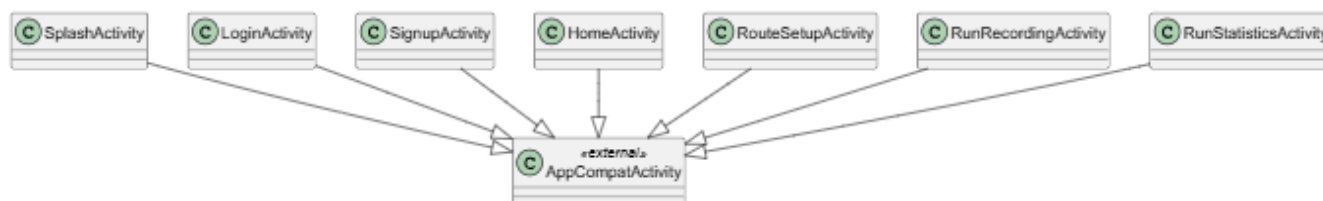
—data_models
    OptimizedRoute.java
    RouteRequestObject.java
    RunDataPoint.java
    RunDataSummary.java
    RunPointsUpdateRequestObject.java
    RunSession.java
    RunStatus.java
    User.java
    Waypoint.java

—fragments
    AccountFragment.java
    HistoryFragment.java
    RoutePreviewFragment.java
    RunFragment.java

—utilities
    RESTCommunicationHelper.java
    RunHistoryAdapter.java
    RunTrackingService.java
    SelectedRunHolder.java
    ServerEndpointsAPI.java
    UIHelper.java
    UserManager.java
```

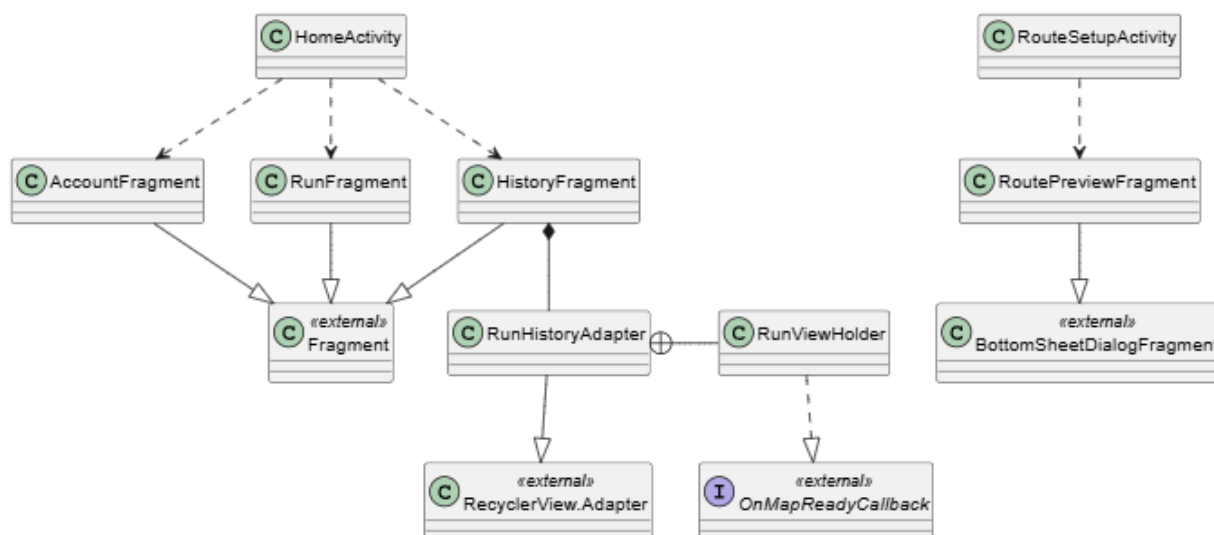
12.2.2 תרשים המחלקות UML Class Diagram

תרשים מסכים וניווט (Navigation Flow & Activities)



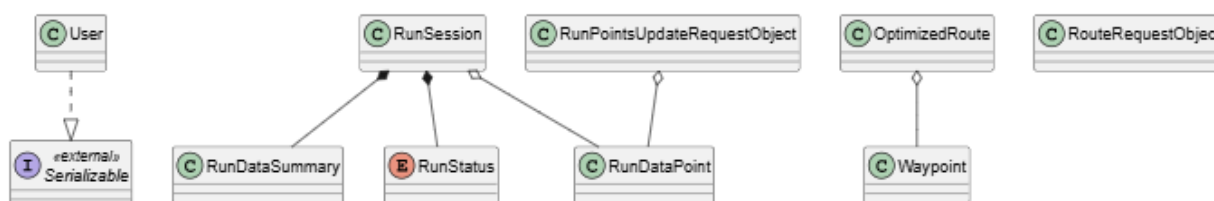
תרשים זה מתאר את מסכי האפליקציה המרכזיים (Activities), את קשרי הירושה שלהם מרכיבי מערכת ההפעלה של Android ואת כיווני זרימת הניווט ומעברי המסכים במערכת.

תרשים רכיבי תצוגה ורשימות (Adapters & Fragments)



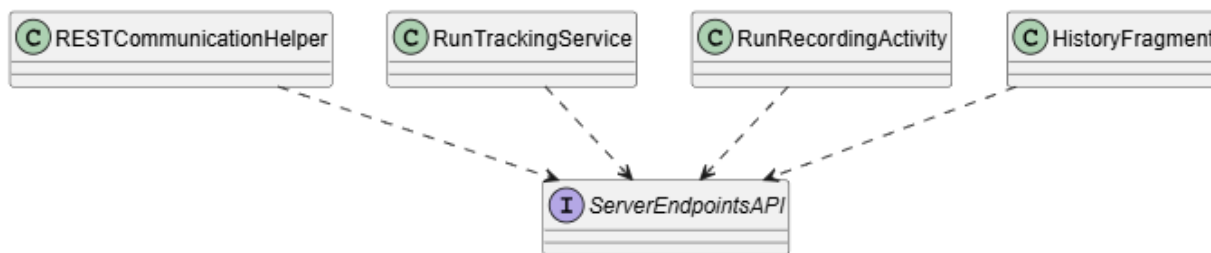
התרשים מציג את רכיבי ממשק המשתמש הדינמיים (Fragments) ואת מתאמי התצוגה הנדרשים לניהול רשימות גלילה מורכבות (Adapters), כולל מיפוי מחלקות פנימיות מוכלות.

תרשים מודלי הנתונים (Entities & Data Models)



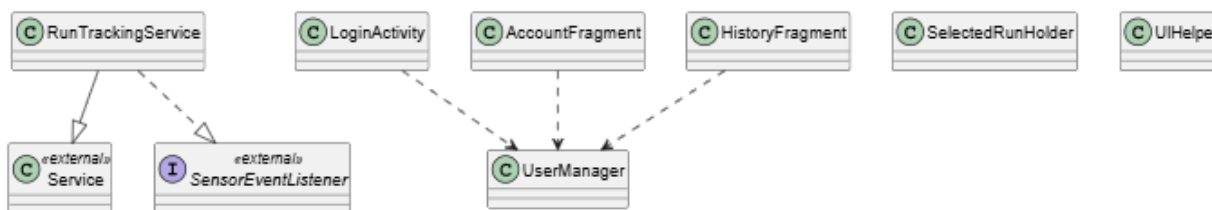
תרשים זה מרכז את כל ישויות ומבני הנתונים הטהורים של האפליקציה, ומגדיר את יחסי ההכלה (קומפוזיציה ואגרגציה) השולטים במחזור החיים וברמת עצמאות המידע בזיכרון.

תרשים שכבת התקשורת וה-API (API Layer & API Network)



תרשים זה מתאר את תשתית התקשורת של האפליקציה, ומציג כיצד מחלקת העזר, שירות הרקע והמסכים השונים מקיימים אינטראקציה מול ממשק נקודות הקצה (API) של השרת המרוחק.

תרשים שירותי רקע וניהול מקומי (Local Management & Background Services)



תרשים זה מרכז את שירות הרקע האקטיבי האחראי על איסוף נתוני החיישנים וה-GPS, לצד מחלקות העזר האחראיות על ניהול הסשן, רכיבי הזיכרון המקומי והשמירה של המשתמש.

12.2.3 תיאור מחלקות מפורט

בצד השרת (VSCode):

המחלקה Candidate

Candidate
id: String lat: double lng: double reward: double
Candidate(id: String, lat: double, lng: double, reward: double) getId(): String getLat(): double getLng(): double getReward(): double setReward(reward: double): void equals(o: Object): boolean hashCode(): int

- **תפקיד המחלקה:** משמשת כאבן הבניין הבסיסית של המערכת ומייצגת יעד או נקודת עניין פוטנציאלית על המפה שהרץ יכול לבקר בה.
- **תכונות משמעותיות:**
 - **reward** - מגדיר את הציון או הגמול ששווה הגעה לנקודה זו. זהו הנתון המרכזי שקובע כמה הנקודה הזו אטרקטיבית עבור אלגוריתם האופטימיזציה.
- **פעולות משמעותיות:**
 - **hashCode** ו- **equals** - פעולות דריסה שמאלצות את המערכת להשוות בין נקודות אך ורק על בסיס המזהה שלהן. זה מונע כפילויות של אותם יעדים בתוך מבני הנתונים בזמן החישובים.

המחלקה Waypoint

C Waypoint
lat: double lng: double placeId: String reward: double distanceToNext: double
Waypoint() Waypoint(lat: double, lng: double, placeId: String, reward: double, distanceToNext: double) getLat(): double getLng(): double getPlaceId(): String getReward(): double getDistanceToNext(): double

- **תפקיד המחלקה:** מייצגת תחנה פיזית קבועה ורשמית בתוך מסלול ריצה שכבר חושב ואושר עבור המשתמש.
- **תכונות משמעותיות:**
 - **distanceToNext** - שומר את המרחק המדויק מהתחנה הנוכחית אל התחנה הבאה בתור במסלול. נתון זה קריטי עבור רכיב הניווט באפליקציה כדי להציג למשתמש כמה מרחק נשאר לו לרוץ עד ליעד הבא.
- **פעולות משמעותיות:**
 - אין פעולות לוגיות מורכבות במחלקה זו מעבר לבנאים וגטרים רגילים המיועדים להעברת המידע בצורה נקייה.

המחלקה OptimizedRoute

C OptimizedRoute
totalDistance: double totalReward: double path: List<Waypoint> polyline: String
OptimizedRoute() OptimizedRoute(path: List<Waypoint>, polyline: String) getTotalDistance(): double getTotalReward(): double getPath(): List<Waypoint> getPolyline(): String calculateTotalReward(): void calculateTotalDistance(): void

- **תפקיד המחלקה:** מייצגת את תוצאת הקצה של התהליך – אובייקט המסלול השלם והמשופר שנשלח ישירות למכשיר הטלפון של המשתמש.

- **תכונות משמעותיות:**

- **polyline** - מחרוזת טקסט מקודדת המכילה את קו הניווט הגיאוגרפי, המאפשרת ל-Google Maps API לצייר את מסלול הריצה המדויק על גבי המפה באפליקציה.

- **פעולות משמעותיות:**

- **calculateTotalReward** - פעולת בקרה שסורקת את כל התחנות במסלול ומסכמת את סך הניקוד שהשתמש ירוויח אם ירוץ בו.
- **calculateTotalDistance** - פעולת בקרה שמסכמת את המרחקים בין כל התחנות כדי לקבוע את אורך הריצה הכולל בקילומטרים.

המחלקה SparseGraph

- **תפקיד המחלקה:** משמשת כמפה הדיגיטלית הפנימית של האלגוריתם, ומייצגת את רשת הכבישים והקשרים הפיזיים בין כל היעדים האפשריים במרחב.

- **תכונות משמעותיות:**

- **distanceMap** - מפה מקוננת המשמשת כטבלת מרחקים מהירה ומחזיקה את עלות המעבר האמיתית בין זוגות של נקודות.

- **פעולות משמעותיות:**

- **getCost** - מחזירה את מרחק הנסיעה או הריצה בין שתי נקודות. אם אין כביש או נתיב שמחבר ביניהן, היא מחזירה ערך אינסופי כדי להבהיר לאלגוריתם שהנתיב הזה אינו חוקי לתנועה.

המחלקה Particle

 Particle
□ position: List<Candidate> □ personalBest: List<Candidate> □ fitness: double □ bestFitness: double
● Particle(initialRoute: List<Candidate>, fitness: double) ● getPosition(): List<Candidate> ● getPersonalBest(): List<Candidate> ● getFitness(): double ● getBestFitness(): double ● setPosition(newPosition: List<Candidate>): void ● updatePersonalBest(newFitness: double): void

- **תפקיד המחלקה:** מייצגת חלקיק בודד בנחיל, המתפקד כגשש עצמאי שמחזיק, בודק ומפתח מסלול ריצה אפשרי במהלך ריצת האלגוריתם.

- **תכונות משמעותיות:**

- **position** - רשימת הנקודות המייצגת את מסלול הריצה הנוכחי שהחלקיק מציע ומנסה לשפר ברגע זה.

- **personalBest** - שומר את תמונת המצב של המסלול הטוב ביותר והאיכותי ביותר שהחלקיק הספציפי הזה הצליח למצוא מאז שהתחיל לרוץ, המוכר גם כ-pBest של החלקיק.

- **פעולות משמעותיות:**

- **updatePersonalBest** - פעולה הבודקת האם המסלול הנוכחי שובר את שיא האיכות האישי של החלקיק. אם כן, היא מעדכנת את ציון השיא ונועלת את המיקום הנוכחי כ-personalBest החדש.

המחלקה DPSOAlgorithmService והמחלקה הסטטית הפנימית DPSORunner



- **תפקיד המחלקה:** מנוע האופטימיזציה הראשי של המערכת המנהל את אלגוריתם ה-Discrete Particle Swarm Optimization כדי לייצר את מסלול הריצה המושלם תחת מגבלות המרחק והזמן.

- **תכונות משמעותיות:**

- **W, C1 ו-C2** - קבועי משקל מתמטיים שמנווטים את תנועת הנחיל ומאזנות בין שלושה כוחות: החלטות אקראיות, הזיכרון האישי של כל חלקיק (personalBest), וההצלחה הקבוצתית של החלקיק המצטיין בנחיל (globalBest).
- **globalBest** - שומר את המסלול האיכותי ביותר שנמצא אי פעם על ידי חבר כלשהו בנחיל. זהו הפתרון הסופי שיוחזר למשתמש בסיום הריצה.

• פעולות משמעותיות:

- **solve** - הלולאה המרכזית שמריצה את מספר האיטרציות המבוקש ומנהלת את תהליך החיפוש והשדרוג של כל החלקיקים במקביל.
- **projectToFeasible** - פונקציית בקרה קריטית שקוטעת מסלולים ומוודאת שהם לא חורגים מתקציב המרחק המקסימלי שהגדיר הרץ, השמור במשתנה T_MAX. במידה והמסלול מוגדר כמעגלי באמצעות הדגל isCircular, היא מחשבת מראש מתי חובה להתחיל לחזור הביתה וסוגרת את המעגל בבטחה אל נקודת המוצא startNode.
- **twoOptOperation** - מנגנון שיפור גיאומטרי מקומי שמסדר מחדש את סדר התחנות במסלול כדי לפתור הצטלבויות כבישים מיותרות במפה ולקצר את מרחק הריצה הפיזי.
- **localSearchOptimize** - פונקציה המפתחת את מסלול החלקיק על ידי ביצוע שינויים מבניים, כמו החלפת תחנות או הזרקת מקטעים מוצלחים, מתוך השראה של ה-personalBest שלו וה-globalBest הכללי.

המחלקה RouteController

C	RouteController
□	routeService: RouteService
●	generateRoute(request: RouteRequestObject): ResponseEntity<OptimizedRoute>

- **תפקיד המחלקה:** משמשת כנקודת הקצה (API Endpoint) של שרת ה-Spring Boot, ומקבלת את בקשות ה-HTTP מהאפליקציה הניידת כדי להתחיל בתהליך יצירת המסלול האופטימלי.

• תכונות משמעותיות:

- **routeService** - מחלקת השירות המרכזית שאליה מוזרקת הלוגיקה העסקית, והיא זו שמנהלת בפועל את כל שלבי בניית המסלול.

• פעולות משמעותיות:

- **generateRoute** - מקבלת בקשת POST המכילה את נתוני המשתמש, קוראת לשירות החישוב, ומחזירה תגובת HTTP מסוג ResponseEntity המכילה את אובייקט המסלול הסופי המובנה בפורמט JSON.

המחלקה RouteRequestObject

C RouteRequestObject
- startLat: double - startLng: double - distance: double - isCircular: boolean - selectedCategories: List<String>
- RouteRequestObject() - RouteRequestObject(startLat: double, startLng: double, distance: double, isCircular: boolean, selectedCategories: ArrayList<String>) - getStartLat(): double - getStartLng(): double - getDistance(): double - getSelectedCategories(): List<String> - isCircular(): boolean

- **תפקיד המחלקה:** אובייקט העברת נתונים (DTO) המייצג את מבנה הבקשה שמגיעה מהמכשיר הנייד של הרץ, ומאגד את הפרמטרים שעל פיהם יחושב המסלול.
- **תכונות משמעותיות:**
 - **startLat** ו-**startLng** - קואורדינטות נקודת הזינוק הגיאוגרפית של הרץ במפה.
 - **distance** - אורך או טווח המסלול המקסימלי המבוקש על ידי המשתמש (במטרים או בקילומטרים).
 - **isCircular** - דגל בוליאני המגדיר האם המשתמש מעוניין במסלול מעגלי שיחזיר אותו לנקודת המוצא, או במסלול קווי מנקודה לנקודה.
- **פעולות משמעותיות:**
 - במחלקה זו קיימים רק בנאים, גטרים וסטרים פשוטים הנחוצים לספריית ג'קסון לצורך המרת נתוני ה-JSON הנכנסים לאובייקט Java.

ה-enum LocationCategory

E LocationCategory
- GREEN_ZONE - SPORTS_AND_CAMPUS - SERVICE_POINTS - TRAFFIC_HUB - ATTRACTIONS - STREET - DEFAULT - baseScore : double - apiTypes : List<String> - extraTypes : List<String> - isSearchable : boolean
- LocationCategory(baseScore : double, apiTypes : List<String>, extraTypes : List<String>, isSearchable : boolean) - getBaseScore() : double - getGlobalSearchableTypes() : List<String> - getHighestScore(googleTypes : List<String>, bonusCategories : List<String>) : double

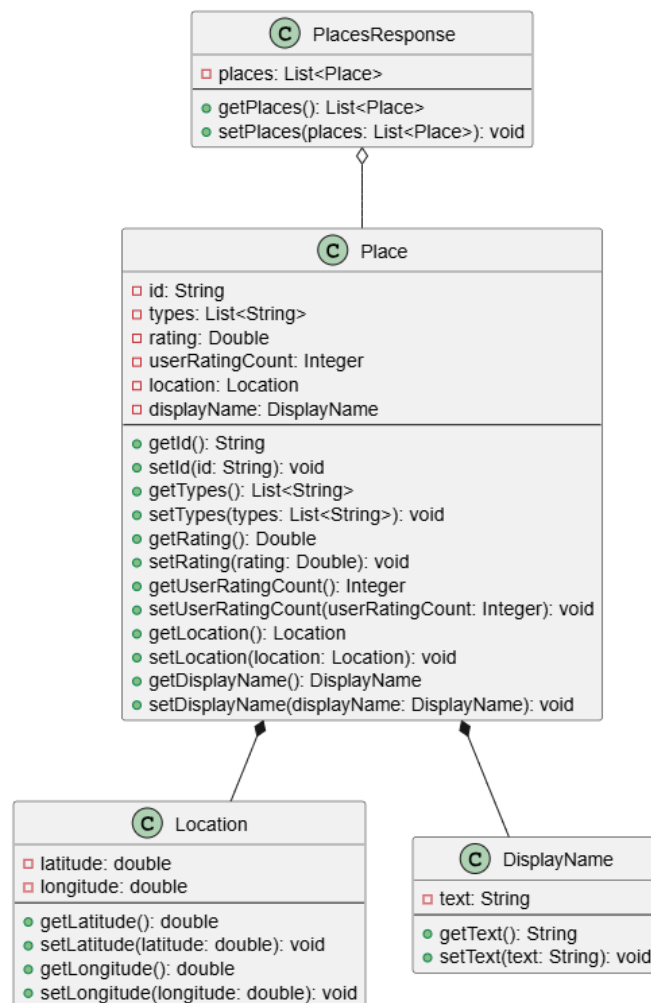
- **תפקיד המחלקה:** קובץ (Enum) המגדיר את קטגוריות המיקום השונות במערכת (כמו אזורים ירוקים, מתחמי ספורט, נקודות שירות, רחובות ועוד), וקובע את ציון הבסיס הניתן לכל סוג מקום.
- **תכונות משמעותיות:**

- **baseScore** - הציון הגולמי שכל קטגוריה מייצגת. ציון זה משפיע ישירות על כדאיות הגעת האלגוריתם אל המקום.
- **apiTypes** - רשימת מילות המפתח התואמות למונחי החיפוש הרשמיים של Google Places API.
- **isSearchable** - דגל המציין האם המערכת צריכה לחפש באופן יזום מקומות מסוג זה, או להשתמש בקטגוריה רק לצורך ניקוד של נקודות קיימות.

• פעולות משמעותיות:

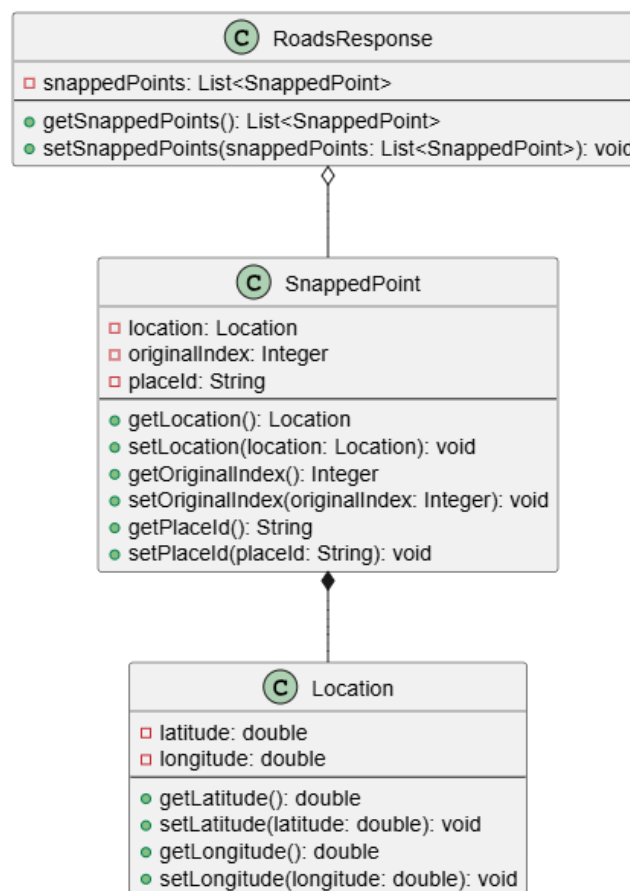
- **getGlobalSearchableTypes** - אוספת ומחזירה רשימה נקייה ומאוחדת של כל סוגי המקומות שמותר לשלוח בבקשות החיפוש הגיאוגרפיות.
- **getHighestScore** - מקבלת רשימת טיפוסים שחזרו מגוגל עבור מיקום מסוים, סורקת אותם, ומחזירה את ציון הבסיס הגבוה ביותר שניתן להתאים לנקודה זו מתוך הגדרות ה-enum.

המחלקה PlacesResponse



- **תפקיד המחלקה:** אובייקט ייעודי המשמש למיפוי (Deserialization) של תגובות ה-JSON החוזרות משרתי Google Places API, ומאפשר גישה נוחה לנתוני נקודות העניין שנמצאו בסביבת הרץ.
- **תכונות משמעותיות:**
 - **places** - רשימה של אובייקטים מטיפוס פנימי בשם Place, המכילים את המידע הגולמי על כל נקודה שנמצאה (מזהה, סוגים, דירוג, כמות מדרגים, מיקום ושם תצוגה).
- **פעולות משמעותיות:**
 - המחלקה מורכבת מתתי-מחלקות סטטיות ופנימיות המשקפות את המבנה ההיררכי של תגובת גוגל, ומחזיקה גטרים וסטרים בסיסיים בלבד בלי לוגיקה עסקית.

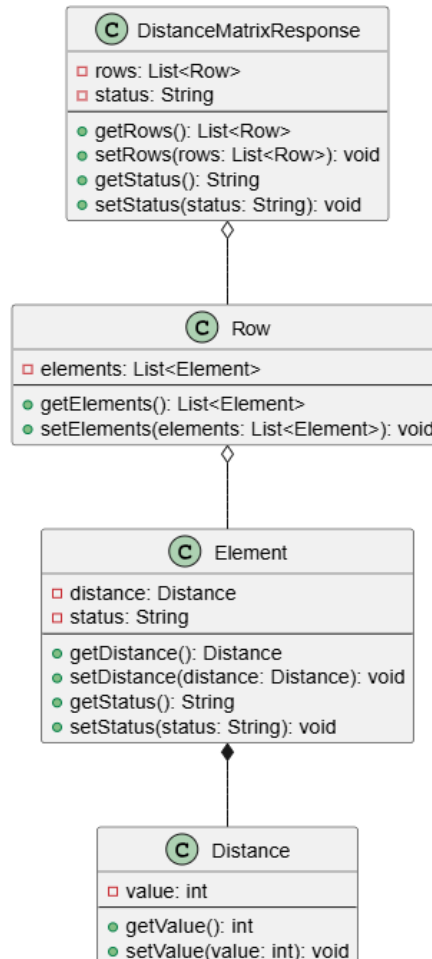
המחלקה RoadsResponse



- **תפקיד המחלקה:** אובייקט המיועד למיפוי תגובת ה-JSON החוזרת מ-Google Roads API, ומשמש לשליפת נקודות שננעלו בצורה מדויקת על כבישים או שבילים מוכרים במפה.
- **תכונות משמעותיות:**
 - **snappedPoints** - רשימת אובייקטים מטיפוס פנימי בשם SnappedPoint, המייצגים את נקודות הציון המתוקנות לאחר שהוצמדו לרשת הדרכים האמיתית, כולל ה-placeId של קטע הכביש הרלוונטי.
- **פעולות משמעותיות:**

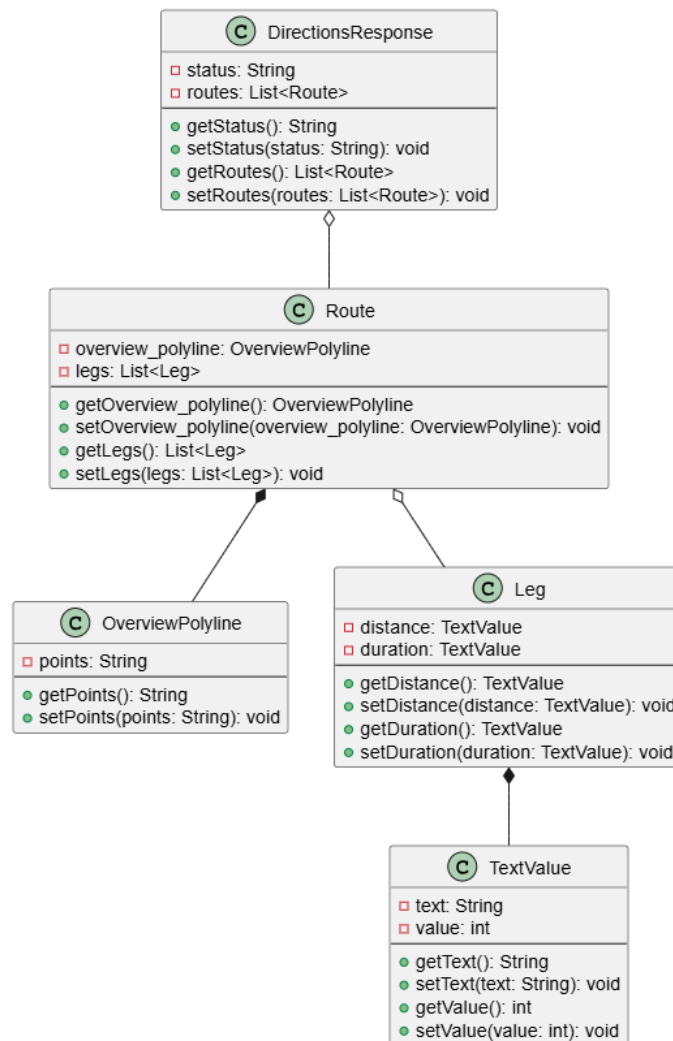
- בדומה לשאר מחלקות התגובה, היא מכילה מבנה היררכי פשוט של גטרים וסטרים המשמשים את תהליך ההמרה האוטומטי.

המחלקה DistanceMatrixResponse



- **תפקיד המחלקה:** אובייקט המשמש למיפוי תגובת ה-JSON של Google Distance Matrix API, ומאפשר לקבל את מטריצת המרחקים והזמנים המדויקים בין נקודות המוצא והיעדים השונים.
- **תכונות משמעותיות:**
 - **rows** - רשימה של שורות במטריצה, כאשר כל שורה מכילה רשימת אלמנטים המייצגים את נתוני המרחק הפיזי במטרים וסטטוס ההצלחה של החישוב עבור אותו זוג נקודות.
- **פעולות משמעותיות:**
 - מספקת מבנה נתונים תואם-API המאפשר לשירות המרכזי לשלוח בקלות את ערכי המרחקים הנדרשים לבניית הגרף.

המחלקה DirectionsResponse



- **תפקיד המחלקה:** אובייקט הממפה את תגובת ה-JSON מ-Google Directions API, המשמשת בשלב האחרון לשליפת נתוני הניווט היוזואליים עבור מסלול הריצה שנבחר.
- **תכונות משמעותיות:**
 - **routes** - רשימת מסלולי הניווט שחושבו, כאשר כל מסלול מכיל אובייקט פנימי בשם OverviewPolyline.
 - **points** - מחרוזת טקסט מקודדת הנמצאת בתוך אובייקט הפוליקלין, והיא זו שמייצגת את קו הניווט הגיאוגרפי השלם והמפותל שיוצג על גבי המפה במכשיר הקצה.
- **פעולות משמעותיות:**
 - משמשת כמבנה נתונים לקריאה בלבד עבור רכיב הפקת הפוליקלין של השרת.

המחלקה RouteService



- **תפקיד המחלקה:** מנהל העבודה המרכזי של הפרויקט. היא מתאמת בין כל שירותי ה-API החיצוניים של גוגל, בונה את רשת המיקומים, מכינה את הנתונים, ומפעילה את אלגוריתם ה-DPSO כדי להפיק את המסלול הסופי.
- **תכונות משמעותיות:**
 - **placesClient, roadsClient, distanceMatrixClient, directionsClient** - רכיבי רטרופיט (Retrofit Clients) המוזרקים למחלקה ומשמשים לביצוע קריאות רשת ישירות לשרתי גוגל השונים.
 - **dpsoAlgorithmService** - הרכיב המכיל את מנוע האופטימיזציה החכם שמחשב את סדר התחנות בתוך מגבלת המרחק.
 - **RouteContext** - מחלקת עזר פנימית המשמשת כקונטקסט דינמי לכל בקשה. היא מחשבת אוטומטית לפי אורך המסלול המבוקש פרמטרים כמו רדיוס חיפוש, כמות נקודות רצויה, גודל רשת התאים, כמות השכנים הקרובים, ומספר איטרציות וגודל נחיל מותאמים.

• פעולות משמעותיות:

- **calculateOptimizedRoute** - הפונקציה הראשית שמנהלת את הצינור התהליכי: יוצרת את הקונטקסט, בונה רשת תאים, אוספת נקודות, מסננת אותן לגודל המטרה, בונה גרף שכנים, שולפת מרחקי הליכה אמיתיים, מריצה את ה-DPSO, ומחזירה אובייקט מסלול שלם.
- **buildGridAndFetchPoints** - מחלקת את מרחב החיפוש הגיאוגרפי סביב הרץ לרשת של תאים (Grid) כדי להבטיח פיזור גיאוגרפי מיטבי, ומנסה לשלוף את הנקודה הטובה ביותר בכל תא.
- **fetchBestInCell** - מבצעת חיפוש מקומות פוטנציאליים בתוך רדיוס של תא ספציפי בעזרת Places API, ומחזירה את המיקום בעל הציון המשולב הגבוה ביותר.
- **fetchSnappedPoint** - במידה ולא נמצא מקום מעניין בתא, היא משתמשת ב-Roads API כדי למצוא נקודת כביש או צומת חוקיים קרובים כדי שהמסלול לא ייקטע.
- **pruneToTargetSize** - מבצעת גיזום וסינון של רשימת הנקודות שנאספו כדי להישאר עם כמות נקודות אופטימלית לחישוב, תוך מתן עדיפות לנקודות עם ה-reward הגבוה ביותר וקירבה יחסית למרכז.
- **getKNearestNeighbors** - בונה גרף קשרים ראשוני המגדיר עבור כל נקודה מיהן הנקודות הקרובות אליה ביותר מבחינה גיאומטרית, מה שמצמצם משמעותית את כמות קריאות המרחק היקרות מול גוגל.
- **fetchRealWalkingDistances** - פונה אל ה-Distance Matrix API ושולפת את מרחקי ההליכה והריצה האמיתיים על גבי כבישים ושבילים עבור כל קשתות הגרף שנבנה.
- **fetchOverviewPolyline** - מקבלת את רשימת התחנות של הפתרון הסופי ומפיקה מגוגל את מחרוזת הניווט היוזאלית המשובחת עבור המפה באפליקציה.

הממשק GoogleDirectionsClient

 <i>GoogleDirectionsClient</i>
 <code>getDirections(origin: String, destination: String, waypoints: String, mode: String): Call<DirectionsResponse></code>

- **תפקיד הממשק:** משמשת כממשק תקשורת מול שירות הניווט של גוגל, ומגדירה את קריאות הרשת הנדרשות לצורך הפקת מסלולי הניווט המפורטים של האפליקציה.

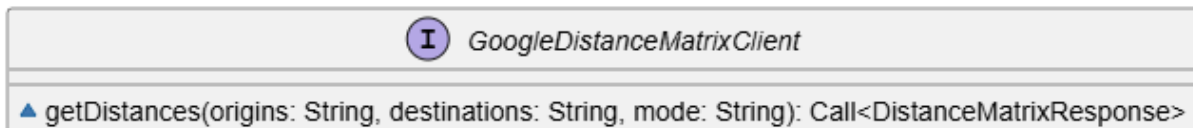
• תכונות משמעותיות:

- אין מחלקות ממשק אלו מחזיקות תכונות או משתני מחלקה, אלא רק הגדרות של פקודות רשת.

- **פעולות משמעותיות:**

- **getDirections** - מגדירה את קריאת ה-GET לשרתי גוגל כדי לשלוף את נתוני הניווט והמסלול המלאים בין נקודת המוצא ליעד, כולל מעבר דרך נקודות ביניים מוגדרות ותחת מצב תנועה של הליכה.

הממשק GoogleDistanceMatrixClient



- **תפקיד המחלקה:** משמשת כממשק תקשורת מול שירות מטריצת המרחקים של גוגל, ומאפשרת לאלגוריתם לקבל את זמני ומרחקי ההליכה האמיתיים בין קבוצות של נקודות במפה.

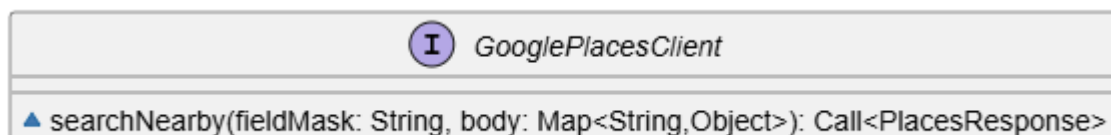
- **תכונות משמעותיות:**

- אין לממשק זה תכונות פנימיות משלו.

- **פעולות משמעותיות:**

- **getDistances** - מגדירה את קריאת ה-GET לחישוב מטריצת המרחקים והזמנים האמיתיים בין רשימה של קואורדינטות מוצא לרשימה של קואורדינטות יעד במצב הליכה.

הממשק GooglePlacesClient



- **תפקיד המחלקה:** משמשת כממשק תקשורת מול הגרסה החדשה של שירות המקומות של גוגל, ואחראית על שליפת נקודות עניין ויעדים פוטנציאליים בריצה סביב המשתמש.

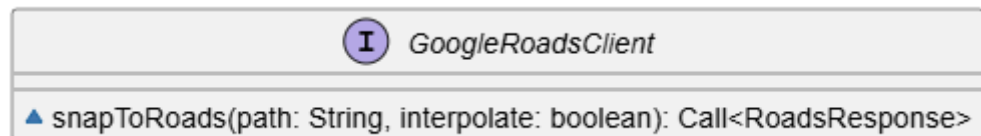
- **תכונות משמעותיות:**

- אין לממשק זה תכונות פנימיות משלו.

- **פעולות משמעותיות:**

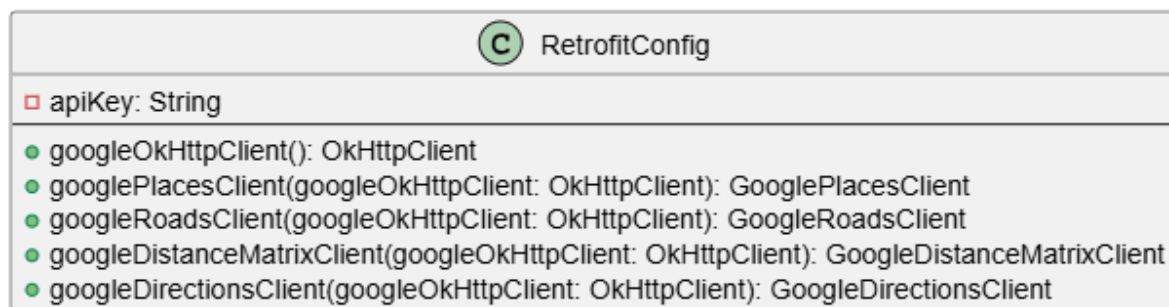
- **searchNearby** - מגדירה את קריאת ה-POST לחיפוש יעדים ונקודות עניין אטרקטיביות בסביבת המשתמש, תוך העברת כותרת של מסכת שדות לצמצום רחב הפס וגוף בקשה מובנה.

הממשק GoogleRoadsClient



- **תפקיד המחלקה:** משמשת כממשק תקשורת מול שירות הדרכים של גוגל, שמטרתו לתקן מיקומי מפה גולמיים ולהצמיד אותם לנתיבים הגיוניים.
- **תכונות משמעותיות:**
 - אין לממשק זה תכונות פנימיות משלו.
- **פעולות משמעותיות:**
 - **snapToRoads** - מגדירה את קריאת ה-GET המשמשת להצמדה ומגנוט של קואורדינטות גיאוגרפיות גולמיות אל כבישים, שבילים או נתיבי ריצה מוכרים הקרובים ביותר אליהן במפה.

המחלקה RetrofitConfig



- **תפקיד המחלקה:** מחלקת הגדרה וניהול (Configuration Class) ב-Spring Boot האחראית על הקמה, סנכרון והזרקה של כל לקוחות ה-HTTP ותשתית התקשורת מול שירותי המפות השונים של גוגל.
- **תכונות משמעותיות:**
 - **apiKey** - מחזיקה את מפתח הגישה הרשמי והמאובטח לשירותי גוגל, הנטען אוטומטית מתוך קובץ הגדרות הסביבה של האפליקציה.
- **פעולות משמעותיות:**
 - **googleOkHttpClient** - מייצרת ומגדירה את לקוח ה-HTTP המרכזי שמנהלת את רישום הלוגים של התקשורת ומזריקה אוטומטית את מפתח הגישה הן כפרמטר בשורת הכתובת והן ככותרת מאובטחת לכל בקשה יוצאת בהתאם לדרישות ה-API השונים.
 - **googlePlacesClient** - בונה ומנגישה את לקוח השרת עבור שירות המקומות, תוך הגדרת כתובת הבסיס הייעודית שלו ושילוב מנגנון המרת הנתונים האוטומטי לג'אווה.

- **googleRoadsClient** - בונה ומנגישה את לקוח השרת עבור שירות הדרכים, תוך שימוש בכתובת הבסיס המתאימה ובלקוח ה-HTTP המשותף שמנהל את האבטחה והלוגים.
- **googleDistanceMatrixClient** - בונה ומנגישה את לקוח השרת עבור שירות מטריצת המרחקים, המבוסס על כתובת המפות הכללית של גוגל.
- **googleDirectionsClient** - בונה ומנגישה את לקוח השרת עבור שירות הניווט וההנחיות הויזואליות, המשתפת את רכיבי התשתית וההמרה של שאר הלקוחות בפרויקט לעבודה אחידה.

המחלקה RunDataPoint

C RunDataPoint
□ latitude: double □ longitude: double □ altitude: double □ speed: float □ timestamp: long □ accuracy: float □ stepDelta: int □ isFirstPoint: boolean
● RunDataPoint() ● getLatitude(): double ● setLatitude(latitude: double): void ● getLongitude(): double ● setLongitude(longitude: double): void ● getAltitude(): double ● setAltitude(altitude: double): void ● getSpeed(): float ● setSpeed(speed: float): void ● getTimestamp(): long ● setTimestamp(timestamp: long): void ● getAccuracy(): float ● setAccuracy(accuracy: float): void ● getStepDelta(): int ● setStepDelta(stepDelta: int): void ● isFirstPoint(): boolean ● setFirstPoint(isFirstPoint: boolean): void

- **תפקיד המחלקה:** מייצגת נקודת מידע בודדת שנמדדה והוקלטה על ידי מכשיר האנדרואיד במהלך הריצה, ומאגדת את נתוני הטלמטריה של הרץ ברגע ספציפי בזמן.
- **תכונות משמעותיות:**
 - **latitude ו-longitude** - קואורדינטות גיאוגרפיות המייצגות את המיקום המדויק של הרץ על המפה בזמן המדידה.

- **altitude** - הגובה הגיאוגרפי של הנקודה מעל פני הים, המשמש לחישוב שינויי גובה לאורך הריצה.
- **speed** - המהירות הרגעית של המשתמש באותו רגע כפי שנקראה מחיישני המכשיר.
- **timestamp** - חותמת הזמן המדויקת של הדגימה, המשמשת לחישוב משך הריצה והקצב הלוגי.
- **accuracy** - רמת מדד הדיוק של קליטת ה-GPS באותו רגע, המשמשת לסיון וניקוי דגימות שגויות.
- **stepDelta** - כמות הצעדים שבוצעו בפועל מאז דגימת הנקודה הקודמת.
- **isFirstPoint** - דגל בוליאני המסמן האם זוהי הנקודה שפותחת את רצף המדידה הנוכחי.

• פעולות משמעותיות:

- במחלקה זו קיימים רק בנאים, גטרים וסטנדרטיים המשמשים לעדכון ושליפה של נתוני הטלמטריה, ללא לוגיקה עסקית מורכבת.

המחלקה RunDataSummary

C RunDataSummary
□ totalDistanceKm: double □ durationMillis: long □ averagePace: double □ totalElevationGain: double □ totalSteps: int
● RunDataSummary(totalDistanceKm: double, durationMillis: long, averagePace: double, totalElevationGain: double, totalSteps: int) ● RunDataSummary() ● getTotalDistanceKm(): double ● setTotalDistanceKm(totalDistanceKm: double): void ● getDurationMillis(): long ● setDurationMillis(durationMillis: long): void ● getAveragePace(): double ● setAveragePace(averagePace: double): void ● getTotalElevationGain(): double ● setTotalElevationGain(totalElevationGain: double): void ● getTotalSteps(): int ● setTotalSteps(totalSteps: int): void

- **תפקיד המחלקה:** מייצגת את אובייקט הסיכום הסטטיסטי של הריצה, המרכז את כל נתוני הביצועים הסופיים של המשתמש לאחר נעילת וסיום הפעילות.

• תכונות משמעותיות:

- **totalDistanceKm** - המרחק המצטבר הסופי שהרץ עבר לאורך כל המסלול, מחושב ביחידות של קילומטרים.
- **durationMillis** - משך הזמן נטו שבו המשתמש היה פעיל, מחושב במילישניות לאחר הפחתת זמני עצירה והפסקות.
- **averagePace** - הקצב הממוצע של הריצה כולה, המבוטא בדקות הנדרשות לעבירת קילומטר בודד.

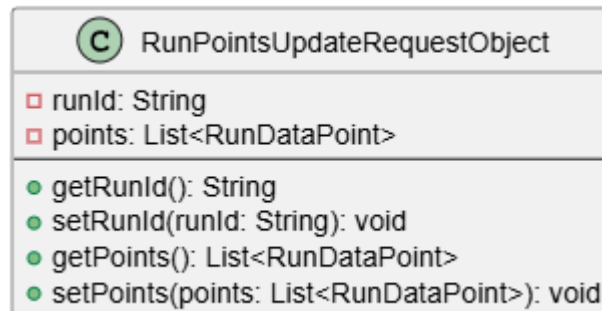
○ **totalElevationGain** - סך כל הגובה האנכי שהמשתמש טיפס במהלך הריצה, תוך התעלמות מירידות.

○ **totalSteps** - כמות הצעדים הכוללת שנצברה לאורך כל סשן הריצה.

• **פעולות משמעותיות:**

○ מחלקה זו משמשת כמבנה נתונים בלבד (DTO) ומכילה בנאים ופונקציות גישה בסיסיות לצורך העברת נתוני הסיכום לאפליקציה או שמירתם.

המחלקה RunPointsUpdateRequestObject



• **תפקיד המחלקה:** אובייקט העברת נתונים (DTO) המשמש את השרת לקבלת מקבץ או חבילה של נקודות ציון חדשות שנשלחו מהאפליקציה הניידת בזמן אמת כדי לעדכן ריצה פעילה.

• **תכונות משמעותיות:**

○ **runId** - המזהה הייחודי של סשן הריצה הנוכחי בשרת שאליה יש להזרים את נקודות המיקום החדשות.

○ **points** - רשימה מסודרת של אובייקטי נקודות דגימה שנאספו במכשיר הנייד מאז העדכון האחרון.

• **פעולות משמעותיות:**

○ מכילה גטרים וסטרים פשוטים המאפשרים לספריית ג'קסון לבצע המרה אוטומטית מקלט ה-JSON של ה-HTTP אל אובייקט Java.

המחלקה RunSession

C RunSession
- runId: String - userId: String - status: RunStatus - startTime: long - dataPoints: List<RunDataPoint> = new ArrayList<>() - summary: RunDataSummary
- RunSession() - RunSession(id: String, userId: String, status: RunStatus, dataPoints: List<RunDataPoint>, summary: RunDataSummary) - getRunId(): String - setRunId(id: String): void - getUserId(): String - setUserId(userId: String): void - getStatus(): RunStatus - setStatus(status: RunStatus): void - getDataPoints(): List<RunDataPoint> - setDataPoints(dataPoints: List<RunDataPoint>): void - getSummary(): RunDataSummary - setSummary(summary: RunDataSummary): void - getStartTime(): long - setStartTime(startTime: long): void

- **תפקיד המחלקה:** ישות הנתונים המרכזית (Entity) המייצגת ריצה שלמה במערכת. היא ממופה כנגד מסמך בקולקשן ייעודי במסד הנתונים מסוג מונגו, ומאגדת את נתוני המשתמש, מצב הפעילות, רצף הנקודות והסיכום הסטטיסטי.

• תכונות משמעותיות:

- **runId** - המפתח הראשי והמזהה הייחודי של הריצה במסד הנתונים.
- **userId** - מזהה המשתמש שביצע את הריצה, לצורך שיוך היסטוריית הפעילות שלו.
- **status** - מצב הסשן הנוכחי, המנוהל על ידי משתנה מסוג אנום המציין האם הריצה פעילה או הסתיימה.
- **dataPoints** - רשימה מובנית המכילה את כל נקודות הציון והמדדים שנאספו ונשמרים בצורה מקוננת בתוך מסמך הריצה.
- **summary** - אובייקט סיכום הסטטיסטיקות המלא של הריצה, המחושב ונעל רק עם הגעת הריצה לסיומה.

• פעולות משמעותיות:

- המחלקה מחזיקה בנאים ומתודות גישה רגילות, ומשמשת בעיקר כמודל הנתונים המרכזי שמולו עובד מאגר השמירה.

ה-RunStatus enum

- **תפקיד המחלקה:** קובץ אנום (Enum) פשוט המגדיר את מצבי המחזור החוקיים של ששן הריצה במערכת במטרה לשמור על עקביות הנתונים.

• תכונות משמעותיות:

- **ACTIVE** - ערך המציין כי הריצה נמצאת בעיצומה ברגעים אלו, והשרת רשאי להמשיך לקלוט ולשרשר עבורה נקודות מיקום חדשות.

- **FINISHED** - ערך המציין כי הריצה נעלה על ידי המשתמש, נתוני הסיכום חושבו בהצלחה, והסשן סגור לעדכונים נוספים.

• פעולות משמעותיות:

- כמחלקת אנום בסיסית, היא אינה מכילה פעולות לוגיות אלא משמשת כסוג נתונים קשיח עבור הסטטוס של הריצה.

המחלקה RunStatisticsController

 RunStatisticsController
 runService: RunStatisticsService
● startRun(userId: String): ResponseEntity<String> ● addPoints(request: RunPointsUpdateRequestObject): ResponseEntity<Void> ● finishRun(runId: String, pauseDurationMillis: long): ResponseEntity<RunSession> ● getUserHistory(userId: String): ResponseEntity<List<RunSession>>

- **תפקיד המחלקה:** רכיב הקונטרולר (REST Controller) החושף את נקודות הקצה של ה-API עבור רכיב הריצה, ומאפשר לאפליקציה הניידת לנהל את מחזור החיים של הריצה (התחלה, הזרמת נקודות, סיום וצפייה בהיסטוריה).

• תכונות משמעותיות:

- **runService** - רכיב שירות הלוגיקה העסקית המוזרק לקונטרולר ומבצע בפועל את פקודות השמירה והחישוב של הסטטיסטיקות.

• פעולות משמעותיות:

- **startRun** - מקבלת בקשת HTTP מסוג POST עם מזהה המשתמש, מייצרת סשן ריצה חדש ומחזירה את ה-id שלו כשתגובה.
- **addPoints** - מקבלת בקשת POST המכילה את אובייקט העדכון, ומעבירה את מקבץ הנקודות החדש לשמירה בתוך הריצה הפעילה התואמת.
- **finishRun** - מקבלת בקשת POST עם מזהה הריצה ומשך זמן ההפסקות, נועלת את הריצה, ומחזירה את אובייקט הריצה עם סיכום הסטטיסטיקות הסופי שחושב.
- **getUserHistory** - מקבלת בקשת POST עם מזהה המשתמש ושולפת את היסטוריית הריצות המלאה והמסודרת שלו, תוך ניהול תגובות HTTP מתאימות במקרה של רשימה ריקה או שגיאת שרת.

המחלקה RunSessionRepository

 RunSessionRepository
 findByUserIdOrderByStartTimeDesc(userId: String): List<RunSession>

- **תפקיד המחלקה:** ממשק (Interface) המשמש כמאגר נתונים מנוהל (Repository) המרחיב את תשתית מונגו רפוזיטורי של Spring Data, ומספק את כל פעולות התקשורת וה-CRUD הישירות מול מסד הנתונים עבור ישות הריצה.
- **תכונות משמעותיות:**
 - אין לממשקי מאגר נתונים משתני מחלקה משלהם, שכן המימוש שלהם נוצר אוטומטית על ידי תשתית ה-Spring.
- **פעולות משמעותיות:**
 - **findByUserIdOrderByStartTimeDesc** - פונקציית שליפה מותאמת אישית המוגדרת על פי מוסכמות השמות של Spring Data. היא מחפשת את כל מסמכי הריצה המשויכים למזהה משתמש ספציפי ומחזירה אותם כשהם ממוינים אוטומטית בסדר יורד לפי זמן התחלת הריצה, מהחדש ביותר לישן ביותר.

המחלקה RunStatisticsService

RunStatisticsService
- runRepository: RunSessionRepository - startRunSession(userId: String): String - addDataPoints(runId: String, newPoints: List<RunDataPoint>): void - finalizeRunStatistics(runId: String, pauseDurationMillis: long): RunSession - calculateDistance(lat1: double, lon1: double, lat2: double, lon2: double): double - getUserHistory(userId: String): List<RunSession>

- **תפקיד המחלקה:** הלב הלוגי ומחלקת השירות (Service) המרכזית האחראית על ניהול המצבים של הריצות, סכימה וסינון של נתוני ה-GPS, וחישוב מדדים מתמטיים מורכבים כמו קצב, מרחק ושינויי גובה אנכיים.
- **תכונות משמעותיות:**
 - **runRepository** - רכיב הגישה למסד הנתונים המוזרק לשירות ומאפשר לו לקרוא, לעדכן ולשמור את מסמכי הריצות השונים.
- **פעולות משמעותיות:**
 - **startRunSession** - מייצרת אובייקט ריצה חדש, משייכת אותו למשתמש, קובעת לו סטטוס פעיל ואת חותמת זמן הנוכחית של השרת, ומבצעת הזרקה ראשונית למסד הנתונים.
 - **addDataPoints** - מאתרת את הריצה הפעילה לפי המזהה שלה, מוסיפה את רשימת נקודות המיקום החדשות שהתקבלו מהמכשיר הנייד לתוך המאגר הקיים, ומבצעת שמירה מעודכנת.
 - **finalizeRunStatistics** - פונקציית סיום הריצה המרכזית. היא שולפת את כל נקודות הריצה שנאספו, רצה עליהן בלולאה ומבצעת סכימת מרחקים (רק עבור נקודות בעלות

רמת דיוק סבירה), סוכמת עליות גובה משמעותיות (תוך שימוש במנגנון סינון רעשים של חצי מטר), וסוכמת צעדים. בנוסף, היא מחשבת את זמן הנטו הפעיל על ידי הפחתת זמני ההפסקות, מפיקה אובייקט סיכום מלא, מעדכנת את מצב הריצה ומבצעת שמירה סופית.

- **calculateDistance** - פונקציית עזר מתמטית פרטית הממשת את נוסחת האברסין לחישוב המרחק הפיזי המדויק במטרים בין שתי נקודות על גבי כדור הארץ בהינתן קווי הרוחב וקווי האורך שלהן.
- **getUserHistory** - פונה אל מאגר הנתונים ומפעילה את השילוב הממוין של היסטוריית הריצות עבור המשתמש המבוקש.

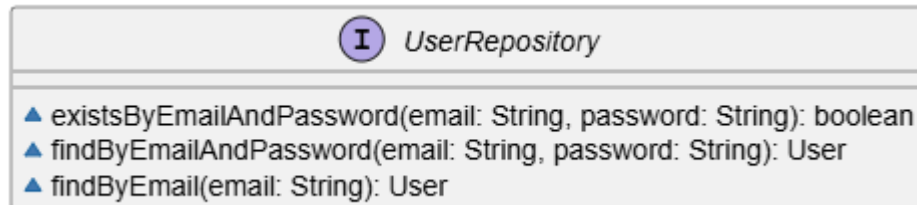
המחלקה User

C User
□ email: String □ username: String □ password: String □ weightKg: double □ heightCm: double □ birthDate: long □ createdAt: long
● User() ● User(username: String, email: String, password: String, weightKg: double, heightCm: double, birthDate: long) ● getUsername(): String ● setUsername(username: String): void ● getEmail(): String ● setEmail(email: String): void ● getPassword(): String ● setPassword(password: String): void ● getWeightKg(): double ● setWeightKg(weightKg: double): void ● getHeightCm(): double ● setHeightCm(heightCm: double): void ● getBirthDate(): long ● setBirthDate(birthDate: long): void ● getCreatedAt(): long ● setCreatedAt(createdAt: long): void ● setCreatedAt(): void

- **תפקיד המחלקה:** מייצגת את ישות המשתמש במערכת וממופה כמסמך בתוך קולקשן ייעודי במסד הנתונים מונגו, שבו נשמרים פרטי הפרופיל והמדדים של הרץ.
- **תכונות משמעותיות:**
 - **email** - משמש כמפתח הראשי והמזהה הייחודי של המשתמש בבסיס הנתונים (מסומן באנוטציית Id).
 - **username, password, weightKg, heightCm, birthDate, createdAt** - נתוני הפרופיל, הסיסמה, המדדים הפיזיים ומועדי הרישום של המשתמש במערכת.
- **פעולות משמעותיות:**

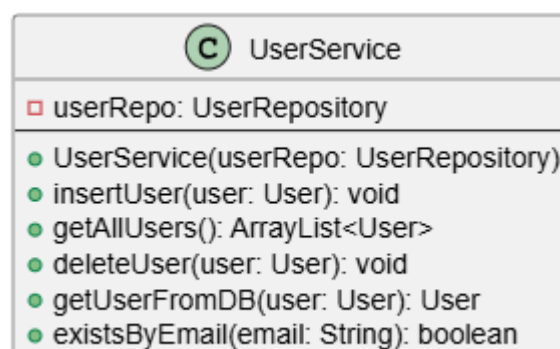
- **User** - בנאים (ריק ומלא) המאפשרים את יצירת האובייקט ואתחול אוטומטי של זמן יצירת החשבון, הנדרשים לצורך עבודה תקינה מול מונגו וספריית ג'קסון.

הממשק UserRepository



- **תפקיד הממשק:** ממשק גישה לנתונים המרחיב את מונגו רפוזיטורי של Spring Data, ומספק את פעולות התקשורת והשאלות הישירות מול מסד הנתונים עבור ישות המשתמש.
- **תכונות משמעותיות:**
 - אין לממשק זה תכונות או משתני מחלקה פנימיים משלו.
- **פעולות משמעותיות:**
 - **existsByEmailAndPassword** - בודקת בצורה מהירה האם קיים כבר במסד הנתונים משתמש בעל שילוב אימייל וסיסמה ספציפיים
 - **findByEmailAndPassword** - מחפשת ושולפת את אובייקט המשתמש המלא על פי השילוב של האימייל והסיסמה שלו.
 - **findByEmail** - מחפשת ושולפת את אובייקט המשתמש המלא על פי האימייל שלו.

המחלקה UserService



- **תפקיד המחלקה:** מחלקת השירות המכילה את הלוגיקה העסקית לניהול המשתמשים, אימות נתונים, והפעלת מנגנוני הבקרה לפני ביצוע שינויים במסד הנתונים.
- **תכונות משמעותיות:**
 - **userRepo** - רכיב הגישה למסד הנתונים המוזרק למחלקה ומאפשר לבצע את פעולות השמירה, המחיקה והחיפוש בפועל.

• פעולות משמעותיות:

- **insertUser** - מקבלת אובייקט משתמש חדש, מוודאת מול מסד הנתונים שהוא לא קיים, ומבצעת שמירה. במידה והוא נמצא, היא זורקת חריגה מתאימה המונעת את הרישום.
- **getUserFromDB** - פונה אל מאגר הנתונים כדי לאתר ולשלוח משתמש ספציפי על פי פרטי הזהוי שלו לצורך המשך תהליכי האימות במערכת.

המחלקה LoginAndSignupController

C LoginAndSignupController
userService: UserService - LoginAndSignupController(userService: UserService) - login(user: User): ResponseEntity<User> - signup(user: User): ResponseEntity<String> - checkUserExists(email: String): ResponseEntity<Void>

- **תפקיד המחלקה:** רכיב הבקרה החושף את נקודות הקצה של ה-API עבור תהליכי הרישום וההתחברות של המשתמשים, ומנהל את החזרת תגובות ה-HTTP המתאימות למכשיר הנייד.

• תכונות משמעותיות:

- **userService** - רכיב הלוגיקה העסקית המוזרק למחלקה ומנהל את פעולות האימות והרישום מאחורי הקלעים.

• פעולות משמעותיות:

- **login** - מקבלת בקשת POST עם פרטי המשתמש, קוראת לשירות האימות, ומחזירה תגובת הצלחה עם אובייקט המשתמש המלא או קוד שגיאה 401 במקרה של פרטים לא נכונים.
- **signup** - מקבלת בקשת POST לרישום משתמש חדש, בודקת זמינות, מעדכנת את חותמת זמן היצירה ומבצעת שמירה, תוך החזרת קוד הצלחה 200, קוד 409 במקרה של משתמש קיים, או קוד 500 במקרה של שגיאת שרת פנימית.
- **checkUserExists** - מקבלת בקשת GET עם האימייל של המשתמש על מנת למצוא האם קיים כבר משתמש עם אימייל זה.

בצד הלקוח (אנדרואיד סטודיו):

המחלקה MainActivity

C MainActivity
□ navbar : BottomNavigationView ◆ onCreate(savedInstanceState : Bundle) : void ■ getFragmentOfItem(item : MenuItem) : Fragment ■ switchFragment(fragment : Fragment) : boolean

- **תפקיד המחלקה:** מסך הבית המרכזי של האפליקציה המנהל את הניווט הראשי בין האזורים השונים באמצעות תפריט תחתון והחלפת תצוגות (Fragments) מבוססות הקשר.
- **תכונות משמעותיות:**
 - **navbar** - רכיב תפריט הניווט התחתון המאפשר למשתמש לעבור בין המסכים השונים באפליקציה.
 - **user** - משתנה המחזיק את נתוני המשתמש המחובר הנוכחי.
- **פעולות משמעותיות:**
 - **onCreate** - פונקציית האתחול של המסך שמגדירה את פריסת הממשק, מאזינה ללחיצות בתפריט הניווט ומציגה את מסך החשבון כברירת מחדל בעת פתיחת האפליקציה.
 - **getFragmentOfItem** - פונקציית עזר המקבלת את הפריט שנבחר בתפריט הניווט ומחזירה את קטע המסך המתאים לו (פרופיל חשבון, הגדרת ריצה או היסטוריית ריצות).
 - **switchFragment** - פונקציה האחראית על ביצוע תהליך החלפת קטעי המסך המוצגים למשתמש בזמן אמת בתוך מכולת התצוגה של האקטיביטי.

המחלקה LoginActivity

C LoginActivity
□ loginBtn : MaterialButton □ signupPassBtn : MaterialButton □ emailETxt : TextInputEditText □ passwordETxt : TextInputEditText □ titleTxtV : TextView ◆ onCreate(savedInstanceState : Bundle) : void ■ activateLogin() : void

- **תפקיד המחלקה:** מסך ההתחברות של האפליקציה המאפשר למשתמש רשום להזין את פרטי הזיהוי שלו, לאמת אותם מול שרת ה-Spring Boot ולעבור למסך הבית.

- **תכונות משמעותיות:**

- **loginBtn** - כפתור ייעודי המפעיל את תהליך אימות הפרטים וההתחברות בעת הלחיצה.
- **signupPassBtn** - כפתור המעביר את המשתמש אל מסך ההרשמה במידה ואין לו חשבון קיים במערכת.
- **emailETxt** - שדה קלט טקסטואלי שבו המשתמש מזין את כתובת הדואר האלקטרוני שלו.
- **passwordETxt** - שדה קלט טקסטואלי מאובטח להזנת סיסמת החשבון.

- **פעולות משמעותיות:**

- **onCreate** - מאתחלת את רכיבי הממשק היוזואליים ומגדירה את המאזינים ללחיצות על כפתורי ההתחברות והמעבר למסך ההרשמה.
- **activateLogin** - פונקציה המוודאת ששדות הקלט אינם ריקים, בונה אובייקט משתמש חלקי ומבצעת קריאת רשת אסינכרונית לשרת באמצעות רטרופיט כדי לאמת את פרטי הגישה. במקרה של הצלחה, היא שומרת את נתוני המשתמש המלאים בזיכרון הזמני ועוברת למסך הבית.

המחלקה RouteSetupActivity

RouteSetupActivity
□ btnStart : MaterialButton □ sliderDistance : Slider □ switchCustomized : MaterialSwitch □ switchCircular : MaterialSwitch □ userLocation : LatLng □ selectedCategories : ArrayList<String>
◆ onCreate(savedInstanceState : Bundle) : void ■ getOptimizedRouteFromServer() : void ■ sendRequestToServer(progressDialog : ProgressDialog) : void ■ showRoutePreviewOnMap(route : OptimizedRoute) : void ■ toggleViewHierarchy(viewGroup : ViewGroup, isEnabled : boolean) : void ■ startRunRecording() : void ● setChosenCategories() : void ● checkPermissionsAndStartRunAction() : void ● onRequestPermissionsResult(requestCode : int, permissions : String[], grantResults : int[], deviceId : int) : void ■ showSettingsDialog() : void ■ showPermissionRationaleDialog() : void

- **תפקיד המחלקה:** מסך הגדרת המסלול המאפשר למשתמש לקבוע את פרמטרי הריצה הרצויים (מרחק וסוג מסלול) ולבקש מסלול מותאם אישית ומבוסס מיקום גיאוגרפי מהשרת.

- **תכונות משמעותיות:**

- **btnStart** - כפתור המפעיל את בדיקת ההרשאות ותחילת הפעילות.
- **sliderDistance** - רכיב בחירה גורר המאפשר למשתמש לקבוע את המרחק המבוקש עבור הריצה הנוכחית.

- **switchCustomized** - מתג לקביעה האם לרוץ במסלול חופשי או להפעיל את מנגנון האופטימיזציה ולבקש מסלול מחושב מהשרת.
- **switchCircular** - מתג המגדיר האם המסלול המבוקש מהשרת צריך להיות מעגלי ולחזור לנקודת ההתחלה.
- **userLocation** - משתנה השומר את נקודת המיקום הגיאוגרפית הנוכחית של המשתמש כפי שנקראה מחיישני ה-GPS של המכשיר.

• פעולות משמעותיות:

- **onCreate** - מאתחלת את פקדי הממשק, מעצבת את תצוגת תוויות המרחק ומגדירה מאזין לשינוי מצב המתג לצורך השבתה או הפעלה ויזואלית של אילוצי הריצה.
- **getOptimizedRouteFromServer** - מציגה דיאלוג טעינה למשתמש ומפעילה את רכיב המיקום כדי לשלוף את קואורדינטות ה-GPS העדכניות של המכשיר בצורה מאובטחת.
- **sendRequestToServer** - שולחת בקשת רשת לשרת עם נתוני המיקום, המרחק המבוקש במטרים ואופי המסלול, ומעבירה את התוצאה המוצלחת להצגה על המפה.
- **showRoutePreviewOnMap** - מייצרת ומציגה כרטיסיית תצוגה מקדימה תחתונה שמציגה את קו הניווט שחושב על גבי המפה הויזואלית עבור המשתמש.
- **toggleViewHierarchy** - פונקציה רקורסיבית המשביתה או מפעילה קבוצות של רכיבי ממשק בהתאם לבחירות המשתמש כדי למנוע קלטים סותרים.
- **checkPermissionsAndStartRunAction** - סורקת את מערך ההרשאות הנדרשות ומוודאת שכולן מאושרות לפני פנייה לשרת או מעבר להקלטת הריצה הפיזית.
- **onRequestPermissionsResult** - מטפלת בתשובת המשתמש לבקשת ההרשאות, ומנווטת להצגת דיאלוג הסבר או להגדרות המכשיר במקרה של סירוב קבוע.

המחלקה RunRecordingActivity

RunRecordingActivity
□ gMap : GoogleMap □ navigationMarker : Marker □ arrowIcon : BitmapDescriptor □ isInitializedLocation : boolean □ polylines : List<Polyline> □ currentPathSegment : List<LatLng> □ plannedRoute : List<LatLng> □ distanceTextView : TextView □ timerTextView : TextView □ isPaused : boolean □ pauseResumeButton : MaterialButton □ stopButton : MaterialButton □ serverRunId : String ○ <u>REQUIRED_PERMISSIONS</u> : String[] □ requestsClient : ServerEndpointsAPI □ runUpdateReceiver : BroadcastReceiver □ runFinishedReceiver : BroadcastReceiver
◇ onCreate(savedInstanceState : Bundle) : void ◇ onResume() : void ◇ onPause() : void ■ togglePauseResume() : void ■ stopRun() : void ○ onMapReady(googleMap : GoogleMap) : void ■ startNewPolylineSegment() : void ■ startRunService() : void ■ updateMapUI(location : Location) : void ■ getBitmapFromVector(vectorResourceId : int) : BitmapDescriptor ■ drawPlannedRoute() : void ■ checkPermissionsAndStartService() : void

- **תפקיד המחלקה:** מסך ההקלטה הפעיל המציג את מפת הניווט בזמן אמת, משרטט את נתיב הריצה הנוכחי ומציג למשתמש נתונים חיים של זמן ומרחק מתוך שירות המעקב הפועל ברקע.
- **תכונות משמעותיות:**
 - **gMap** - אובייקט המפה של גוגל המשמש להצגת המיקום ונתיב הריצה באופן ויזואלי על המסך.
 - **navigationMarker** - סמן המפה המייצג את המיקום והכיוון הנוכחי של הרץ באמצעות חץ גיאוגרפי.
 - **polylines** - רשימה של קווי מפה המשמשים לשרטוט חלקי המסלול השונים על גבי המסך בצורה רציפה.
 - **currentPathSegment** - רשימת קואורדינטות המייצגת את מקטע הריצה הנוכחי שנצבר מאז עצירת ההשהיה האחרונה.
 - **distanceTextView** - רכיב טקסט המציג למשתמש את מרחק הריצה המצטבר במטרים.
 - **timerTextView** - רכיב טקסט המציג את הזמן הפעיל והמדויק של הריצה בפורמט שעות.

- **pauseResumeButton** - כפתור המאפשר להשהות את הריצה או להמשיך אותה בכל שלב.
- **runUpdateReceiver** - מקלט שידורים פנימי המאזין לשידורי עדכוני המיקום והזמן שנשלחים משירות המעקב ברקע ומעדכן את רכיבי הממשק בהתאם.
- **requestsClient** - לקוח ה-API המשמש לביצוע קריאות רשת ישירות מול שרת ה-Spring Boot.

• פעולות משמעותיות:

- **onCreate** - מגדירה את פריסת הממשק, מאתחלת את רכיבי המפה ומקשרת את כפתורי השליטה לפונקציות העצירה וההשהיה.
- **onPause**-ו **onResume** - מנהלות את רישום וביטול הרישום של מקלט השידורים הדינמי כדי לחסוך במשאבי מערכת וסוללה כשהמסך אינו מוצג ישירות.
- **togglePauseResume** - משנה את מצב הריצה, מעדכנת את חזות הכפתור בהתאם לסטטוס (כתום להשהיה, ירוק להמשך) ושולחת פקודה מתאימה לשירות המעקב ברקע.
- **stopRun** - עוצרת ומכבה את שירות מעקב הרקע של המכשיר וסוגרת את מסך ההקלטה הנוכחי.
- **onMapReady** - נקראת כאשר המפה מוכנה לשימוש, מגדירה את מאפייני התצוגה שלה, מזיזה את המצלמה למיקום ברירת מחדל ויוצרת את מקטע קו הריצה הראשון.
- **startRunService** - פונה לשרת כדי לפתוח סשן ריצה רשמי, מקבלת את מזהה הריצה שהופק ומפעילה איתו את שירות מעקב הרקע של המכשיר במצב פורגראונד מאובטח.
- **updateMapUI** - מקבלת עדכון מיקום חדש, מוסיפה אותו לנתיב, מעדכנת את מיקום וסיבוב הסמן בהתאם לכיוון התנועה ומזיזה את מצלמת המפה בהתאם לקצב של הרץ.

המחלקה RunStatisticsActivity

C RunStatisticsActivity
□ elevationChart : LineChart □ speedChart : LineChart □ tvDistance : TextView □ tvTime : TextView □ tvPace : TextView □ tvCalories : TextView
◆ onCreate(savedInstanceState : Bundle) : void ■ updateUI(session : RunSession) : void ■ setupElevationChart(points : List<RunDataPoint>) : void ■ setupSpeedChart(points : List<RunDataPoint>) : void ■ styleChart(chart : LineChart, yAxisLabel : String) : void ■ formatDuration(millis : long) : String ■ formatPace(pace : double) : String ◆ onDestroy() : void

- **תפקיד המחלקה:** מסך סיכום סטטיסטי המציג נתוני ביצועים מלאים וגרפים ויזואליים של מהירות וגובה עבור ריצה שנבחרה מההיסטוריה או הסתיימה ברגע זה.
- **תכונות משמעותיות:**
 - **elevationChart** - גרף קווי המציג את שינויי הגובה הגיאוגרפיים של הרץ לאורך זמן הריצה בקשר למיקומו במפה.
 - **speedChart** - גרף קווי המציג את שינויי המהירות וקצב הריצה של המשתמש לאורך דקות הפעילות.
 - **tvDistance, tvTime, tvPace, tvCalories** - רכיבי טקסט ייעודיים להצגת מדדי המרחק, הזמן, הקצב הממוצע ושריפת הקלוריות המוערכת של הפעילות.
- **פעולות משמעותיות:**
 - **onCreate** - מקשרת את פקדי הממשק, שולפת את אובייקט הריצה מתוך מחזיק הזיכרון הזמני הסטטי SelectedRunHolder ומפעילה את עדכון רכיבי המסך.
 - **updateUI** - מחלצת את נתוני הסיכום, מציגה את ערכי המרחק והזמן, מחשבת הערכת קלוריות משוערת על בסיס משקל ומרחק, ומפעילה את בניית הגרפים.
 - **setupElevationChart** - מעבדת את רשימת נקודות הריצה, מחשבת את זמן הדגימה היחסי בדקות ומשרטטת גרף גובה מעוצב וממולא בצבע ירוק.
 - **setupSpeedChart** - מעבדת את נקודות המיקום, ממירה את נתוני המהירות הגולמיים מהחיישנים לקמ"ש ומשרטטת גרף מהירות מעוצב בצבע כחול לאורך ציר הזמן.
 - **styleChart** - פונקציית עזר פרטית המעצבת את צירי ה-X וה-Y של הגרפים, מסירה רכיבי עיצוב מיותרים ומגדירה תבניות תצוגה מותאמות ליחידות המדידה (דקות, מטרים, קמ"ש).

- **onDestroy** - מנקה את אובייקט הריצה מזיכרון המחזיק הסטטי כשהמסך נסגר כדי למנוע זליגות זיכרון במכשיר.

המחלקה SignupActivity

SignupActivity
- signupBtn : MaterialButton - loginPassBtn : MaterialButton - usernameETxt : TextInputEditText - emailETxt : TextInputEditText - passwordETxt : TextInputEditText - weightETxt : TextInputEditText - heightETxt : TextInputEditText - birthDateETxt : TextInputEditText - selectedBirthDateTimestamp : long
- onCreate(savedInstanceState : Bundle) : void - showDatePicker() : void - registerUser() : void

- **תפקיד המחלקה:** מסך ההרשמה של האפליקציה המאפשר למשתמשים חדשים להזין את פרטיהם האישיים ומדדיהם הפיזיים כדי ליצור חשבון חדש בשרת המערכת.

• תכונות משמעותיות:

- **signupBtn** - כפתור המפעיל את בדיקת השדות ותהליך הרישום הרשמי מול השרת בעת הלחיצה.
- **loginPassBtn** – כפתור המעביר את המשתמש למסך ההתחברות במידה ויש לו כבר חשבון קיים.
- **usernameETxt, emailETxt, passwordETxt, weightETxt, heightETxt, birthDateETxt** - שדות קלט ייעודיים לקליטת שם המשתמש, הדואר האלקטרוני, הסיסמה, המשקל, הגובה ותאריך הלידה של הנרשם.
- **selectedBirthDateTimestamp** - משתנה מספרי השומר את תאריך הלידה שנבחר כערך זמן ארוך (Timestamp) לצורך שמירה מדויקת ואחידה במסד הנתונים.

• פעולות משמעותיות:

- **onCreate** - מחברת את רכיבי הממשק היוזואליים ומגדירה מאזיני לחיצה ייעודיים עבור שדה תאריך הלידה וכפתור ההרשמה הראשי.
- **showDatePicker** - מציגה חלונת בחירת תאריך מובנית (DatePickerDialog) המוגדרת מראש, ומעדכנת את השדה היוזואלי ואת משתנה ה-Timestamp בהתאם לבחירת המשתמש.
- **registerUser** - פונקציית הרישום המרכזית. היא מוודאת שכל השדות מלאים ותקינים, ממירה את ערכי המשקל והגובה למספרים, מייצרת אובייקט משתמש מלא, מציגה דיאלוג טעינה ומבצעת קריאת רשת לשרת ה-Spring Boot. במקרה של הצלחה היא מציגה הודעת אישור ומעבירה את המשתמש למסך ההתחברות.

המחלקה SplashActivity

C	SplashActivity
□	apiClient : ServerEndpointsAPI
◆	onCreate(savedInstanceState : Bundle) : void
■	verifyUserWithServer(email : String) : void
■	navigateToLogin() : void

- **תפקיד המחלקה:** מסך פתיחה (Splash Screen) המשמש כנקודת הכניסה הראשונית לאפליקציה, האחראי על בדיקת מצב ההתחברות של המשתמש וניתובו למסך המתאים (מסך הבית או מסך ההתחברות) בהתאם לתקינות הסשן שלו מול השרת.
- **תכונות משמעותיות:**
 - **apiClient** – רכיב ממשק מסוג ServerEndpointsAPI המשמש לביצוע קריאות רשת אסינכרוניות מול נקודות הקצה של השרת לצורך אימות המשתמש.
- **פעולות משמעותיות:**
 - **onCreate** – מתודה המופעלת עם יצירת המסך, הבודקת באופן ראשוני מול הזיכרון המקומי האם המשתמש מחובר; אם לא, היא מנתבת אותו למסך ההתחברות, ואם כן, היא פונה לאימות אקטיבי מול השרת.
 - **verifyUserWithServer** – מבצעת קריאת רשת לבדיקת קיומו של המשתמש במסד הנתונים; במקרה של הצלחה המשתמש מועבר למסך הבית, במקרה של שגיאה מבוצע ניתוק מקומי והעברה למסך ההתחברות, ובמצב של חוסר קליטה מתאפשרת כניסה במצב לא מקוון על בסיס הזיכרון המקומי.
 - **navigateToLogin** – פונקציית עזר המעבירה את המשתמש אל מסך ה-LoginActivity וסוגרת את מסך הפתיחה הנוכחי.

המחלקה AccountFragment

C	AccountFragment
□	tvInitial : TextView
□	tvName : TextView
□	tvEmail : TextView
□	tvWeight : TextView
□	tvHeight : TextView
□	tvBirth : TextView
□	tvJoined : TextView
□	btnLogout : MaterialButton
●	AccountFragment()
●	onCreateView(inflater : LayoutInflater, container : ViewGroup, savedInstanceState : Bundle) : View
●	onViewCreated(view : View, savedInstanceState : Bundle) : void
■	populateUserData(user : User) : void

- **תפקיד המחלקה:** קטע תצוגה (Fragment) המייצג את מסך פרופיל המשתמש באפליקציה, האחראי על הצגת הנתונים האישיים והמדדים הגופניים של הרץ המחובר, וכן על מתן אפשרות להתנתקות מהחשבון.
- **תכונות משמעותיות:**
 - **tvInitial, tvName, tvEmail** - רכיבי טקסט המשמשים להצגת האות הראשונה של שם המשתמש בתוך עיגול האוואטר, שמו המלא וכתובת הדואר האלקטרוני שלו.
 - **tvWeight, tvHeight, tvBirth, tvJoined** - רכיבי טקסט המציגים את מדדי המשקל והגובה הנוכחיים של הרץ, לצד תאריך הלידה שלו ומועד הצטרפותו לאפליקציה.
 - **btnLogout** - כפתור המאפשר למשתמש להתנתק מהחשבון הנוכחי, המנקה את נתוני הזיכרון ומחזיר אותו למסך ההתחברות.
- **פעולות משמעותיות:**
 - **onViewCreated** - מקשרת את רכיבי הממשק היוזואליים, שולפת את נתוני המשתמש מתוך רכיב הסינגלטון המחזיק את המידע ומגדירה מאזין לחיצה עבור כפתור ההתנתקות.
 - **populateUserData** - מקבלת את אובייקט המשתמש, מעדכנת את שדות הטקסט השונים, ומבצעת עיצוב של המדדים המספריים והמרת חותמות הזמן הדיגיטליות לתאריכים קריאים במבנה המקובל.

המחלקה HistoryFragment

HistoryFragment
- recyclerView : RecyclerView - progressBar : ProgressBar - adapter : RunHistoryAdapter - runList : List<RunSession> - requestsClient : ServerEndpointsAPI
- HistoryFragment() - onCreateView(inflater : LayoutInflater, container : ViewGroup, savedInstanceState : Bundle) : View - onViewCreated(view : View, savedInstanceState : Bundle) : void - fetchUserHistory() : void

- **תפקיד המחלקה:** קטע תצוגה (Fragment) האחראי על הצגה וניהול של מסך היסטוריית הפעילות של המשתמש, השולף בצורה אסינכרונית את רשימת כל ריצות העבר מהשרת ומציג אותן ברשימה דינמית גוללת.

- **תכונות משמעותיות:**

- **recyclerView** - רכיב תצוגה רשימתי מתקדם המשמש לפריסת והצגת הריצות הקודמות של המשתמש בצורה גוללת וחסכונית במשאבים.
- **progressBar** - רכיב חיווי המציג רכיב טעינה מעגלי בזמן שהאפליקציה ממתינה לקבלת נתוני ההיסטוריה מהשרת.
- **adapter** - רכיב מתווך (Adapter) האחראי על החיבור הלוגי והזרקת הנתונים מתוך רשימת הריצות אל תוך רכיבי הממשק היוזאליים של הרשימה.
- **runList** - רשימה מובנית המחזיקה את אוסף אובייקטי סשני הריצה שהתקבלו מהשרת עבור המשתמש הנוכחי.
- **requestsClient** - לקוח ה-API המשמש לביצוע קריאות הרשת הישירות מול שרת ה-Spring Boot.

- **פעולות משמעותיות:**

- **onViewCreated** - מאתחלת את פקדי הממשק, מגדירה את מנהל הפריסה והמתאם עבור רשימת התצוגה, ומזמנת את פונקציית שליפת הנתונים.
- **fetchUserHistory** - מפעילה ומציגה את רכיב הטעינה, שולפת את מזהה המשתמש הנוכחי, ומבצעת פנייה לשרת לקבלת רשימת הריצות. בעת תגובה מוצלחת היא מעדכנת את רשימת הנתונים, מעדכנת את הרשימה היוזאלית ומחביאה את חיווי הטעינה.

המחלקה RoutePreviewFragment

C RoutePreviewFragment
□ mMap : GoogleMap □ tvDistance : TextView □ tvElevation : TextView □ startRecordingBtn : MaterialButton □ mapCard : CardView □ mapPolyline : String
● onCreateView(inflater : LayoutInflater, container : ViewGroup, savedInstanceState : Bundle) : View ● onViewCreated(view : View, savedInstanceState : Bundle) : void ● onMapReady(googleMap : GoogleMap) : void ■ drawRouteAndZoom() : void ■ startRunRecording() : void

- **תפקיד המחלקה:** כרטיסיית תצוגה מקדימה תחתונה (BottomSheetDialogFragment) הנפתחת מעל מסך הגדרת הריצה, המציגה למשתמש את תוצאות אופטימיזציית המסלול שחושב בשרת על גבי מפה אינטראקטיבית ומאפשרת לו לצאת ישירות להקלטת הריצה.

- **תכונות משמעותיות:**

- **mMap** - אובייקט המפה של גוגל המשמש להצגה, ציור וניהול של שכבות הניווט הגיאוגרפיות.
- **tvDistance, tvElevation** - רכיבי טקסט המציגים למשתמש את נתוני המרחק הכולל המדויק ונתוני הטיפוס של המסלול המוצע.
- **startRecordingBtn** - כפתור הפעלה המעביר את המשתמש ישירות אל מסך ההקלטה הפעיל בזמן אמת.
- **mapCard** - רכיב כרטיסייה המכיל את המפה, המשמש כבסיס לפתרון התנגשויות מגע בין מחוות הגלילה הצידית של המפה לבין מחוות הגרירה מעלה ומטה של הכרטיסייה התחתונה.
- **mapPolyline** - מחרוזת טקסט מקודדת המייצגת את קו הניווט הגיאוגרפי של המסלול שהתקבל כפלט מהשרת.

- **פעולות משמעותיות:**

- **onViewCreated** - מחלצת את נתוני המרחק והנתיב מתוך הארגומנטים שהועברו לכרטיסייה, מאתחלת את רכיבי הטקסט, ומגדירה מאזין מגע מיוחד עבור כרטיסיית המפה הנועל את אפשרות גרירת הדיאלוג כאשר המשתמש מזיז את המפה או מבצע זום.
- **onMapReady** - נקראת כאשר תשתית המפה מוכנה לעבודה, מפעילה את פקדי הזום והמחוות של המפה, ומפעילה את תהליך השרטוט.
- **drawRouteAndZoom** - מפענחת את מחרוזת הפוליקלין לרשימה של קואורדינטות, משרטטת קו מסלול כחול וגיאודזי על המפה, מוסיפה סמני התחלה וסיום, מחשבת את גבולות המסלול הגיאוגרפיים ומבצעת הנפשה מותאמת של המצלמה כדי שהמסלול כולו ייפרס בצורה מושלמת על המסך.

- **startRunRecording** - מייצרת ומפעילה מעבר למסך הקלטת הריצה הפעיל וסוגרת את כרטיסיית התצוגה המקדימה הנוכחית.

המחלקה RunFragment

C RunFragment
□ startRunBtn : Button
● RunFragment() ● onCreate(savedInstanceState : Bundle) : void ● onCreateView(inflater : LayoutInflater, container : ViewGroup, savedInstanceState : Bundle) : View

- **תפקיד המחלקה:** קטע תצוגה (Fragment) בסיסי המשמש כאחד מטאבי הניווט הראשיים של מסך הבית, ותפקידו המרכזי הוא להציג את שער הכניסה הויזואלי לתחילת פעילות וריצה עבור המשתמש.
- **תכונות משמעותיות:**
 - **startRunBtn** - כפתור בולט במרכז המסך המאפשר למשתמש להתחיל את תהליך הריצה.
- **פעולות משמעותיות:**
 - **onCreateView** - מייצרת ומנפחת את פריסת ה-XML של הטאב, מאתרת את כפתור תחילת הריצה ומגדירה לו מאזין לחיצה פשוט המעביר את המשתמש ישירות אל מסך הגדרת המסלול והאילוצים הכלליים.

המחלקה OptimizedRoute

C OptimizedRoute
□ totalDistance : double □ totalReward : double □ path : List<Waypoint> □ polyline : String
● OptimizedRoute() ● getTotalDistance() : double ● getTotalReward() : double ● getPath() : List<Waypoint> ● getPolyline() : String

- **תפקיד המחלקה:** מייצגת באפליקציה את אובייקט המסלול השלם והמשופר שהתקבל מהשרת, המשמש להצגת הנתונים המסכמים והתוויית הניווט הויזואלית על גבי המפה עבור הרץ.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RouteRequestObject

C RouteRequestObject
- latitude : double - longitude : double - maxLength : double - isCircular : boolean - selectedCategories : List<String>
- RouteRequestObject(latitude : double, longitude : double, maxLength : double, isCircular : boolean, selectedCategories : ArrayList<String>) - getSelectedCategories() : List<String> - isCircular() : boolean - getLatitude() : double - getLongitude() : double - getMaxLength() : double

- **תפקיד המחלקה:** משמשת לבניית אובייקט הבקשה באפליקציה המאגד את כל הפרמטרים והאילוצים, כמו נקודת מיקום, מרחק מבוקש ואופי המסלול, שנשלחים אל השרת לצורך חישוב האופטימיזציה.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RunDataPoint

C RunDataPoint
- latitude : double - longitude : double - altitude : double - speed : float - timestamp : long - accuracy : float - stepDelta : int - isFirstPoint : boolean
- RunDataPoint() - RunDataPoint(latitude : double, longitude : double, altitude : double, speed : float, timestamp : long, accuracy : float, stepDelta : int, isFirstPoint : boolean) - getLatitude() : double - getLongitude() : double - getAltitude() : double - getSpeed() : float - getTimestamp() : long - getAccuracy() : float - getStepDelta() : int - isFirstPoint() : boolean

- **תפקיד המחלקה:** מייצגת דגימת טלמטריה בודדת שנמדדת בזמן אמת על ידי חיישני מכשיר האנדרואיד (כמו מיקום, גובה וצעים) ומיועדת לשילוח ושרשור מול השרת לאורך הפעילות.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RunDataSummary

C RunDataSummary
totalDistanceKm : double durationMillis : long averagePace : double totalElevationGain : double totalSteps : int
RunDataSummary() getTotalDistanceKm() : double getDurationMillis() : long getAveragePace() : double getTotalElevationGain() : double getTotalSteps() : int

- **תפקיד המחלקה:** משמשת לקבלת ולהצגת הסיכום הסטטיסטי הסופי של הריצה (מרחק כולל, קצב, צעדים וזמן נטו) בממשק המשתמש לאחר שהפעילות ננעלה וחושבה בשרת.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RunPointsUpdateRequestObject

C RunPointsUpdateRequestObject
runId : String points : List<RunDataPoint>
RunPointsUpdateRequestObject() RunPointsUpdateRequestObject(runId : String, points : List<RunDataPoint>) getRunId() : String setRunId(runId : String) : void getPoints() : List<RunDataPoint> setPoints(points : List<RunDataPoint>) : void

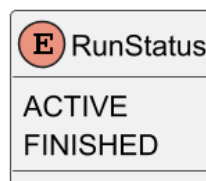
- **תפקיד המחלקה:** אובייקט העברת נתונים (DTO) המשמש את האפליקציה לאריזה ומשלוח של מקבצי נקודות מיקום חדשות אל השרת במהלך הריצה לצורך עדכון הסשן הפעיל.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RunSession

C RunDataPoint
latitude : double longitude : double altitude : double speed : float timestamp : long accuracy : float stepDelta : int isFirstPoint : boolean
RunDataPoint() RunDataPoint(latitude : double, longitude : double, altitude : double, speed : float, timestamp : long, accuracy : float, stepDelta : int, isFirstPoint : boolean) getLatitude() : double getLongitude() : double getAltitude() : double getSpeed() : float getTimestamp() : long getAccuracy() : float getStepDelta() : int isFirstPoint() : boolean

- **תפקיד המחלקה:** ישות המידע המרכזית באפליקציה המרכזת את כל הנתונים של סשן ריצה שלם, כולל מצב הריצה, רשימת הנקודות שנמדדו ואובייקט הסיכום הסופי.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

ה-RunStatus enum



- **תפקיד המחלקה:** קובץ אנום (Enum) המגדיר את מצבי מחזור החיים של הריצה באפליקציה (פעילה או הסתיימה) כדי להבטיח תיאום מלא ומניעת שגיאות לוגיות מול השרת.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה User

C User
email : String username : String password : String weightKg : double heightCm : double birthDate : long createdAt : long
User() User(username : String, email : String, password : String, weightKg : double, heightCm : double, birthDate : long) getUsername() : String setUsername(username : String) : void getEmail() : String setEmail(email : String) : void getPassword() : String setPassword(password : String) : void getWeightKg() : double setWeightKg(weightKg : double) : void getHeightCm() : double setHeightCm(heightCm : double) : void getBirthDate() : long setBirthDate(birthDate : long) : void getCreatedAt() : long setCreatedAt(createdAt : long) : void

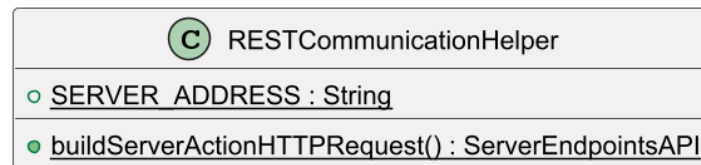
- **תפקיד המחלקה:** מייצגת את פרטי פרופיל המשתמש, נתוני הגישה שלו ומדדיו הפיזיים באפליקציה לצורך תהליכי רישום, התחברות והתאמת חישובים אישיים.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה Waypoint

C Waypoint
lat : double lng : double placeId : String reward : double distanceToNext : double
Waypoint() Waypoint(lat : double, lng : double, placeId : String, reward : double, distanceToNext : double) getLat() : double getLng() : double getPlaceId() : String getReward() : double getDistanceToNext() : double

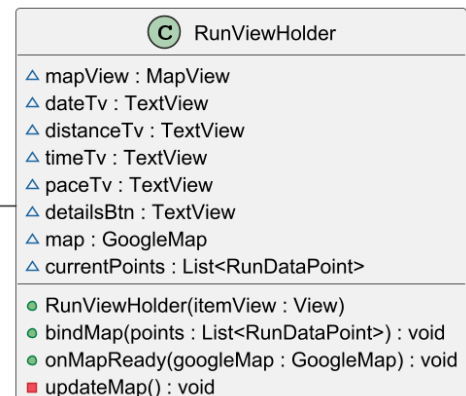
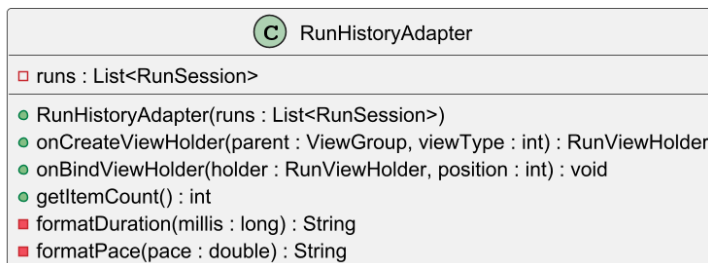
- **תפקיד המחלקה:** מייצגת נקודת תחנה רשמית במסלול הריצה שחושב, ומחזיקה את נתוני המיקום והמרחק הנדרש למעבר אל התחנה הבאה לצורך הניווט הויזואלי באפליקציה.
- אובייקט זה זהה לחלוטין ומקביל למודל הנתונים הקיים בצד השרת, ולכן לא נפרט כאן מחדש את תכונותיו ופעולותיו.

המחלקה RESTCommunicationHelper



- **תפקיד המחלקה:** מחלקת עזר האחראית על בנייה והגדרה של תשתית התקשורת של האפליקציה מול שרת ה-Spring Boot, ומגדירה זמני קציבת זמן ארוכים המותאמים לעיבוד נתונים אלגוריתמי כבד.
- **תכונות משמעותיות:**
 - **SERVER_ADDRESS** - מחרוזת קבועה המגדירה את כתובת השרת המרכזית שאליה האפליקציה תפנה לצורך ביצוע כל קריאות הרשת וה-API.
- **פעולות משמעותיות:**
 - **buildServerActionHttpRequest** - מייצרת ומגדירה לקוח רשת עם קציבת זמן לחיבור ולקריאה, בונה את אובייקט הרטרופיט המרכזי עם ממירי טקסט ו-JSON, ומחזירה מימוש דינמי של ממשק נקודות הקצה של השרת.

המחלקה RunHistoryAdapter



- **תפקיד המחלקה:** מתאם ייעודי עבור רכיב הרשימה האחראי על הצגה ויזואלית של היסטוריית הריצות של המשתמש, כולל שרטוט מפת מסלול ממוזערת לכל ריצה וניהול המעבר למסך הסטטיסטיקות המורחב.
- **תכונות משמעותיות:**
 - **runs** - רשימה של אובייקטי סשן ריצה המכילים את נתוני הריצות הקודמות של המשתמש שנשלפו ממסד הנתונים של השרת.

- **פעולות משמעותיות:**

- **onBindViewHolder** - מקשרת את נתוני הריצה הנוכחית באינדקס הרלוונטי לרכיבי התצוגה היוזואליים, מפעילה את תהליך שרטוט המפה, ומגדירה מאזין לחיצה המאחסן את הריצה שנבחרה ומנווט למסך הסטטיסטיקות המלא.
- **updateMap** - פונקציית עזר פנימית בתוך מחלקת העזר של התצוגה המנקה את המפה הממוזערת, מייצרת קו פוליקלין אדום מתוך רשימת נקודות המיקום הגיאוגרפיות של הריצה, ומשרטטת אותו תוך התאמת מצלמת המפה לגבולות המסלול המדויקים.

המחלקה RunTrackingService

RunTrackingService	
○ ACTION_RUN_UPDATE : String	
□ CHANNEL_ID : String	
□ NOTIFICATION_ID : int	
□ BATCH_INTERVAL_MS : int	
○ ACTION_PAUSE : String	
○ ACTION_RESUME : String	
○ ACTION_FINISH_RUN : String	
□ fusedLocationClient : FusedLocationProviderClient	
□ locationCallback : LocationCallback	
□ totalDistanceInMeters : float	
□ lastLocationForDistance : Location	
□ timerHandler : Handler	
□ startTime : long	
□ isRunning : boolean	
□ currentTimerString : String	
□ currentRunId : String	
□ pointBatch : List<RunDataPoint>	
□ networkHandler : Handler	
□ sensorManager : SensorManager	
□ stepSensor : Sensor	
□ currentStepDelta : int	
□ totalPauseTime : long	
□ pauseStartTime : long	
□ isPaused : boolean	
□ pointsAfterResumeCount : int	
△ requestsClient : ServerEndpointsAPI	
○ onCreate() : void	
■ registerStepSensor() : void	
○ onStartCommand(intent : Intent, flags : int, startId : int) : int	
■ pauseTracking() : void	
■ resumeTracking() : void	
■ trackLocationUpdates() : void	
■ processNewLocation(location : Location) : void	
■ startNetworkBatching() : void	
■ sendBatchToServer() : void	
■ startTimer() : void	
■ createNotification() : Notification	
■ createNotificationChannel() : void	
○ onDestroy() : void	
■ finishRunAndStopService() : void	
○ onBind(intent : Intent) : IBinder	
○ onAccuracyChanged(sensor : Sensor, i : int) : void	
○ onSensorChanged(sensorEvent : SensorEvent) : void	

- **תפקיד המחלקה:** שירות רקע פעיל במצב Foreground המהווה את הלב המבצעי של מעקב הריצה באפליקציה. השירות אחראי על איסוף רציף של מיקומי GPS, ספירת צעדים מחיישני המכשיר, ניהול שעון הריצה, והזרמת הנתונים במקבצים אל השרת ובשידור חי אל מסך המשתמש.
- **תכונות משמעותיות:**

- **fusedLocationClient** - רכיב הגישה המרכזי של גוגל לשליפת עדכוני מיקום מדויקים וחיישני ה-GPS של המכשיר.
- **pointBatch** - רשימה זמנית המשמשת כמחסנית אגירה לנקודות הטלמטריה החדשות שנאספו, לפני שילוחן המרוכז והקבוצתי לשרת.
- **timerHandler** ו-**networkHandler** - מנגנוני תזמון האחראים על הרצה מחזורית של שעון הריצה הויזואלי ועל שליחת מקבצי הנקודות לשרת במרווחי זמן קבועים של 15 שניות.
- **currentStepDelta** - מונה זמני הסופר את כמות הצעדים שבוצעו בפועל מאז דגימת המיקום הקודמת על בסיס חיישן זיהוי הצעדים הפיזי של המכשיר.
- **totalPauseTime** ו-**pauseStartTime** - משתני תזמון המשמשים לחישוב מדויק של זמן ההפסקות וההשהיות המצטבר בריצה, לצורך ניכוי מזמן הנטו הפעיל של המשתמש.

• פעולות משמעותיות:

- **onStartCommand** - מנהלת את פקודות השליטה המשודרות לשירות (התחלה, השהיה, המשך) מתוך מסכי האפליקציה, ומפעילה את רכיבי הפורגראונד, ההתראות ואיסוף המידע.
- **pauseTracking** ו-**resumeTracking** - מנגנוני ניהול מצבי ההשהיה העוצרים לחלוטין את קבלת עדכוני ה-GPS וחיישן הצעדים כדי לחסוך בסוללה ולמנוע זליגת נתונים שגויים בזמן מנוחה, ומחזירים אותם לפעילות תקינה תוך חישוב פער הזמנים המדויק.
- **processNewLocation** - מפעילה מסנן דיוק על המיקומים הנכנסים, מנהלת תקופת חימום של שתי נקודות לאחר חזרה מהשהיה כדי למנוע קפיצות GPS לא הגיוניות, מחשבת את המרחק הפיזי שנצבר באנדרואיד, מייצרת אובייקט נקודה ומזריקה אותו למחסנית, ומשדרת את המידע המעודכן בבראדקסט אל מסך ההקלטה.
- **sendBatchToServer** - משכפלת ומרוקנת אסינכרונית את מחסנית הנקודות הזמנית, אורזת אותן באובייקט עדכון ייעודי ומזרזת את שליחתן לשרת דרך קריאת API מתאימה.
- **onDestroy** - מתרחשת בעת עצירה סופית של הריצה. הפעולה מחשבת את זמן ההשהיה הסופי, מרוקנת את מקבץ הנקודות האחרון, מבצעת קריאת רשת נועלת לשרת לצורך הפקת סטטיסטיקות סופיות, ומבטלת את רישום כל הרכיבים, החיישנים והתזמונים בתוכנה.

המחלקה SelectedRunHolder

C SelectedRunHolder
selectedSession : RunSession
setData(session : RunSession) : void
getData() : RunSession
clear() : void

- **תפקיד המחלקה:** מחלקת עזר המממשת מנגנון אחסון סטטי בזיכרון, המשמש כגשר להעברת אובייקט הריצה השלם בין מסך היסטוריית הריצות למסך הסטטיסטיקות המפורט, ובכך מונעת את הצורך בהעברת אובייקטים כבדים ומנופחים דרך ה-Intent הסטנדרטי של אנדרואיד.
- **תכונות משמעותיות:**
 - **selectedSession** - משתנה סטטי המחזיק את אובייקט ששן הריצה שנבחר על ידי המשתמש לצורך הצגת הסטטיסטיקות שלו.
- **פעולות משמעותיות:**
 - **setData** - נועלת ומאחסנת את אובייקט הריצה שנבחר בזיכרון המשותף של האפליקציה.
 - **getData** - שולפת את אובייקט הריצה השמור עבור מסך היעד.
 - **clear** - מאפסת את המשתנה הסטטי ומרוקנת את הזיכרון עם סגירת המסך כדי למנוע זליגות זיכרון במכשיר.

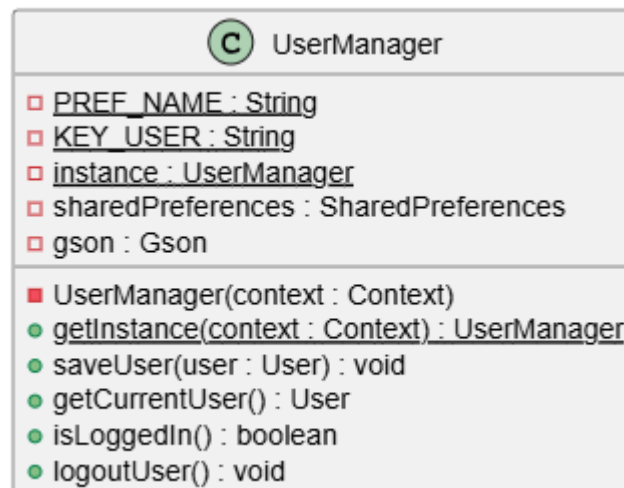
הממשק ServerEndpointsAPI

I ServerEndpointsAPI
• checkUserExists(email : String) : Call<Void> • login(user : User) : Call<User> • signup(user : User) : Call<String> • generateRoute(requestObject : RouteRequestObject) : Call<OptimizedRoute> • startRun(userId : String) : Call<String> • finishRun(runId : String, pauseDurationMillis : long) : Call<RunSession> • sendDataPoints(requestObject : RunPointsUpdateRequestObject) : Call<Void> • getUserHistory(userId : String) : Call<List<RunSession>>

- **תפקיד המחלקה:** ממשק רטרופיט המגדיר את החוזה הדיגיטלי וכל נקודות הקצה של ה-REST API מול שרת ה-Spring Boot, ומאפשר לאפליקציה לבצע קריאות רשת מובנות לניהול המשתמשים, חישוב המסלולים וסנכרון הריצות.
- **תכונות משמעותיות:**
 - אין לממשק זה תכונות או משתני מחלקה פנימיים משלו, שכן הוא משמש כהגדרה הצהרתית בלבד עבור מבנה הקריאות בפרויקט.
- **פעולות משמעותיות:**

- **login ו-signup** - מגדירות בקשות POST לאימות נתוני התחברות ורישום של משתמשים חדשים במערכת.
- **generateRoute** - מגדירה בקשת POST לשליחת אילוצי המרחק והמיקום הגיאוגרפי וקבלת מסלול אופטימלי מחושב.
- **startRun ו-finishRun** - מגדירות בקשות POST לפתיחת סשן ריצה חדש בשרת ולנעילתו בסיום הפעילות תוך העברת נתוני זמני ההפסקות לצורך חישוב זמן נטו.
- **sendDataPoints** - מגדירה בקשת POST להזרמה תקופתית של מקבצי נקודות הטלמטריה לאורך הריצה הפעילה.
- **getUserHistory** - מגדירה בקשת POST לשליפת כל הריצות הקודמות של המשתמש בצורה ממוינת.

המחלקה UserManager



- **תפקיד המחלקה:** רכיב ניהול (Singleton) האחראי על ניהול סשן המשתמש ברמת המכשיר המקומי, המאפשר שמירה, שליפה ומחיקה של נתוני המשתמש המחובר באופן קבוע באמצעות תשתית SharedPreferences וסריאליזציית JSON.
- **תכונות משמעותיות:**
 - **PREF_NAME, KEY_USER** – קבועים המגדירים את שם קובץ ה-XML המקומי של ה-SharedPreferences ואת מפתח השמירה הייחודי שבו מאוחסן הטקסט של המשתמש.
 - **instance** – משתנה סטטי המחזיק את המופע היחיד של המחלקה בהתאם לתבנית העיצוב Singleton.
 - **sharedPreferences** – אובייקט ייעודי של מערכת ההפעלה אנדרואיד המשמש לקריאה וכתובה של נתונים זעירים בדיסק המקומי במצב פרטי.
 - **gson** – רכיב של ספריית Google Gson המשמש להמרת אובייקט המשתמש למחרוזת טקסט בפורמט JSON ולהיפך.

• פעולות משמעותיות:

- **getInstance** – מתודה סטטית ומסונכרנת (Thread-safe) המבטיחה יצירה וגישה למופע יחיד של מנהל המשתמשים מכל מקום ברחבי האפליקציה.
- **saveUser** – מקבלת אובייקט User, ממירה אותו למחרוזת JSON ושומרת אותו באופן קבוע בתוך קובץ ה-SharedPreferences.
- **getCurrentUser** – שולפת את מחרוזת ה-JSON השמורה במכשיר ומפענחת אותה בחזרה לאובייקט User מלא, או מחזירה null אם אף משתמש אינו מחובר.
- **isLoggedIn** – פונקציית עזר מהירה המחזירה ערך בוליאני המעיד האם קיים משתמש מחובר כעת במערכת.
- **logoutUser** – מבצעת ניקוי והתנתקות של סשן המשתמש על ידי מחיקת המפתח הרלוונטי מקובץ ה-SharedPreferences.

המחלקה UIHelper

UIHelper
● <code>buildAlertDialog(context : Context, title : String, message : String, isCancelable : boolean) : AlertDialog.Builder</code>

- **תפקיד המחלקה:** מחלקת עזר קטנה המרכזת את הלוגיקה של יצירת אלמנטים חזותיים אחידים במערכת, ומיועדת לפשט את הקמת תיבות הדיאלוג וההתרעות באפליקציה.
- **תכונות משמעותיות:**
 - אין למחלקה זו תכונות או משתנים פנימיים משלה.
- **פעולות משמעותיות:**
 - **buildAlertDialog** - מקבלת קונטקסט, כותרת, הודעה ודגל קביעה האם הדיאלוג ניתן לביטול, בונה אובייקט בנאי מותאם של AlertDialog ומחזירה אותו להמשך עיצוב או הצגה ישירה במסך.

12.4 מבני נתונים בהם נעשה שימוש

במסגרת פיתוח צד השרת של המערכת, נעשה שימוש במבני נתונים דינמיים מתוך תשתית האוספים של ג'אווה (Java Collections Framework). מבני נתונים אלו נבחרו בהתאם לצורכי האחסון,

הגישה והיעילות האלגוריתמית הנדרשים לניהול המשתמשים, ניטור הריצות והפעלת מנוע האופטימיזציה.

רשימה רציפה ודינמית (List / ArrayList)

מבנה נתונים ליניארי דינמי המאפשר שמירה של איברים ברצף מוגדר, גישה אליהם לפי אינדקס, ושינוי גמיש של גודל האוסף בזמן ריצה (הוספה ומחיקה של איברים).

איפה נעשה בו שימוש? במחלקת UserService לצורך ריכוז כלל רשומות המשתמשים.

במחלקת RunStatisticsService לצורך ניהול נקודות הציון הגיאוגרפיות של הריצה ולצורך שליפת היסטוריית האימונים של הרץ.

במחלקות RouteService ו-DPSOAlgorithmService לצורך ניהול רשימות צמתי המועמדים, נקודות הציון במסלול הסופי (Waypoints), וריכוז ישויות החלקיקים המרכיבים את נחיל האופטימיזציה.

לאיזה צורך הוא משמש? מבנה זה משמש בכל מקום שבו נדרש לשמור על סדר חשיבות או כרונולוגיה של נתונים. במערכת המעקב, הוא חיוני לשמירת נתיב הריצה כרצף עוקב ורציף של נקודות זמן ומיקום. במנוע האופטימיזציה, הוא משמש לייצוג פרמוטציות (סדרים שונים של מסלולים) שבהן סדר האיברים ברשימה קובע את תוואי הדרך הפיזי של הרץ.

טבלת גיבוב ומיפוי (Map / HashMap)

מה סוגו? מבנה נתונים מבוסס מפתח-ערך (Key-Value), המאפשר מיפוי ייחודי ושליפה מהירה מאוד (בזמן קבוע בממוצע) של ערך באמצעות המפתח המשוך אליו.

איפה נעשה בו שימוש? במחלקת RouteService לצורך הקמת תשתית הגרף הגיאוגרפי (הן עבור מפת השכנים הקרובים והן עבור מטריצת המרחקים הממשית).

לאיזה צורך הוא משמש? מבנה זה משמש לייצוג מבני נתונים מורכבים ורשתות קשרים (גרפים):

מיפוי מסוג Map<Candidate, List<Candidate>> משמש לייצוג גרף שכנים קרובים (KNN Adjacency List), שבו כל צומת גיאוגרפי (המפתח) ממופה אל רשימת הצמתים הקרובים אליו פיזית (הערך).

מיפוי מורכב ודו-ממדי מסוג Map<Candidate, Double>Map משמש כרשת העלויות ומטריצת המרחקים הדינמית של המערכת. הוא מאפשר למנוע האופטימיזציה לשלוף באופן מיידי את מרחק ההליכה הממשי בין כל שני צמתים בגרף, ובכך לבחון במהירות את חוקיות תקציב המרחק של המסלול המוצע.

12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

בסעיף זה מוצגות הפעולות והאלגוריתמים המרכזיים המרכיבים את ליבת המערכת בצד השרת. עבור כל רכיב מוצג קוד המקור הרלוונטי, לצד הגדרת הפרמטרים, תיאור פונקציונלי מורחב של זרימת התוכנית וניתוח הנדסי של סיבוכיות זמן הריצה.

1. אלגוריתם הליבה המטא-היוריסטי – הפעלת נחיל החלקיקים (solve)

• קוד המקור:

```
public Particle solve() {
    initializeSwarm();

    for (int i = 0; i < iterations; i++) {
        for (Particle particle : swarm) {
            List<Candidate> newPos = localSearchOptimize(particle);
            newPos = projectToFeasible(newPos);
            newPos = twoOptOperation(newPos);
            newPos = projectToFeasible(newPos);

            double fitness = evaluateFitness(newPos);
            particle.setPosition(newPos);
            particle.updatePersonalBest(fitness);
            updateGlobalBest(particle);
        }
    }
    return new Particle(globalBest, evaluateFitness(globalBest));
}
```

• תיעוד טענת כניסה ויציאה:

- **פרמטרים (טענת כניסה):** הפונקציה אינה מקבלת פרמטרים ישירים בממשק שלה. היא ניזונה משדות המחלקה הפנימית שבה היא מתקיימת, המכילים את כמות איטרציות היעד, גודל נחיל החלקיקים, רשת הגרף הטופוגרפית ותקציב המרחק המקסימלי.
- **ערך מוחזר (טענת יציאה):** אובייקט מסוג Particle המייצג את הישות האיכותית ביותר שהתגלתה בתום הריצה. אובייקט זה מחזיק את רשימת הצמתים הגאוגרפיים המרכיבים את המסלול הסופי המיטבי ואת ציון הכושר הכולל שלו.

- **מה מבצע?** הפונקציה מנהלת את לולאת הריצה המרכזית של אלגוריתם ה-DPSO (Discrete Particle Swarm Optimization) בצד השרת, ומנווטת את נחיל החלקיקים להתכנסות מהירה לעבר מסלול ריצה חוקי ומקסימל-תגמול בזמן אמת.

• אופן פעולתו בצורה מפורטת:

- **אתחול המערך:** ראשית, נקראת פעולת העזר `initializeSwarm` המייצרת ומפזרת במרחב הדיסקרטי פתרונות מסלול ראשוניים, מגוונים וחוקיים עבור כל אחד מהחלקיקים בנחיל.
- **לולאת החיפוש המרכזית:** התוכנית מבצעת סבב ריצה קבוע מראש המוגדר על ידי כמות האיטרציות הנדרשת.
- **עדכון המיקומים הדיסקרטיים:** בתוך הלולאה, עבור כל חלקיק בנחיל, מופעל אלגוריתם חיפוש מקומי המעדכן את רצף הצמתים שלו על בסיס שקלול הסתברותי של משקל האינרציה, המיקום הטוב ביותר שלו בעבר, והמיקום המוביל של הנחיל כולו.
- **אכיפת אילוצים ושיפור מבני:** מיד לאחר תנועת החלקיק, המסלול החדש מוטל למרחב הפתרונות החוקיים כדי לוודא שאינו חורג מתקציב המרחק. בהמשך, מופעל אלגוריתם שיפור מקומי למניעת הצטלבויות, והמסלול מועבר סבב תיקון ואכיפה נוסף.
- **עדכון שיאי כושר (Fitness):** המערכת מעריכה את ציון התגמול המשוקלל של המיקום החדש, מעדכנת את שיאו האישי של החלקיק, ומבצעת בדיקה והחלפה של הפתרון המוביל הכללי במידה ונמצא מסלול איכותי יותר.
- **החזרת תוצאה:** עם סיום כל האיטרציות, מיוצר אובייקט פתרון סופי המבוסס על רצף הצמתים המוביל של הנחיל כולו.
- **סיבוכיות זמן ריצה:** נסמן ב- I את מספר האיטרציות, ב- S את גודל נחיל החלקיקים, וב- N את כמות הצמתים בגרף המועמדים הגאוגרפי.
 - בכל סבב פנימי עבור חלקיק בודד, הפעולה הדומיננטית ביותר היא הרצת אלגוריתם ה- $opt-2$ שעלותה החישובית עומדת על סדר גודל של $O(N^2)$.
 - לפיכך, סיבוכיות זמן הריצה הכוללת של פעולת הליבה היא $O(I * S * N^2)$ מאחר וכמויות האיטרציות וגודל הנחיל קבועים או תלויים ליניארית בפרמטרים המרחביים, השרת משיג זמני ביצוע מהירים ביותר בזמן אמת.

2. אלגוריתם אופטימיזציה ושיפור מקומי גאומטרי (`twoOptOperation`)

• הקוד:

```
private List<Candidate> twoOptOperation(List<Candidate> route) {
    if (route.size() < 4)
        return new ArrayList<>(route);

    List<Candidate> improved = new ArrayList<>(route);
    boolean improvement = true;

    while (improvement) {
        improvement = false;
        for (int i = 0; i < improved.size() - 3; i++) {
            for (int j = i + 2; j < improved.size() - 1; j++) {
```

```

double before = graph.getCost(improved.get(i), improved.get(i + 1))
+
graph.getCost(improved.get(j), improved.get(j +
1));

double after = graph.getCost(improved.get(i), improved.get(j)) +
graph.getCost(improved.get(i + 1), improved.get(j +
1));

if (after < before) {
    improved = reverseSegmentCopy(improved, i + 1, j);
    improvement = true;
}
}
}
}
return improved;
}

```

• תיעוד טענת כניסה ויציאה:

- **פרמטרים (טענת כניסה):** `route <Candidate>List` – רשימה דינמית של אובייקטי מיקום המייצגים את סדר הצעידה והחיבורים הנוכחי במסלול הריצה המוצע.
- **ערך מוחזר (טענת יציאה):** `<Candidate>List` – רשימה חדשה של אובייקטי מיקום המייצגת מסלול מתוקן, קצר יותר, אשר עבר אופטימיזציה מבנית למניעת כשלים גאומטריים.

- **מה מבצע?** אלגוריתם זה מיועד לפתור את בעיית ההצטלבויות הגאומטריות והלולאות המיותרות במסלול. הוא מנתח את עלויות המעבר הריאליות ומבצע היפוך מקטעים בכל מקום שבו קיצור פיזי חוסך במרחק הכולל, מה שמפנה תקציב מרחק פנוי לטובת הוספת נקודות עניין חדשות.

• אופן פעולתו בצורה מפורטת:

- **בדיקת תנאי סף:** אם אורך המסלול קטן מארבעה צמתים, אין אפשרות מבנית לבצע ניתוק והחלפת קשתות, והמסלול מוחזר כעותק זהה ללא שינוי.
- **לולאת התכנסות אקטיבית:** התוכנית מגדירה משתנה בוליאני ומריצה לולאת `while` הממשיכה לפעול כל עוד מתגלים שיפורים נוספים במסלול.
- **סריקת זוגות קשתות:** באמצעות שתי לולאות `for` מקוננות, האלגוריתם סורק את כל זוגות הקשתות שאינן עוקבות או סמוכות במסלול (קשת בין אינדקס `i` ל-`i+1`, וקשת מרוחקת בין `j` ל-`j+1`).
- **הערכת עלות אלטרנטיבית:** המערכת פונה לגרף העלויות ומחשבת את סכום מרחקי ההליכה הנוכחי של שתי הקשתות (`before`), מול סכום המרחקים שיעלה אם ננתק אותן

ונחבר את המסלול בצורה אלטרנטיבית – חיבור צומת i לצומת j , וחיבור צומת $i+1$ לצומת $j+1$ (after).

- **היפוך מקטע המסלול:** אם החיבור החדש קצר וחסכוני יותר במרחק, התוכנית הופכת את סדר האיברים הפנימי במקטע שבין $i+1$ ל- j באמצעות פונקציית עזר, ומסמנת כי בוצע שיפור המצדיק סבב סריקה מקיף נוסף.

• סיבוכיות זמן ריצה:

- עבור מסלול המכיל N נקודות ציון, שתי הלולאות המקוננות סורקות את כל השילובים האפשריים של זוגות קשתות, פעולה המייצרת סיבוכיות של $O(N^2)$ בכל מעבר מלא.
- פעולת היפוך והעתקת המקטע אורכת זמן ליניארי של $O(N)$.
- במקרה הגרוע, כמות סבבי השיפור החוזרים חסומה תיאורטית, ובפועל האלגוריתם מתכנס במהירות רבה לאחר מספר קטן של סבבים קבועים. לפיכך, סיבוכיות זמן הריצה המעשית והמקובלת של האלגוריתם היא $O(N^2)$.

3. אלגוריתם תיקון והטלה למרחב פתרונות חוקיים (projectToFeasible)

• הקוד:

```
public List<Candidate> projectToFeasible(List<Candidate> toImprove) {
    if (toImprove == null || toImprove.isEmpty())
        return toImprove;

    List<Candidate> result = new ArrayList<>();
    result.add(toImprove.get(0));
    double cost = 0.0;

    for (int i = 1; i < toImprove.size(); i++) {
        Candidate currentValid = result.get(result.size() - 1);
        Candidate nextAttempt = toImprove.get(i);
        double edgeCost = graph.getCost(currentValid, nextAttempt);

        if (edgeCost == Double.POSITIVE_INFINITY)
            continue;

        if (isCircular) {
            double returnCost = graph.getCost(nextAttempt, startNode);
            if (cost + edgeCost + returnCost > T_MAX) {
                break;
            }
        } else {
            if (cost + edgeCost > T_MAX) {
                break;
            }
        }
    }
}
```

```

    }
}

result.add(nextAttempt);
cost += edgeCost;
}

if (isCircular && result.size() > 1) {
    Candidate lastNode = result.get(result.size() - 1);
    if (!lastNode.getId().equals(startNode.getId())) {
        result.add(startNode);
    }
}

return result;
}

```

- **תיעוד טענת כניסה ויציאה:**

- **פרמטרים (טענת כניסה):** `tolImprove <Candidate>List` – רשימת מסלול מוצעת שנוצרה על ידי מנגנוני השינוי של הנחיל, אשר עלולה להכיל מרחקים ארוכים מדי או קשרים חורגים מהתקציב.
- **ערך מוחזר (טענת יציאה):** `<Candidate>List` – רשימת מסלול מתוקנת, המובטחת כחוקית לחלוטין, שאינה חורגת בשום תרחיש מתקציב המרחק שהוגדר וסגורה כמעגל במידת הצורך.

- **מה מבצע? פונקציה זו משמשת כמנגנון אכיפת האילוצים (Constraint Enforcement) המרכזי של האלגוריתם.** היא בוחנת את המסלולים סדרתית, קוטעת אותם ברגע שהם מנצלים את מגבלת המרחק המקסימלית, ומבטיחה חזרה בטוחה וחוקית הביתה.

- **אופן פעולתו בצורה מפורטת:**

- **אתחול ובנייה סדרתית:** התוכנית מייצרת רשימה ריקה פנימית עבור המסלול החוקי, מציבה בה את נקודת המוצא, ומנהלת משתנה צובר למדידת המרחק המצטבר (`cost`).
- **סריקה ובדיקת עלויות:** האלגוריתם עובר על פני נקודות המסלול המקורי אחת אחרי השנייה, ושולף מתוך מטריצת המרחקים של הגרף את עלות הצעידה הריאלית מהנקודה התקפה האחרונה אל הנקודה הבאה בתור.
- **אכיפת תקציב לפי סוג מסלול:**

- **במסלול קווי (Point-to-Point):** המערכת בודקת האם הוספת הצומת החדש תגרום לסך המרחק המצטבר לעבור את מגבלת המרחק המותרת (`T_MAX`).
- **במסלול מעגלי (Circular):** המערכת מפעילה כלל מחמיר ומתוחכם יותר: היא מחשבת האם עלות המרחק שנצברה, בתוספת עלות הקשת אל הצומת החדש,

בתוספת עלות החזרה המיידית והישירה מאותו צומת אל נקודת הזינוק המקורית (startNode), תחרוג מהתקציב הכללי T_MAX.

- **קטיעת המסלול:** אם החישוב מראה חריגה מתקציב המרחק, הלולאה נקטעת מיד (break), וכל שאר הנקודות העודפות במסלול נזרקות. אם יש מספיק מרחק פנוי, הצומת מתווסף לרשימה התקפה והעלות מתעדכנת.
- **סגירה לוגית של המעגל:** בתום הלולאה, במידה והמסלול הוגדר כמעגלי, המערכת מוודא באופן אקטיבי כי נקודת הסיום של הרשימה תהיה זהה לנקודת ההתחלה המקורית כדי להבטיח לולאה שלמה.

• סיבוכיות זמן ריצה:

- האלגוריתם מבצע מעבר סדרתי יחיד (Single Pass) לאורך רשימת הנקודות המוצעת.
- שליפת עלות המרחק הריאלית מתוך מבנה הנתונים הדו-ממדי של הגרף מתבצעת בזמן קבוע של $O(1)$.
- על כן, סיבוכיות זמן הריצה היא ליניארית לחלוטין ותלויה ישירות באורך המסלול, כלומר $O(N)$, כאשר N הוא מספר הצמתים ברשימה הנבדקת.

4. אלגוריתם עיבוד גאודזי וסיכום נתוני אימון הריצה (finalizeRunStatistics)

• הקוד: 000000000

```
public RunSession finalizeRunStatistics(String runId, long pauseDurationMillis) {
    RunSession session = runRepository.findById(runId)
        .orElseThrow(() -> new RuntimeException("Run session not found"));

    List<RunDataPoint> points = session.getDataPoints();

    RunDataSummary summary = new RunDataSummary();

    if (points == null || points.isEmpty()) {
        session.setSummary(summary);
    } else {
        double totalDistanceMeters = 0;
        double totalElevationGain = 0;
        int totalSteps = 0;

        for (int i = 1; i < points.size(); i++) {
            RunDataPoint prev = points.get(i - 1);
```

```
RunDataPoint curr = points.get(i);

if (curr.isFirstPoint())
    continue;
if (curr.getAccuracy() < 20.0) {
    totalDistanceMeters += calculateDistance(
        prev.getLatitude(), prev.getLongitude(),
        curr.getLatitude(), curr.getLongitude());
}

double elevationDiff = curr.getAltitude() - prev.getAltitude();
if (elevationDiff > 0.5) { // Hysteresis:
    totalElevationGain += elevationDiff;
}

totalSteps += curr.getStepDelta();
}

long endTime = System.currentTimeMillis();

long grossDuration = endTime - session.getStartTime();

long activeDurationMillis = grossDuration - pauseDurationMillis;

double totalDistanceKm = totalDistanceMeters / 1000.0;
double averagePace = 0;
if (totalDistanceKm > 0) {
    double durationMinutes = activeDurationMillis / 60000.0;
    averagePace = durationMinutes / totalDistanceKm;
}

summary.setTotalDistanceKm(totalDistanceKm);
summary.setDurationMillis(activeDurationMillis);
summary.setAveragePace(averagePace);
```

```

summary.setTotalElevationGain(totalElevationGain);
summary.setTotalSteps(totalSteps);
}

session.setSummary(summary);
session.setStatus(RunStatus.FINISHED);
runRepository.save(session);

return session;
}

```

• תיעוד טענת כניסה ויציאה:

- **פרמטרים (טענת כניסה):** String runId – המזהה הייחודי של סשן האימון הפעיל במסד הנתונים; long pauseDurationMillis – משך הזמן המצטבר שבו המתאמן שהה במצב הפסקה (Pause) במהלך הריצה בשטח, כפי שנמדד ודווח מאפליקציית המובייל.

- **ערך מוחזר (טענת יציאה):** אובייקט מוגמר ומעובד מסוג RunDataSummary, המרכז את כלל מדדי הביצוע הסופיים של הריצה (מרחק בקילומטרים, משך זמן נטו, קצב תנועה ממוצע, גובה טופוגרפי מצטבר וסך הצעדים).

- **מה מבצע?** פונקציה זו מתעוררת בצד השרת ברגע שהמשתמש מדווח על סיום הפעילות הפיזית שלו. תפקידה לבצע טיוב, סינון ועיבוד מתמטי של כל נקודות המיקום הגולמיות שנאספו לאורך האימון, להפיק מהן תובנות אנליטיות מדוייקות, ולעדכן את מצב הריצה ברישומיו של מסד הנתונים.

• אופן פעולתו בצורה מפורטת:

- **שליפת רשומת האימון:** המערכת פונה אל מאגר הנתונים ומביאה את ישות סשן הריצה המתאימה, הכוללת את מערך נקודות הציון הגיאוגרפיות שנקלטו ברקע.

- **לולאת עיבוד סדרתית:** התוכנית סורקת את מערך נקודות המיקום מהאינדקס השני ואילך, ומבצעת שלוש פעולות טיוב וסכימה:

- **חישוב מרחק גאודזי ומסונן:** המערכת בודקת את רמת הדיוק של אות הלוויין באותה נקודה. במידה ורמת הדיוק תקינה (מתחת ל-20 מטרים), מופעלת פונקציית עזר המחשבת את המרחק במטרים בין הנקודה הנוכחית לקודמתה על בסיס נוסחת ה-Haversine, הלוקחת בחשבון את רדיוס כדור הארץ והפרשי קווי הרוחב והאורך הטריגונומטריים.

- **סינון רעשי גובה (Hysteresis):** הפרש הגובה בין הדגימות מחושב ומתווסף לצובר העלייה רק אם השינוי חיובי וגדול מחצי מטר (0.5), כדי להתגבר על קפיצות ורעשי חומרה של חיישן הגובה.
- **סכימת צעדים:** כמות הצעדים שבוצעה בין הדגימות מתווספת לצובר הצעדים הכללי.
- **ניתוח זמנים וקצב ממוצע:** המערכת שולפת את זמן הסיום הנוכחי, מחשבת את משך הזמן הכולל שחלף (ברוטו), ומקזזת ממנו את משכי ההפסקות שהגיעו מהאנדרואיד כדי לקבל את זמן הריצה הריאלי (נטו). בהסתמך על זמן הנטו והמרחק הכולל, מחושב הקצב הממוצע בדקות לקילומטר.
- **עדכון שכבת האחסון:** המדדים המחושבים מוצבים באובייקט הסיכום, מצב הריצה מעודכן לסטטוס סגור, והמסמך כולו נשמר מחדש בבסיס הנתונים בענן.
- **סיבוכיות זמן ריצה:**
 - נסמן ב-P את כמות נקודות המיקום הגולמיות (dataPoints) שנאספו לאורך זמן הריצה מהמכשיר הנייד.
 - האלגוריתם מבצע סריקה ליניארית אחת באורך P על פני המערך. כלל החישובים המתמטיים ופונקציות העזר בתוך הלולאה (כולל נוסחת ה-Haversine) מבוצעים בזמן קבוע של $O(1)$.
 - פעולות השליפה והכתיבה מול מסד הנתונים מבוססות על אינדקסים ומזהים ייחודיים שזמן עבודתם מהיר וקבוע. על כן, סיבוכיות זמן הריצה הכוללת של הפעולה היא ליניארית לחלוטין ועומדת על $O(P)$.

13. תיאור מסד הנתונים

בסעיף זה מתוארת שכבת אחסון הנתונים של המערכת, הטכנולוגיה הנבחרת לניהול בסיס הנתונים ומבנה הישויות הלוגיות הלכה למעשה.

ניהול המידע במערכת מתבסס על מסד נתונים לא-רלציוני (NoSQL) מסוג **MongoDB**, המנוהל ופועל כספק שירות מבוסס ענן. הבחירה בארכיטקטורה זו נבעה מתוך מספר שיקולים הנדסיים ומבניים:

- **תפיסת ה-NoSQL וגמישות סכמה (Flexible Schema):** בשונה ממסדי נתונים קלאסיים (SQL) המאלצים מבנה טבלאי קשיח ויחסי גומלין מורכבים, תפיסת ה-NoSQL ב-MongoDB מבוססת על מסמכים (Documents) עצמאיים. מאחר ואימון ריצה כולל רצף דינמי, משתנה ולא צפוי של דגימות מיקום ונתוני חיישנים, מודל זה מאפשר להרחיב ולשנות את מבנה הנתונים בקלות ללא צורך בפעולות תחזוקה מורכבות.

- **אחסון מונחה מסמכים (Document-Oriented Storage):** הנתונים ב-MongoDB נשמרים בפורמט BSON (גרסה בינארית של JSON). פורמט זה תואם באופן מושלם לעבודה מול שפת Java וסביבת Spring Boot, שכן הוא מאפשר לייצג אובייקטים מורכבים הכוללים מערכים מקוננים (Nested Arrays) כישות אחת מלוכדת, ומבטל את הצורך בביצוע פעולות חיבור טבלאות (JOINS) יקרות ומעכבות.
- **בסיס נתונים מנוהל בענן (Cloud Database):** מסד הנתונים מנוהל ומארח על גבי תשתית ענן, דבר המבטיח זמינות גבוהה, יתירות (Redundancy) וגיבוי אוטומטי של נתוני המשתמשים. פריסה זו מאפשרת לאפליקציית המובייל ולשרת ה-Backend לגשת בצורה מאובטחת ומהירה למרחב האחסון מכל מקום, ותומכת ביכולת גידול אופקית (Scalability) חלקה עם קליטת משתמשים חדשים.

מבנה הישויות במערכת

להלן הגדרת המבנה המתוכנן והגנרי עבור אוספי הנתונים (Collections) המרכיבים את המערכת:

1. אוסף משתמשים (Users Collection)

אוסף זה מיועד לשמירה וניהול של פרופילי הרצים במערכת. המפתח הראשי נקבע באופן ייחודי על פי כתובת הדואר האלקטרוני של המשתמש.

מבנה ג'ייסון כללי וגנרי:

```
{
  "_id": "user_email@example.com",
  "username": "string",
  "password": "string_encrypted_or_plain",
  "weightKg": "number (double/int)",
  "heightCm": "number (double/int)",
  "birthDate": {
    "$numberLong": "timestamp_in_milliseconds"
  },
  "createdAt": {
    "$numberLong": "timestamp_in_milliseconds"
  },
  "_class": "shaleva.run_app_proj.datamodels.User"
}
```

- **הסבר השדות:** id_ : מזהה ייחודי של המסמך, המיוצג על ידי אימייל המשתמש (מונע כפל חשבונות).

- username ו-password: פרטי ההזדהות של המשתמש לצורך גישה למערכת.
- heightCm ו-weightKg: מאפיינים פיזיולוגיים המשמשים את שכבת הלוגיקה לעיבוד מדדים.
- birthDate ו-createdAt: חותמות זמן המיוצגות במילישניות לצורך חישובי גיל ותאריך הקמת החשבון.
- class_: שדה פנימי של Spring Data הממפה את המסמך לקלאס המתאים ב-Java.

2. אוסף אימונים וריצות (Runs Collection)

אוסף זה מייצג את "סשן" הריצה. הוא מדגים את העוצמה של NoSQL, שכן כל נקודות הציון הגיאוגרפיות שנאספו לאורך האימון נשמרות כמערך אובייקטים מקונן בתוך מסמך הריצה האב. מבנה ג'ייסון כללי וגנרי:

```
{
  "_id": {
    "$oid": "unique_database_generated_object_id"
  },
  "userId": "user_email@example.com",
  "status": "string_enum (e.g., ACTIVE / FINISHED)",
  "startTime": {
    "$numberLong": "timestamp_in_milliseconds"
  },
  "dataPoints": [
    {
      "latitude": "number (double)",
      "longitude": "number (double)",
      "altitude": "number (double)",
      "speed": "number (float/double)",
      "timestamp": {
        "$numberLong": "timestamp_in_milliseconds"
      },
      "accuracy": "number (float/double)",
      "stepDelta": "number (int)",
      "isFirstPoint": "boolean"
    }
  ],
}
```



```
"_class": "shaleva.run_app_proj.datamodels.RunSession"
}
```

• הסבר השדות:

- id_: מזהה ייחודי דינמי המיוצר אוטומטית על ידי מסד הנתונים (ObjectId).
- userId: מפתח זר המקשר את הריצה למשתמש הספציפי מתוך קולקשן המשתמשים.
- status: מצבו הלוגי של האימון הנוכחי, המאפשר מעקב אחר ריצות פעילות או כאלו שהסתיימו.
- startTime: חותמת זמן המגדירה את רגע אתחול הריצה בשרת.
- dataPoints: מערך דינמי ומקונן של אובייקטי מיקום. כל אובייקט במערך מכיל נתונים גאומרחביים מלאים (altitude, longitude, latitude), מדדי תנועה וחומרה מהשטח (stepDelta, accuracy, speed), וחותמת זמן אישית לצורך סכימה ועיבוד של מדדי ביצוע אנליטיים בסיום הריצה.

14. מדריך למשתמש

מדריך זה מציג את תהליך העבודה (Workflow) של המשתמש באפליקציה מקצה לקצה. המדריך מנוסח בצורה חווייתית וברורה, ומלווה את המתאמן בכל שלב – החל מיצירת החשבון הראשונית, דרך תכנון המסלול החכם באמצעות השרת, ועד לניהול הריצה הפעילה וניתוח המדדים בסיומה.

שלב 1: הצטרפות וכניסה למערכת (הרשמה והתחברות)

בעת פתיחת האפליקציה לראשונה, תופנה אל מסך ההתחברות. אם כבר יש ברשותך חשבון, תוכל להיכנס מיד; אחרת, תוכל לעבור בקלות למסך ההרשמה.

• כיצד להירשם במערכת?

1. לחץ על כפתור הטקסט בתחתית המסך: **"Don't have an account? Sign Up"**.
2. במסך ההרשמה שנפתח, הזן את פרטיך הבסיסיים: שם משתמש, כתובת אימייל וסיסמה מאובטחת (ניתן ללחוץ על אייקון העין כדי לוודא שהסיסמה הוקלדה נכון).
3. הזן את הנתונים הפיזיולוגיים שלך: **משקל** (ביחידות קילוגרם) ו**גובה** (ביחידות סנטימטר). נתונים אלו חיוניים לצורך חישוב מדויק של שריפת קלוריות וניתוח מדדי ריצה בהמשך.
4. לחץ על שדה **"Birth Date"** – חלון לוח שנה קופץ (DatePicker) יופיע על המסך. בחר את תאריך הלידה שלך ולחץ על אישור.
5. לסיום, לחץ על הכפתור הכחול הרחב **"Sign Up"**. עם השלמת הרישום בהצלחה, החשבון שלך יישמר בענן ותועבר למסך הבית.

• כיצד להתחבר לחשבון קיים?

1. במסך הבית, הזן את כתובת האימייל והסיסמה שאיתם נרשמת.
2. לחץ על כפתור "Login". במידה והפרטים נכונים, המערכת תאשר את כניסתך ותטען את הפרופיל האישי שלך ברקע.

שלב 2: הגדרת אילוצים ותכנון מסלול אימון חכם

לאחר הכניסה, תגיע למסך הראשי הכולל את כפתור ה-"START recording a new run". לחיצה עליו תוביל אותך אל מסך הגדרת הריצה המותאמת אישית, המהווה את הלב החכם של האפליקציה.

1. **בחירת סוג האימון:** ודא כי מתג "**מסלול מותאם אישית**" פעיל (Checked). אם תכבה אותו, האפליקציה תעבור למצב "מעקב חופשי" ללא תכנון מראש.
2. **קביעת מגבלת המרחק:** גרור את סרגל הבחירה (Slider) של המרחק המבוקש. תוכל לקבוע כל מרחק בטווח של בין 1.0 ק"מ ל-25.0 ק"מ, בקפיצות של חצי קילומטר. שרת התוכנה מחויב לייצר עבורך מסלול שיעמוד בדיוק במרחק הפיזי שקבעת בסרגל זה.
3. **קביעת מגבלת הפרש גבהים:** אם אתה מעוניין בריצה מישורית או לחלופין בריצת טיפוס מאתגרת, גרור את סרגל הגבהים לקביעת הפרש הגבהים המקסימלי (במטרים) המותר במסלול.
4. **בחירת העדפות אזור ונקודות עניין:** תחת כותרת "העדפות מסלול", לחץ וסמן את התגיות (Chips) של המקומות שתרצה לשלב בנתיב הריצה שלך. תוכל לבחור שטחים ירוקים (פארקים, גינות), מתחמי ספורט (אצטדיונים), או נקודות שירות ועניין (בתי קפה, חנויות נוחות). האלגוריתם בשרת ישקלל את הבחירות שלך וימשוך את תוואי הדרך לעבר האזורים הללו.
5. **בחירת תוואי המסלול (מעגלי או קווי):** קבע באמצעות המתג התחתון האם ברצונך לקבל "**מסלול מעגלי**" (האלגוריתם יתכנן לולאה שלמה שתחזיר אותך בדיוק לנקודה ממנה התחלת) או מסלול קווי מנקודה לנקודה.
6. **שליחה ועיבוד:** לאחר שסיימת להגדיר את האילוצים, לחץ על הכפתור הגדול "**צא לדרך!**".

שלב 3: תצוגה מקדימה וסקירת המסלול המוצע

מיד עם לחיצה על "צא לדרך", השרת יריץ את אלגוריתם האופטימיזציה ויקפיץ על גבי מסך המפה חלונית סקירה תחתית (Bottom Sheet).

• מה ניתן לראות בחלונית התצוגה המקדימה?

- **מפה דינמית:** על גבי המפה ישורטט קו נתיב רציף וכחול המציג את תוואי הריצה המדויק שנבחר עבורך, לצד סימון נקודות העניין והפארקים שנכללו במסלול.
- **אורך כולל ריאלי:** הטקסט יציג את אורכו הממשי של המסלול (לדוגמה: "5.2 ק"מ"), כדי שתוכל לוודא שהוא אכן עונה על הציפיות שלך.

- **הפרש גבהים מתוכנן:** יוצג סך העליות הטופוגרפיות המצטברות שתעבור לאורך הדרך המתוכננת.
- **אישור ויציאה:** במידה והמסלול מתאים לך, לחץ על הכפתור הכחול "**התחל פעילות**" (הנושא אייקון של Play) כדי להתחיל את המעקב בשטח ולצאת לריצה.

שלב 4: ניטור הריצה בזמן אמת

המסך הבא מלווה אותך באופן אקטיבי במהלך הריצה הפיזית בשטח. המערכת מציגה חזותית את התקדמותך ומוודדת את ביצועיך.

- **כיצד לעקוב אחר המדדים בזמן הריצה?**
 - **רכיב המפה העליון:** מציג את מיקומך הנוכחי ברישום חי, ומשרטט בזמן אמת את קו ההתקדמות הפיזי שביצעת בפועל על גבי תוואי המסלול המתוכנן.
 - **שעון העצר (TIME):** מציג את זמן הריצה הנטו שלך בפורמט שעות, דקות ושניות. השעון סופר רק את הזמן שבו רצת בפועל.
 - **מד מרחק (DISTANCE):** מציג בדיוק גבוה את המרחק המצטבר שעברת עד כה במטרים, תוך סינון אוטומטי של סטיות ורעשי קליטה של רכיב המיקום.
- **שליטה במצבי הריצה:**
 - **ביצוע הפסקה (Pause):** אם ברצונך לעצור למנוחה או להמתין ברמזור, לחץ על הכפתור העגול הכתום "**PAUSE**". עם הלחיצה, שעון העצר וחישובי המרחק יוקפאו לחלוטין, והכפתור ישתנה למצב "**RESUME**". לחיצה מחדש על "**RESUME**" תחזיר את המערכת למצב מדידה אקטיבי.
 - **סיום האימון (Stop):** ברגע שהגעת לסוף המסלול וסיימת את הריצה, לחץ על הכפתור העגול האדום "**STOP**".

שלב 5: צפייה בסיכום האימון ובדשבורד האנליטי

מיד לאחר אישור סיום הריצה (או בעת לחיצה על אימון עבר מתוך מסך היסטוריית הריצות), ייפתח בפניך דשבורד סיכום אימון עשיר, המציג ניתוח מתקדם של ביצועיך: [כאן יש להדביק צילום מסך של מסך סיכום האימון והגרפים האנליטיים]

1. **כרטיסיית מדדים מספריים:** בחלק העליון תוכל לצפות בסיכום מרוכז של ארבעה מדדי מפתח:

- **DISTANCE:** המרחק הסופי המדויק שגמעת (בקילומטרים).
- **TOTAL TIME:** משך הפעילות הכולל נטו (בקיזוז כלל ההפסקות שביצעת).
- **AVG PACE:** הקצב הממוצע של הריצה, המיוצג בדקות שנדרשו לך כדי לעבור קילומטר בודד.

○ **EST. CALORIES:** אומדן שריפת האנרגיה והקלוריות שלך (Kcal), המחושב בצד האפליקציה על ידי שילוב של משך הריצה נטו יחד עם משקל הגוף המעודכן בפרופיל שלך.

2. **ניתוח פרופיל גובה (Elevation Profile):** גרף קו דינמי המציג ויזואלית את שינויי הגובה הטופוגרפיים (במטרים) שעברת לאורך ציר מרחק הריצה. הגרף מאפשר לך לנתח את רגעי הטיפוס והירידה באימון.

3. **ניתוח מהירויות (Speed Analysis):** גרף קו רציף המציג את שינויי מהירות התנועה שלך (בקילומטרים לשעה) לאורך שלבי האימון, ומאפשר לך לזהות נקודות מאמץ, האצות או נפילות קצב.

שלב 6: דפדוף בהיסטוריית האימונים וניהול הפרופיל

בכל שלב, תוכל להשתמש בסרגל הניווט התחתון של האפליקציה כדי לעבור בין הפרגמנטים השונים:

- **צפייה ביומן הריצות (Run History):** מעבר לפרגמנט ההיסטוריה יציג בפניך רשימה סדורה וכרונולוגית של כלל הריצות שביצעת ושמרת בענן. לחיצה על כל כרטיסיית ריצה ברשימה תפתח מחדש את דשבורד האנליטיקה המלא של אותו אימון ספציפי.
- **צפייה בנתוני הפרופיל (Account Profile):** מעבר לפרגמנט הפרופיל יציג כרטיסיית מידע לבנה ואסתטית המציגה את שמך, כתובת המייל שלך, והאות הראשונה של שמך המופיעה בתוך אווטאר כחול עיגולי. הכרטיסייה מציגה את נתוני הגובה, המשקל ותאריך הלידה שלך, לצד ציון תאריך ההצטרפות הרשמי שלך לאפליקציה (**Member Since**).
- **התנתקות מהמערכת (Log Out):** בתחתית מסך הפרופיל קיים כפתור אדום מותווך בשם "Log Out". לחיצה עליו תקפיץ דיאלוג אישור, המאפשר לך להתנתק בצורה מאובטחת מהחשבון הנוכחי ולהחזיר את האפליקציה למסך ההזדהות הראשי.

15. בדיקות והערכה

תהליך הבדיקה והערכת המערכת היווה נדבך קריטי במחזור חיי הפיתוח, ומטרתו הייתה להבטיח את תקינות הרכיבים, יציבות התקשורת ואיכות המסלולים המופקים. הבדיקות חולקו למספר שלבים מתודולוגיים:

- **בדיקות יחידה (Unit Testing):** בשלב זה נבחנו רכיבי הלוגיקה המבודדים בצד השרת כדי לוודא את נכונותם הלוגית והמתמטית. הבדיקות התמקדו באימות פונקציות העזר הגאודזיות, דוגמת חישוב מרחקים מדויק בין קואורדינטות, ומנגנוני שקלול ציוני התגמול עבור נקודות העניין בהתאם למדדי הפופולריות שלהן.

- **בדיקות שילוב (Integration Testing):** שלב זה התרכז בבחינת זרימת המידע וסנכרון הנתונים בין שכבות המערכת השונות. נבדקה אמינות התקשורת האסינכרונית בין אפליקציית המובייל לשרת הליבה, תקינות שאילתות השליפה והכתיבה מול מסד הנתונים, וכן נכונות חילוף המידע והתרגום של המענים המתקבלים משירותי המיפוי החיצוניים.
- **בדיקות מערכת ועומסים (Stress Testing & System):** בדיקות אלו נועדו לבחון את ביצועי מנוע האופטימיזציה תחת תרחישי קיצון. נבדקה יכולת ההתכנסות של האלגוריתם המרכזי כאשר המשתמש מגדיר אילוצי מרחק גדולים במיוחד, המייצרים רשת קשרים עמוסה ורחבה. הבדיקות אישרו כי השרת מצליח לשמור על זמני תגובה מהירים ויציבים, ללא חריגה ממכסות הזיכרון או יצירת השבתה.
- **בדיקות שטח והערכת משתמש (User Evaluation & Field):** הבדיקות המסכמות בוצעו בסביבה האמיתית מחוץ למעבדה על ידי הרצת האפליקציה תוך כדי תנועה פיזית בשטח. בשלב זה נבחנה יעילותם של מסנני החומרה בניקוי רעשי קליטה וסטיות של רכיב המיקום, ואומתה חסינות המערכת במצבים של ניתוקי רשת זמניים והצגה חזותית נכונה של תוואי הדרך.

16. מסקנות

פיתוח המערכת והפעלתה הניבו מספר מסקנות הנדסיות ומעשיות מרכזיות בנוגע לארכיטקטורת תוכנה ולפתרון בעיות ניתוב בזמן אמת:

- **יעילות גישות מטא-היוריסטיות בשרת רשת:** המחקר והמימוש הוכיחו כי פתרון בעיות אופטימיזציה מורכבות תחת מגבלות קשיחות אינו מחייב זמני עיבוד ארוכים ומתישים. הבחירה באלגוריתם מבוסס אינטליגנציית נחיל, בשילוב מנגנוני שיפור גאומטריים מקומיים, מאפשרת להגיע לרמת דיוק גבוהה ולמסלולים איכותיים ומגוונים בתוך שניות, דבר ההופך את הגישה לשימה לחלוטין עבור שירותי רשת דינמיים.
- **חשיבות שלב עיבוד מקדים ותיקון נתונים:** אחת המסקנות הבולטות היא שלא ניתן להסתמך על נתוני עולם אמיתי (הן מחיישי המכשיר והן משירותי מפה) בצורתם הגולמית. ללא שלב הכנה קפדני המצמיד נקודות לרשת שבילים ריאלית, מסנן רעשי חומרה ומתקן את חוקיות המסלול באופן אקטיבי כדי להבטיח את סגירת הלולאה המעגלית, המסלולים המופקים עלולים להיות פגומים מבחינה מבנית ולא בטיחותיים לריצה.
- **עדיפות לארכיטקטורה מבוצרת ומנותקת (Decoupled Architecture):** החלוקה הברורה בין לקוח קל הממוקד בניהול ממשק משתמש ואיסוף נתוני חיישנים, לבין שרת חזק המרכז את כלל הלוגיקה החישובית והאלגוריתמית, הוכחה כנכונה ארכיטקטונית. הפרדה זו מאפשרת לשמור על חוויית משתמש חלקה ורציפה באפליקציה, ומותירה את העבודה הנדסית הכבדה לסביבת ענן המסוגלת להתמודד איתה ביעילות.

17. פיתוחים עתידיים

במטרה להרחיב את יכולות המערכת ולשפר את מעטפת השירות עבור המתאמנים, הוגדרו מספר כיווני פיתוח והרחבה עתידיים:

- **מנגנון חישוב מסלול מחדש דינמי ובזמן אמת (Dynamic Rerouting Engine):** הרחבת רכיב הניטור בצד הלקוח כך שיזהה באופן אוטומטי מצבים שבהם הרץ חורג מתוואי הדרך המקורי שהוצע לו. במקרה של סטייה מכוונת או אילוץ בשטח, האפליקציה תתקשר מול השרת כדי להפעיל מחדש את מנוע האופטימיזציה מנקודת המיקום החדשה, ותפיק נתיב חלופי מתוקן שיעמוד ביתרת תקציב המרחק שנותר לאימון.
- **אינטגרציה עם רכיבי קצה ומכשירים לבישים (Wearable Devices Integration):** התאמת ממשק המשתמש ומערכת איסוף הנתונים לעבודה מול שעונים חכמים וצמידי כושר (כגון שעוני אנדרואיד וציוד ניטור ספורט). פיתוח זה יאפשר למשתמש לצפות בהנחיות הניווט ובנתוני המרחק ישירות מפרק כף היד, ויעשיר את שכבת האנליטיקה באמצעות קליטה דינמית של מדדים פיזיולוגיים מורחבים, דוגמת דופק וקצב נשימה.
- **התאמה אישית ולמידת העדפות משתמש (Machine Learning Personalization):** שילוב מודלים של למידת מכונה בשכבת השרת שינתחו את היסטוריית האימונים והתנהגות הרץ לאורך זמן. המערכת תלמד לזהות באופן אוטומטי אילו סוגי מסלולים המשתמש מעדיף (למשל, העדפה למסלולים מישוריים, נתיבים מרובי פארקים ושטחים ירוקים, או לחלופין ריצה עירונית), ותשנה דינמית את משקולות התגמול באלגוריתם כדי לתעדף אזורים אלו בסבבי ההפקה הבאים.
- **סנכרון נתוני סביבה, אקלים וטופוגרפיה (Environmental Aware Routing):** חיבור מנוע העיבוד הגאוגרפי אל ממשקי תכנות חיצוניים המספקים נתוני מזג אוויר, זיהום אוויר ועומסי חום בזמן אמת. פיתוח זה יאפשר למערכת לשנות את תוואי הדרך המוצע בהתאם לתנאי הסביבה הנוכחיים, למשל על ידי בחירת מסלולים מוצלים יותר בימים חמים או הימנעות מאזורים בעלי רמת זיהום אוויר גבוהה, לטובת שמירה על בריאות המתאמן.

18. ביבליוגרפיה

JSON: <https://he.wikipedia.org/wiki/JSON>

HTTP: https://he.wikipedia.org/wiki/Hypertext_Transfer_Protocol

TCP/IP: <https://he.wikipedia.org/wiki/TCP/IP>

:Orienteering

https://www.researchgate.net/publication/309124071_The_orienteering_problem